Guía de estudio: el código C# del sistema escolar

Oct 13, 2026 · @juan carlos bodoque III

1. Cómo usar esta guía y mapa del código

Esta guía es la hermana de la guía de la base de datos y sigue la misma estructura: primero los fundamentos, luego cada pieza explicada con código real, después tres flujos de punta a punta, un plan de repaso y, al final, preguntas con respuestas largas y un glosario (sección 18) con las palabras técnicas como auditoría, trigger, hash o transacción. La diferencia es el punto de vista. En la otra guía aprendiste qué decide y qué guarda la base. Aquí aprendes qué hace el C#: cómo le habla a SQL, cómo organiza la lógica de cada pantalla y cómo dibuja el diseño que hace reconocible al proyecto.

La idea que conviene que lleves a la defensa es esta: **el C# no guarda nada ni decide las reglas de fondo; pide, valida lo cómodo, pinta y reacciona**. Guardar y garantizar lo hace SQL Server. Pero el C# es lo que el usuario ve y toca, y por eso es donde más se nota el trabajo del equipo. Si dominas tres cosas, lo dominas casi todo: (1) cómo se escribe y se manda una consulta SQL desde C#, (2) cómo se organiza una pantalla (carga, filtro, validación, guardado) y (3) cómo se dibujan los controles propios.

Qué contiene el código

Pieza	Dato	Para qué sirve
Proyectos	2	`Modelos` (biblioteca de clases: datos, servicios, exportadores) y `Vistass` (la aplicación de Windows con las pantallas). Ambos en .NET Framework 4.8
Código escrito a mano	~17.600 líneas en 65 archivos	La lógica real: aquí se estudia
Código de diseñador	~14.400 líneas en 30 archivos `.Designer.cs`	Lo genera Visual Studio cuando arrastras controles a un formulario. No se estudia línea por línea, solo se reconoce

Pieza	Dato	Para qué sirve
Paquetes	4 que importan	`System.Data.SqlClient` (hablar con SQL Server), `BCrypt.Net-Next` (contraseñas), `iTextSharp` (PDF) y `System.Windows.Forms.DataVisualization` (los gráficos del inicio)

Los archivos que debes reconocer

No hace falta memorizar los 65. Estos son los que sostienen todo lo demás, con la sección de esta guía donde se explican.

Archivo	Qué hace	Sección
`Program.cs`	Punto de entrada: decide si toca el asistente de primer uso, el login o el panel	7 y 10
`Conexion.cs`	Arma la conexión a SQL Server y le pone la etiqueta del usuario	4
`Datos.cs`	Los cinco métodos con los que todo el programa consulta y escribe	3 y 4
`Sesion.cs`	Guarda en memoria quién inició sesión y con qué rol	7
`ServicioUsuarios.cs`	Administra usuarios, claves temporales y reglas de contraseña	7
`PasswordHasher.cs`	Convierte y verifica contraseñas con BCrypt	7
`ServicioCorreo.cs`	Envía por correo los códigos de recuperación y los reportes de soporte	8
`ExportadorExcel.cs`, `ExportadorPdf.cs`, `DocumentosPdf.cs`	Generan los archivos que se exportan	8
`frmLogin`, `frmRecuperarContraseña`, `frmConfiguracionInicial`	Las tres puertas de entrada al sistema	7

Archivo	Qué hace	Sección
`frmDashboard`	El panel principal con el menú lateral y la navegación	6 y 9
`frmInicio`	Los indicadores y gráficos de la pantalla de inicio	3
`frmGestionPersonal`, `frmAgregarPersonal` y las demás `frmGestion...`	Las pantallas de cada módulo	6
`ControlesTema.cs`, `ItemMenuLateral.cs`, `FechaOscura.cs`	Los controles que se dibujan por código	9

Las capas, de arriba hacia abajo

Una forma de ordenarlo en la cabeza es pensar en cuatro capas. Arriba están las **pantallas** (`frmXxx`), que reciben clics y muestran datos. Debajo, los **servicios y el acceso a datos** (`Datos`, `Conexion`, `ServicioUsuarios`…), que saben hablar con la base y con el exterior. A un lado, la **capa de dibujo** (los controles personalizados), que no tiene lógica de negocio pero sí mucho código: es lo que le da identidad visual al sistema. Y al fondo, **SQL Server**, donde viven las reglas y los datos. Una pantalla casi nunca habla con SQL Server directamente: pasa por `Datos` (con algunas pocas excepciones que se explican en la sección 4).

Cómo leer cualquier pantalla

Todas las pantallas del sistema siguen el mismo ritmo, y por eso aprender una es aprender todas. Cuando abras un archivo `frmXxx.cs`, hazte tres preguntas en este orden. Primero: **¿qué consulta carga los datos?**, normalmente en el evento `Load` o en un método `Cargar...`. Segundo: **¿qué evento dispara cada acción?**, un clic de botón, un cambio en un cuadro de búsqueda, una selección en la tabla. Tercero: **¿qué se valida antes de guardar y a quién se le delega el guardado?**, es decir, si la pantalla lo guarda ella misma o llama a un procedimiento. Con esas tres respuestas puedes explicar cualquier formulario sin haberlo memorizado.

2. Fundamentos de C# y WinForms que necesitas dominar

Antes de ver SQL y pantallas conviene afirmar los conceptos de C# que el proyecto usa una y otra vez. No es un curso de C#: es la lista de ideas que, si las entiendes, hacen que el resto del código deje de verse como texto y empiece a leerse como una historia. Cada una viene con el ejemplo del propio proyecto donde la puedes señalar.

2.1 Clases, objetos y `static`

Una clase es un molde y un objeto es una pieza hecha con ese molde. `ItemMenuLateral` es un molde: cada opción del menú lateral (Inicio, Activos, Personal…) es un objeto distinto con su propio texto, su propio icono y su propio estado. Pero hay clases que no sirven de molde porque no hace falta hacer copias: `Datos`, `Conexion`, `Sesion` y `PasswordHasher` son clases `static`, un conjunto de herramientas y valores compartidos por todo el programa. No haces `new Datos()`, escribes directamente `Datos.Tabla(...)`.

Esa diferencia explica una decisión de diseño. `Sesion` es `static` porque solo puede haber un usuario con sesión a la vez en el programa, y cualquier pantalla necesita saber quién es: `Sesion.Usuario`, `Sesion.Rol`, `Sesion.NombreCompleto`. En cambio cada formulario de empleados es un objeto nuevo, porque puedes abrir el de «Nuevo empleado» y después el de «Editar empleado» con datos distintos.

2.2 Propiedades, campos y modificadores de acceso

Un campo es una variable que vive dentro del objeto; una propiedad es la puerta controlada para leerlo o cambiarlo. Mira cómo lo usa `ItemMenuLateral`:

```
private bool activo = false;

public bool Activo
{
    get { return activo; }
    set { if (activo != value) { activo = value; Invalidate(); } }
}
```

El campo `activo` es privado: nadie de afuera lo toca. La propiedad `Activo` es la única forma de cambiarlo, y su `set` hace algo más que guardar el valor: si cambió, llama a `Invalidate()`, que le pide a Windows que vuelva a dibujar el control. Así el menú se repinta solo cuando algo cambia, y no antes ni de más. Ese patrón (cambiar un estado y pedir un repintado) se repite en casi todos los controles de diseño.

Los modificadores marcan quién puede ver qué. `public` lo ve cualquiera; `private` solo la propia clase; `internal` solo el mismo proyecto (por eso `IconosMenu` y `BarraFina` son `internal`: son piezas de apoyo que no deben usarse desde fuera). `const` es un valor fijo que se decide al compilar (`MaxFallos = 5`), y `static readonly` es un valor compartido que se asigna una vez (los colores de `PaletaSistema`).

2.3 Herencia, `override` y clases `partial`

Una clase puede heredar de otra y quedarse con todo lo que ella sabe hacer. `BotonTema : Button` es un botón normal de Windows al que se le añade la paleta del sistema; `GridOscuro : DataGridView` es una tabla normal con colores y comportamiento propios; `ItemMenuLateral : Control` parte de un control vacío y lo dibuja todo él mismo. Heredar ahorra escribir de nuevo lo que ya existe.

Para cambiar el comportamiento heredado se usa `override`. Cuando un control debe dibujarse, Windows llama a su método `OnPaint`; si lo sobrescribes con `protected override void OnPaint(PaintEventArgs e)`, tú decides qué aparece. Y `base.OnPaint(e)` significa «haz primero lo que haría la clase de la que heredo». Con `PanelTarjeta` ves ambos: llama a `base.OnPaint(e)` y después dibuja encima un borde fino.

La palabra `partial` aparece en todos los formularios (`public partial class frmLogin : Form`). Significa que la clase está repartida en dos archivos: `frmLogin.cs`, que escribes tú, y `frmLogin.Designer.cs`, que escribe Visual Studio cuando arrastras controles en el diseñador. El método `InitializeComponent()` que ves al inicio de cada constructor vive en el segundo y es quien crea los botones y cuadros de texto. Los dos archivos son una sola clase.

2.4 El ciclo de vida de un formulario

Un formulario nace, se muestra, responde a eventos y muere, y el código se organiza alrededor de ese ciclo. El constructor llama a `InitializeComponent()`. Después se dispara el evento `Load`, que es el lugar natural para cargar datos y configurar controles (por eso casi todas las pantallas tienen un `frmXxx_Load`). Luego la pantalla vive, esperando clics. Al cerrarse se dispara `FormClosing` (aquí `frmDashboard` detiene sus temporizadores) y después `FormClosed`.

Hay dos maneras de abrir un formulario y no son lo mismo. `Show()` lo abre y sigue ejecutando la línea siguiente. `ShowDialog()` lo abre como ventana modal: la ejecución se detiene ahí hasta que se cierre, y devuelve un `DialogResult` que dice cómo se cerró. Así

trabajan los formularios de captura:

```
frmAgregarPersonal frm = new frmAgregarPersonal();
if (frm.ShowDialog(this) == DialogResult.OK) MostrarEmpleados();
```

La pantalla de personal abre el formulario de alta y se queda esperando. Si el formulario se cierra con `DialogResult.OK` (porque guardó bien), recarga la lista; si se cancela, no hace nada. Es una conversación entre dos pantallas con una sola respuesta.

2.5 Eventos, manejadores y delegados

Windows Forms funciona por eventos: algo ocurre (un clic, una tecla, un cambio de texto) y el programa reacciona. Un manejador de evento es un método con una firma fija, `(object sender, EventArgs e)`, que se conecta a un evento con `+=`:

```
timerMenu.Tick += timerMenu_Tick;
timerInsignias.Tick += (s, ev) => ActualizarInsignias();
```

La primera línea conecta un método con nombre. La segunda usa una expresión lambda (`(s, ev) => ...`), un método pequeño sin nombre escrito en el mismo sitio: es lo mismo, más corto. El parámetro `sender` es el control que disparó el evento, y por eso una misma función (como `Campo_Paint` del login) puede servir para varios controles y distinguir cuál la llamó.

Detrás de los eventos están los delegados: variables que guardan una función. `Func<Form>` es «una función que no recibe nada y devuelve un formulario», y `Action` es «una función que no devuelve nada». El menú lateral los usa así: `Navegar(itmPersonal, () => new frmGestionPersonal())` le pasa a `Navegar` una receta para crear la pantalla, no la pantalla ya creada. La pantalla se crea solo en el momento en que hace falta y adentro de un `try/catch`. Es una forma elegante de decir «haz esto, pero con cuidado».

2.6 `using` y la liberación de recursos

Algunos objetos retienen recursos que no son solo memoria: una conexión abierta a la base, un pincel de dibujo, un archivo. Se les llama `IDisposable` y hay que liberarlos. `using` lo hace automáticamente:

```
using (SqlConnection con = Conexion.Conectar())
using (SqlCommand cmd = new SqlCommand(sql, con))
{
    con.Open();
    return cmd.ExecuteScalar();
}
```

Al salir del bloque, aunque haya ocurrido un error, la conexión se cierra y el comando se libera. Es el equivalente a un `try/finally` que no puedes olvidar. En todo el proyecto verás `using` con conexiones y con objetos de dibujo (`using (Pen p = new Pen(color))`, `using (GraphicsPath ruta = ...)`). La razón es seria: una conexión que no se cierra se acumula y, tras unas cuantas, la base deja de aceptar más.

2.7 Excepciones: `try`, `catch`, `finally` y `throw`

Una excepción es un aviso de que algo salió mal. Si nadie la atrapa, el programa se cierra. `try` envuelve lo que puede fallar y `catch` decide qué hacer. Se pueden poner varios `catch` de lo más específico a lo más general: `catch (SqlException ex)` para errores de la base y después `catch (Exception ex)` para todo lo demás, como se ve en `frmAgregarPersonal`. `finally` se ejecuta siempre, haya o no error: el login lo usa para volver a habilitar el botón «Iniciar sesión» pase lo que pase.

La instrucción `throw;` sin nada más significa «vuelve a lanzar la misma excepción que atrapé»: se usa tras deshacer una transacción para no esconder el error. Hay también un `catch { }` vacío que se ve en varios lugares, por ejemplo al registrar un evento de auditoría. Es una decisión consciente: «si esto falla, que no impida seguir». Pero conviene usarlo solo para cosas secundarias, nunca para el guardado de datos.

2.8 Tipos, valores nulos y conversiones

La base de datos tiene `NULL`; C# tiene `null`; y entre las dos hay un tercer valor, `DBNull.Value`, que es cómo ADO.NET representa un `NULL` de SQL. Por eso verás tanto este patrón:

```
txtTelefono.Text = reader["Telefono"] != DBNull.Value ?
reader["Telefono"].ToString() : "";
```

Si la columna es nula, mostrar `.ToString()` fallaría o daría un texto raro; se pregunta primero. Y al revés, para guardar un valor opcional hay que mandar `DBNull.Value` y no `null`: de eso se encarga `Datos.P`, con el operador `??` («si es null, usa esto otro»): `valor`

`?? DBNull.Value`.

Las conversiones se hacen con `Convert.ToInt32(...)`, `Convert.ToDecimal(...)`, `Convert.ToDateTime(...)`, porque los valores que salen de una tabla llegan como `object`. Los tipos que admiten nulos se escriben con signo de interrogación: `bool?` puede ser verdadero, falso o «sin dato», y es lo que usa la preferencia del menú plegado para distinguir «el usuario eligió plegar» de «nunca ha elegido nada».

2.9 Colecciones y LINQ

Cuatro colecciones aparecen constantemente. `List<T>` es una lista que crece: las condiciones de un filtro se acumulan en una `List<string>` y luego se unen. `Dictionary<K,V>` relaciona una clave con un valor: el login guarda los intentos fallidos como `Dictionary<string,int>`, de usuario a conteo. `HashSet<T>` es un conjunto sin repetidos y con búsqueda rápida: el panel lo usa para saber qué opciones del menú están ocultas. Y los arreglos `T[]`, como la lista de opciones del menú.

LINQ son métodos para consultar colecciones con una sola línea: `.Where(i => !ocultas.Contains(i)).ToList()` se queda con los ítems que cumplen la condición; `.Cast<Control>().ToList()` convierte los controles de un panel en una lista de la que se puede borrar mientras se recorre. `string.Join(" AND ", cond)` pega una lista de textos con un separador y es la pieza con la que se arman los filtros.

2.10 Cadenas de texto

Hay tres formas de escribir texto que importan aquí. La normal, `"hola"`. La **literal** con arroba, `@"..."`, que respeta los saltos de línea y no interpreta las barras invertidas: es la que se usa para las consultas SQL largas, porque permite escribirlas en varias líneas legibles. Y la **interpolada**, `$"user{idEmpleado}"`, que inserta valores dentro del texto. Para trabajar con ellas: `Trim()` quita espacios de los extremos, `Split()` corta el texto en partes, `Replace()` cambia fragmentos y `ToLower()` pasa a minúsculas. En `GenerarCredenciales` los cuatro trabajan juntos para quitar tildes y armar un nombre de usuario.

2.11 Hilos, `async` y el hilo de la pantalla

Windows Forms dibuja y responde en un único hilo, el de la interfaz. Si una operación tarda (enviar un correo por internet), ese hilo se bloquea y la ventana se congela. Para evitarlo, `ServicioCorreo.EnviarCodigoAsync` devuelve un `Task` y hace el trabajo pesado con `Task.Run`, y la pantalla de recuperación lo espera con `await` dentro de un método

marcado `async`. Mientras tanto, la ventana sigue viva y el cursor muestra el reloj de espera. Cuando necesitas pedirle al hilo de la interfaz que haga algo desde otro lado, se usa `BeginInvoke`: `GridOscuro` lo usa para programar la actualización de sus barras de desplazamiento sin interrumpir el repintado que está ocurriendo.

3. SQL dentro de C#: cómo se escribe, cómo se manda y qué significa cada parte

Esta es la sección más importante de la guía, porque es el punto donde la base de datos y el C# se encuentran. La idea central es sencilla y conviene fijarla antes de ver código: **para C#, una consulta SQL es solo un texto**. C# no sabe SQL. Arma una cadena de caracteres, se la entrega a SQL Server junto con los valores que necesita, y SQL Server responde con una tabla o con un número. Todo lo demás (los `JOIN`, los filtros, los cálculos) lo resuelve la base. Por eso, para entender cómo «vive» la base de datos dentro del programa, hay que mirar cuatro piezas que aparecen juntas en casi cada consulta: el **texto SQL**, los **parámetros**, el **método que lo ejecuta** y el **resultado** que vuelve.

3.1 El texto: la cadena literal con arroba

Las consultas largas se escriben así, abriendo la cadena con `@"` y cerrándola con `"`:

```
string query = @"SELECT COUNT(*)
    FROM Empleados e
    INNER JOIN Cargos c ON e.IdCargo = c.IdCargo
    WHERE LOWER(c.NombreCargo) LIKE '%director%'
      AND LOWER(c.NombreCargo) NOT LIKE '%sub%'
      AND e.IdEmpleado <> @idEmpleadoExcluido";
```

La arroba delante de las comillas es una función de C#: hace que la cadena sea **literal**, es decir, que pueda ocupar varias líneas y que las barras invertidas no se interpreten como códigos especiales. Gracias a eso la consulta se escribe legible, con una cláusula por línea, igual que se vería en Management Studio.

Aquí hay una confusión muy común, así que señálala ahora: **hay dos arrobas que no tienen nada que ver entre sí**. La de afuera (`@"..."`) es de C# y habla de cómo se escribe el texto. La de adentro (`@idEmpleadoExcluido`) es de SQL y marca un **parámetro**, un hueco que se llenará después con un valor. Una es sintaxis de C#, la otra es sintaxis de SQL Server; coinciden por casualidad en el símbolo.

3.2 Primera consulta, desmontada: ¿ya existe un director?

Esta consulta vive en `frmAgregarPersonal`, dentro de un método que responde sí o no. Así se ve completo:

```
private bool YaExisteDirector(int idEmpleadoExcluido)
{
    using (SqlConnection conexion = Conexion.Conectar())
    {
        string query = @"SELECT COUNT(*)
            FROM Empleados e
            INNER JOIN Cargos c ON e.IdCargo = c.IdCargo
            WHERE LOWER(c.NombreCargo) LIKE '%director%'
              AND LOWER(c.NombreCargo) NOT LIKE '%sub%'
              AND e.IdEmpleado <> @idEmpleadoExcluido";

        SqlCommand cmd = new SqlCommand(query, conexion);
        cmd.Parameters.AddWithValue("@idEmpleadoExcluido",
idEmpleadoExcluido);

        conexion.Open();
        int count = Convert.ToInt32(cmd.ExecuteScalar());
        return count > 0;
    }
}
```

La pregunta de negocio es: «¿hay ya un empleado cuyo cargo sea Director, sin contar Subdirector, y que no sea el empleado que estoy editando?». La consulta se lee de arriba abajo así:

- **`SELECT COUNT(*)`**: «cuéntame cuántas filas cumplen todo lo que sigue». No pide datos de empleados, solo un número. Es la forma más barata de preguntar «¿existe?»: el resultado es una sola fila con una sola columna.
- **`FROM Empleados e`**: la tabla principal, con un **alias** corto (`e`). El alias evita escribir `Empleados.` cada vez y es obligatorio cuando una tabla aparece dos veces en la misma consulta.
- **`INNER JOIN Cargos c ON e.IdCargo = c.IdCargo`**: une cada empleado con su cargo. `ON` dice cómo se emparejan las filas: el `IdCargo` del empleado debe ser igual al `IdCargo` de la tabla de cargos (esa igualdad es la llave foránea de la que se habló en la guía de la base). Con `INNER`, solo quedan los empleados que sí tienen un cargo que existe.

- **`WHERE LOWER(c.NombreCargo) LIKE '%director%'`**: de esas filas unidas, quedáte solo con las que cumplan esto. `LOWER` pasa el texto a minúsculas para que «Director», «DIRECTOR» y «director» sean lo mismo. `LIKE` compara con un patrón, y el `%` es un comodín que significa «cualquier cantidad de caracteres»: `'%director%'` se cumple si la palabra director aparece en cualquier parte del texto.
- **`AND LOWER(c.NombreCargo) NOT LIKE '%sub%'`**: pero que no contenga «sub». Esta línea existe por un motivo muy concreto: «Subdirectora» también contiene la palabra «director». Sin esta exclusión, una subdirectora contaría como director y bloquearía al director de verdad.
- **`AND e.IdEmpleado <> @idEmpleadoExcluido`**: y que no sea el empleado excluido. `<>` significa «distinto de». Esto resuelve un problema sutil: cuando editas al propio director (para cambiarle el teléfono, por ejemplo), la consulta lo encontraría a él mismo y diría «ya existe un director». Al excluir su propio id, solo se detectan *otros* directores. Al crear un empleado nuevo se pasa `0`, que no coincide con ningún empleado real, así que no se excluye a nadie.

La parte de C# hace tres cosas en orden. Abre una conexión (con `using`, para que se cierre sola). Crea un `SqlCommand` con el texto y le agrega el parámetro con `AddWithValue`, que asocia el nombre `@idEmpleadoExcluido` con el valor que llegó al método. Y ejecuta con `ExecuteScalar()`, el método que devuelve **un solo valor** (la primera columna de la primera fila), que se convierte a entero con `Convert.ToInt32`. Si el conteo es mayor que cero, la respuesta es sí.

Y así se usa en la regla de negocio, dentro de `btnGuardar_Click`:

```
if (nombreCargo.ToLower().Contains("director") &&
!nombreCargo.ToLower().Contains("sub")
    && YaExisteDirector(idEmpleadoEditar))
{
    MessageBox.Show("Ya existe un Director registrado en el sistema.",
"Restricción de Cargo", ...);
    return;
}
```

Fíjate que el mismo criterio (contiene «director», no contiene «sub») se aplica en C# al cargo elegido y en SQL a los cargos de la tabla. Es la misma pregunta hecha en los dos lados.

Hay tres detalles honestos que conviene conocer de esta regla, porque son justo lo que podría preguntar alguien atento. Primero, **identifica los cargos por su nombre**, no por un número, así que depende de que el cargo se llame «Director». Segundo, la consulta **no filtra por estado**: cuenta también a un director dado de baja, de modo que mientras figure en la tabla bloqueará el alta de otro. Tercero, esta regla vive **solo en C#**; a diferencia de la mayoría de reglas del sistema, la base de datos no la garantiza, así que quien modificara datos directamente en SQL podría saltarla.

3.3 Segunda consulta, desmontada: la lista de personal

Esta es la consulta que llena la tabla de empleados. Es más larga, pero usa las mismas ideas con algunas más:

```
dtEmpleados = Datos.Tabla(
    @"SELECT e.IdEmpleado, e.Nombre, c.NombreCargo AS Cargo, est.NombreEstado
AS Estado,
             e.FechaContratacion, p.SalarioNeto AS [Salario Neto],
e.Telefono, e.Correo,
             e.IdCargo, e.IdPlanilla, p.SalarioBase AS [Salario Base],
             e.FechaNacimiento, e.EstadoCivil, e.Religion, e.Direccion
      FROM Empleados e
      INNER JOIN Cargos c ON e.IdCargo = c.IdCargo
      INNER JOIN EstadosEmpleado est ON e.IdEstado = est.IdEstado
      LEFT JOIN Planillas p ON e.IdPlanilla = p.IdPlanilla
      WHERE est.NombreEstado != 'Inactivo'
      ORDER BY e.Nombre");
```

Diferencia con la anterior: esta no devuelve un número sino una **tabla** (`Datos.Tabla`), así que pide columnas concretas en lugar de `COUNT(*)`. Cada pieza tiene su razón.

- **La lista de columnas** nombra cada dato con su tabla de origen (`e.Nombre`, `c.NombreCargo`). Con tres tablas unidas, indicar de cuál viene cada columna evita ambigüedades, porque varias tablas podrían tener una columna con el mismo nombre.
- **`AS`** le pone un nombre nuevo a la columna del resultado: `c.NombreCargo AS Cargo`. Este nombre, el **alias de columna**, es el que después usa el C#: `r["Cargo"]`. Es el contrato entre las dos partes. Si alguien lo cambia en SQL y no en C#, el programa compila bien pero falla al ejecutarse, porque C# busca una columna que ya no existe con ese nombre.

- **Los corchetes** de `[Salario Neto]` permiten que un nombre lleve un espacio. SQL Server entiende que lo que va entre corchetes es un solo nombre. Se hizo así para que el encabezado de la tabla del programa diga «Salario Neto» sin tener que renombrarlo después.
- **Tres `JOIN` de dos tipos.** `INNER JOIN Cargos` y `INNER JOIN EstadosEmpleado` exigen pareja: todo empleado debe tener cargo y estado. `LEFT JOIN Planillas` conserva a todos los empleados aunque no tengan una estructura salarial asignada; en ese caso las columnas de `p` salen vacías (`NULL`). La siguiente subsección lo ilustra con un ejemplo.
- **`WHERE est.NombreEstado != 'Inactivo'`** deja fuera a los dados de baja. `!=` y `<>` significan lo mismo: son dos formas de escribir «distinto». Así funciona la baja lógica del lado del programa: el empleado sigue en la base pero la lista no lo muestra.
- **`ORDER BY e.Nombre`** ordena alfabéticamente. Sin `ORDER BY`, SQL Server no garantiza ningún orden.

Un detalle que se aprende al leer el resto del código: la consulta trae columnas que la tabla no muestra (`IdCargo`, `IdPlanilla`, `Salario Base`, la fecha de nacimiento…). Se piden de todos modos porque la pantalla las necesita por detrás: `IdCargo` sirve para el filtro por cargo, y el resto lo aprovecha el expediente del empleado sin tener que volver a consultar. Una consulta hecha una vez y reutilizada vale más que cinco viajes a la base.

Después de la consulta, el flujo en C# es: `Datos.Tabla` devuelve un `DataTable`; ese `DataTable` se envuelve en un `DataView` (`vista = new DataView(dtEmpleados)`); y el `DataView` se asigna como `DataSource` de la tabla visual. Esa envoltura es lo que permite filtrar y buscar sin volver a la base, y se explica en la sección 4.

3.4 `INNER JOIN` y `LEFT JOIN`, con un ejemplo que se ve

La diferencia entre las dos uniones es qué pasa con las filas que no encuentran pareja. Imagina estos datos de ejemplo (inventados, solo para ilustrar):

Empleado	IdPlanilla
Ana	5
Luis	*(vacío)*

Si unes con la tabla de planillas usando `INNER JOIN`, solo sale Ana: Luis no tiene número de planilla y por tanto no encuentra pareja, así que desaparece del resultado. Con `LEFT JOIN`, salen los dos: Ana con su salario y Luis con el salario vacío. La palabra *left* (izquierda) se refiere a la tabla que se nombra primero, `Empleados`: de esa tabla se conservan todas las filas, y de la otra se trae lo que haya.

Por eso la lista de personal usa `LEFT JOIN Planillas`: un empleado recién creado sin estructura salarial debe aparecer en la lista, no esfumarse. La regla práctica para decidir es preguntarse «¿puede existir la fila principal sin su pareja?». Si no, `INNER JOIN`. Si sí, `LEFT JOIN`. Un matiz importante: la tabla de empleados permite que `IdCargo` sea nulo, pero esta pantalla usa `INNER JOIN Cargos`, así que un empleado sin cargo no aparecería en la lista. En la práctica no ocurre porque el formulario de alta exige elegir un cargo, pero es la clase de detalle que explica por qué elegir bien el tipo de unión importa.

3.5 Filtrar texto: `LOWER`, `LIKE` y los comodines

`LIKE` compara un texto con un patrón. Dos comodines hacen el trabajo: `%` reemplaza cualquier cantidad de caracteres (incluso ninguno) y `_` reemplaza exactamente uno. Los tres patrones que aparecen en el proyecto son:

- `LIKE 'docente%'`: el texto **empieza** con «docente». Es la forma en que el sistema reconoce los cargos de docentes.
- `LIKE '%director%'`: el texto **contiene** «director» en cualquier posición.
- `NOT LIKE '%sub%'`: el texto **no** contiene «sub».

Nota técnica que sirve para una pregunta: con la configuración por defecto, SQL Server ya ignora mayúsculas y minúsculas al comparar. El proyecto usa `LOWER(...)` de todos modos, como hábito defensivo. No hace daño, aunque no siempre es necesario. En cambio, un detalle que sí importa: envolver una columna en una función (`LOWER(c.NombreCargo)`) impide que SQL Server use un índice sobre esa columna. Para tablas pequeñas como estas no se nota; en tablas enormes habría que evitarlo.

Cuando el texto a buscar lo escribe el usuario, los comodines se agregan en C# y viajan dentro del parámetro, no dentro del SQL:

```
da.SelectCommand.Parameters.AddWithValue("@Criterio", "%" +
criterio.Trim().ToLower() + "%");
```

Y en la consulta aparece simplemente `LIKE @Criterio`. Es una buena práctica: el SQL queda fijo y el valor completo (con sus `%`) es lo único que cambia.

3.6 Parámetros: cómo viajan los valores y por qué es la base de la seguridad

Un parámetro es un hueco con nombre dentro del texto SQL (`@id`, `@nombre`) cuyo valor se envía por separado. Esto tiene tres consecuencias que justifican toda la técnica. La primera, **seguridad**: como el valor nunca se mezcla con el texto de la consulta, el motor lo trata siempre como un dato y jamás como una instrucción. Un nombre como `'; DROP TABLE Empleados; --` se guarda tal cual, como un nombre extraño. Esta defensa se llama prevención de inyección SQL y es el motivo por el que ninguna consulta de usuario en el sistema se arma pegando textos. La segunda, **tipos correctos**: una fecha viaja como `DateTime` y un número como `decimal`, de modo que no hay problemas con formatos (`dd/MM` contra `MM/dd`) ni con la coma o el punto decimal. La tercera, **rendimiento**: el motor reconoce que la consulta es la misma aunque cambie el valor y reutiliza su plan.

Hay tres maneras de crear un parámetro en el proyecto:

```
cmd.Parameters.AddWithValue("@id", idEmpleado);      // la forma directa de
ADO.NET
Datos.P("@id", idEmpleado);                           // el ayudante del
proyecto
Datos.PTexto("@codigo", txtCodigo.Text);             // para texto opcional
```

`Datos.P` crea el parámetro y, si el valor es `null`, lo reemplaza por `DBNull.Value` (el `NULL` de SQL). `Datos.PTexto` hace lo mismo y además recorta los espacios y convierte un texto vacío en `NULL`, que es lo que quieres para columnas opcionales como el código o el DUI: nunca se guarda una cadena vacía donde debería haber un vacío real. Y cuando el valor no es un tipo común, como una foto, se declara el tipo explícitamente: `cmd.Parameters.Add("@foto", SqlDbType.VarBinary, -1).Value = fotoActual;`.

Un mismo parámetro se puede usar varias veces en una consulta: en la revisión previa a dar de baja a un empleado, `@id` aparece cuatro veces dentro de cuatro subconsultas y solo se declara una vez.

Los parámetros tienen un límite: **sustituyen valores, no nombres ni listas**. No puedes poner el nombre de una tabla en un parámetro, ni una lista variable de números. Para listas de números, el proyecto concatena en un solo lugar y lo documenta:

```
// los ids son enteros, no hay riesgo de inyección
Datos.Tabla("SELECT NombreMateria FROM Materias WHERE IdMateria IN (" +
            string.Join(",", materiasSel) + ") ORDER BY NombreMateria");
```

Esto es seguro por una razón concreta: `materiasSel` es una `List<int>`, y un número entero no puede contener comillas ni instrucciones. Si esa lista fuera de textos, la concatenación sería peligrosa. La regla que puedes defender: **nunca se concatena texto del usuario; solo números enteros que el propio programa generó**.

3.7 Contar, sumar y agrupar: las consultas de los indicadores

La pantalla de inicio calcula sus tarjetas y gráficos con consultas que resumen datos. Es donde aparecen más ideas nuevas. Esta cuenta el equipo y cuánto está en mal estado:

```
SELECT COUNT(*) AS Total,
       ISNULL(SUM(CASE WHEN e.NombreEstado IN ('Malo','En
reparación','Mantenimiento Pendiente','Dado de baja')
                       THEN 1 ELSE 0 END), 0) AS Problema,
       (SELECT COUNT(*) FROM GestionActivos) AS Tipos
FROM UnidadesActivo u
JOIN Estados e ON e.IdEstado = u.IdEstado
```

La pieza clave es `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`, un truco para **contar solo lo que cumple una condición**. `CASE` es el «si… entonces… si no…» de SQL: aquí da 1 a cada unidad cuyo estado es problemático y 0 a las demás. Sumar esos unos y ceros da el número de unidades con problema, y todo en una sola pasada por la tabla. El `IN (...)` es una forma corta de escribir varias comparaciones con `OR`. `ISNULL(..., 0)` cubre un caso borde: si no hubiera ninguna unidad, `SUM` devolvería `NULL` en vez de 0, y el C# fallaría al convertirlo; `ISNULL` lo cambia por 0. Y `(SELECT COUNT(*) FROM GestionActivos) AS Tipos` es una **subconsulta** dentro del `SELECT`: una consulta pequeña que devuelve un solo valor y se trata como una columna más.

Para los gráficos aparecen `GROUP BY` y `TOP`:

```
SELECT ISNULL(c.NombreCategoria, 'Sin categoría') AS Etiqueta, COUNT(*) AS
Total
FROM UnidadesActivo u
JOIN GestionActivos a ON a.IdActivo = u.IdActivo
LEFT JOIN Categorias c ON c.IdCategoria = a.IdCategoria
GROUP BY c.NombreCategoria
ORDER BY COUNT(*) DESC
```

`GROUP BY` agrupa las filas que comparten un valor (la categoría) y `COUNT(*)` cuenta cuántas hay en cada grupo. `ORDER BY COUNT(*) DESC` pone primero la categoría con más equipos (`DESC` es de mayor a menor). Con `SELECT TOP 8 ...` se limitaría a las ocho

primeras filas, como hace el gráfico de salones.

El más ingenioso es el gráfico de reportes por mes, porque debe mostrar los últimos seis meses **incluso los que no tuvieron reportes**:

```
SELECT m.Mes, COUNT(r.IdReporte) AS Total
FROM (SELECT DATEFROMPARTS(YEAR(DATEADD(MONTH, -n, GETDATE())),
                           MONTH(DATEADD(MONTH, -n, GETDATE())), 1) AS Mes
      FROM (VALUES (5),(4),(3),(2),(1),(0)) v(n)) m
LEFT JOIN ReportesDanio r ON r.FechaReporte >= m.Mes AND r.FechaReporte <
DATEADD(MONTH, 1, m.Mes)
GROUP BY m.Mes
ORDER BY m.Mes
```

En lugar de partir de los reportes (que dejaría sin fila a un mes sin reportes), parte de una **tabla inventada al vuelo** con `VALUES (5),(4),...(0)`: seis números que representan «hace cinco meses… este mes». Con `DATEADD(MONTH, -n, GETDATE())` convierte cada número en una fecha, y `DATEFROMPARTS(año, mes, 1)` la lleva al primer día del mes. Después hace un `LEFT JOIN` con los reportes cuya fecha cae dentro de cada mes y cuenta con `COUNT(r.IdReporte)`. Un mes sin reportes aparece con total 0, porque `COUNT` de una columna ignora los `NULL`. Es la misma lógica de «partir de lo que siempre debe aparecer» que se vio con `vw_SalonesResumen` en la guía de la base.

3.8 Otras formas de combinar datos: `UNION ALL`, `EXISTS` y `OUTER APPLY`

`UNION ALL` pega los resultados de dos consultas una debajo de otra, siempre que tengan el mismo número de columnas y tipos compatibles. La pantalla de salones lo usa para mostrar en una sola tabla las unidades de equipo y los consumibles entregados:

```
SELECT IdUnidad, IdActivo, Activo AS Equipo, Categoria, 1 AS Cantidad,
CodigoInventario, Estado
FROM vw_UnidadesDetalle WHERE IdSalon = @s
UNION ALL
SELECT NULL, 0, Consumible, 'Consumible entregado', Entregado, NULL,
       'Última entrega: ' + CONVERT(VARCHAR(10), UltimaEntrega, 103)
FROM vw_ConsumiblesPorSalon WHERE IdSalon = @s
```

Fíjate en el truco para que las columnas encajen: la primera consulta tiene `1 AS Cantidad` (cada unidad cuenta como una) y la segunda trae la cantidad entregada; donde la segunda no tiene un dato (`IdUnidad`, `CodigoInventario`) pone `NULL`. `ALL` conserva

todo; sin esa palabra, `UNION` eliminaría duplicados, lo cual cuesta más y aquí no hace falta. Y el `+` entre textos, como en `'Última entrega: ' + CONVERT(...)`, concatena cadenas, igual que en C#; `CONVERT(VARCHAR(10), fecha, 103)` da formato de fecha día/mes/año.

`EXISTS` pregunta «existe al menos una fila que cumpla esto» sin traerla. En su forma `NOT EXISTS` se usa para borrar solo lo seguro:

```
DELETE TOP (@n) FROM UnidadesActivo
WHERE IdSalon = @IdSalon AND IdActivo = @IdActivo
  AND NOT EXISTS (SELECT 1 FROM Prestamos p WHERE p.IdUnidad =
UnidadesActivo.IdUnidad)
  AND NOT EXISTS (SELECT 1 FROM MantenimientosActivo m WHERE m.IdUnidad =
UnidadesActivo.IdUnidad)
  AND NOT EXISTS (SELECT 1 FROM ReportesDanio r WHERE r.IdUnidad =
UnidadesActivo.IdUnidad)
```

Se lee: «borra hasta `@n` unidades de este salón y este equipo, pero solo las que no tengan préstamos, mantenimientos ni reportes». Las que tienen historial se conservan. El `SELECT 1` dentro de `EXISTS` no importa por lo que devuelve (el 1 es arbitrario), sino por si encuentra algo o no. Y quien llama a esto recupera cuántas filas se borraron con `ExecuteNonQuery()`, para avisar si no se pudieron quitar todas.

`OUTER APPLY` sirve para traer, para cada fila, «la última fila relacionada» de otra tabla. El expediente del empleado lo usa para saber el estado de su pago del mes actual:

```
OUTER APPLY (SELECT TOP 1 x.EstadoPago
             FROM PagosPlanilla x
             WHERE x.IdEmpleado = e.IdEmpleado AND x.Anulado = 0
               AND x.Periodo = DATEFROMPARTS(YEAR(GETDATE()),
MONTH(GETDATE()), 1)
             ORDER BY x.IdPago DESC) pg
```

Por cada empleado, ejecuta la subconsulta con *su* `IdEmpleado` y se queda con el pago más reciente. Un `JOIN` normal no puede hacer eso porque no permite que la tabla de la derecha dependa de la fila de la izquierda, y el `OUTER` (como el `LEFT`) mantiene al empleado aunque no tenga pago.

3.9 Escribir datos: `INSERT`, `UPDATE`, `DELETE` y qué devuelve cada uno

Las consultas de lectura devuelven datos; las de escritura devuelven, como mucho, un número. Cada una usa un método distinto para ejecutarse, y elegir el correcto es parte de entender el código.

- **`INSERT` que devuelve el id nuevo.** Al crear un empleado, el texto termina con `SELECT CAST(SCOPE_IDENTITY() AS INT);`. `SCOPE_IDENTITY()` devuelve el último número que generó `IDENTITY` en esta conexión, es decir, el `IdEmpleado` recién creado. Como la consulta termina con un `SELECT`, se ejecuta con `ExecuteScalar()` y el programa recupera el id para usarlo enseguida al guardar la foto y las materias.
- **`UPDATE` y `DELETE`.** Se ejecutan con `ExecuteNonQuery()`, que devuelve **cuántas filas se vieron afectadas**. La línea más importante de un `UPDATE` es su `WHERE`: sin él se modifican todas las filas de la tabla. Por eso la baja de un empleado es `UPDATE Empleados SET IdEstado = 2 WHERE IdEmpleado = @id`, y nunca una instrucción sin su `WHERE`.
- **Insertar o actualizar según exista.** La foto del empleado usa un patrón compacto: `IF EXISTS (SELECT 1 FROM EmpleadoFotos WHERE IdEmpleado = @id) UPDATE ... ELSE INSERT ...`. Si ya había foto, la reemplaza; si no, la crea.
- **Un lote con transacción y bloqueo.** El alta de un activo del catálogo manda varias instrucciones juntas: `SET XACT_ABORT ON; BEGIN TRAN; DECLARE @nuevoId INT = (SELECT ISNULL(MAX(IdActivo), 0) + 1 FROM GestionActivos WITH (UPDLOCK, HOLDLOCK)); INSERT ...; COMMIT;`. Es la misma técnica de `MAX + 1` con bloqueo que usa `sp_GuardarSalon` en la base de datos, pero escrita en C#. Funciona porque SQL Server recibe todo el texto como un solo lote.

3.10 Llamar a un procedimiento almacenado

Hay dos maneras de ejecutar un procedimiento desde C#, y se eligen según lo que devuelva. Si solo hace algo y no devuelve filas (registrar un préstamo, anular un pago), se usa `Datos.Procedimiento("sp_RegistrarPrestamo", parámetros...)`, que configura el comando como de tipo procedimiento y lo ejecuta con `ExecuteNonQuery()`. Si el procedimiento **devuelve un resultado** (como `sp_GenerarPlanilla`, que responde cuántos pagos creó), se llama con texto y `Datos.Tabla`:

```
DataRow r = Datos.Tabla(
    "EXEC dbo.sp_GenerarPlanilla @Periodo = @p, @Usuario = @u, @Simular = 1",
    Datos.P("@p", MesElegido()), Datos.P("@u", Usuario())).Rows[0];
```

`EXEC` ejecuta el procedimiento, los parámetros `@p` y `@u` se asocian como siempre, y el resultado vuelve como una tabla de la que se lee la primera fila. Es la misma idea de siempre: C# manda texto y valores, SQL Server responde.

Si te quedas con una sola idea de esta sección, que sea esta: **el SQL es el lenguaje de lo que quieres, y C# solo lo transporta y presenta**. Cada vez que veas una consulta en el código, ubícala en una de estas categorías (contar, listar, resumir, combinar, escribir o llamar) y sabrás qué método la ejecuta y qué forma tendrá su respuesta.

4. La conexión y la clase `Datos`: cómo el programa le habla a SQL Server

Si la sección anterior explicó *qué* se manda a la base, esta explica *cómo* se manda. Son dos archivos pequeños, `Conexion.cs` (55 líneas) y `Datos.cs` (83 líneas), por los que pasa casi todo el tráfico del sistema. Vale la pena conocerlos casi de memoria: si entiendes estas 140 líneas, entiendes cómo funciona toda la comunicación con la base.

4.1 `Conexion`: la dirección de la base y la etiqueta del usuario

```
public const string Servidor = @"(localdb)\MSSQLLocalDB";
public const string BaseDatos = "Gestion_administrativa_y_escolar";

public static SqlConnection Conectar()
{
    string cadena = "Data Source=" + Servidor + "; Initial Catalog=" +
BaseDatos + "; Integrated Security=true;";
    SqlConnection con = new SqlConnection(cadena);

    con.StateChange += AvisarUsuarioALaBaseDeDatos;
    return con;
}
```

La **cadena de conexión** es un texto con pares `clave=valor` que le dice a .NET dónde está la base y cómo entrar. `Data Source` es el servidor (aquí `(localdb)\MSSQLLocalDB`, la instancia ligera que viene con Visual Studio), `Initial Catalog` es el nombre de la base, e `Integrated Security=true` significa que se usa la cuenta de Windows de quien ejecuta el programa, así que **no hay usuario ni contraseña escritos en el código**. Que el servidor y la base estén en constantes dentro de una sola clase tiene una consecuencia práctica: si mañana se pasa a un servidor en red, se cambia en un solo lugar y todas las pantallas lo siguen.

Fíjate en algo que se pasa por alto: `Conectar()` devuelve una conexión **que todavía no está abierta**. Quien la recibe decide cuándo abrirla con `Open()`. Es un detalle que explica por qué unos métodos llaman a `con.Open()` y otros no (se verá en `Datos`).

La línea que más importa es `con.StateChange += AvisarUsuarioALaBaseDeDatos`. `StateChange` es un evento que dispara la conexión cada vez que cambia de estado (cerrada, abierta…). El método conectado hace esto:

```
private static void AvisarUsuarioALaBaseDeDatos(object sender,
StateChangeEventArgs e)
{
    if (e.OriginalState != ConnectionState.Closed || e.CurrentState !=
ConnectionState.Open)
        return;

    try
    {
        SqlConnection con = (SqlConnection)sender;
        using (SqlCommand cmd = new SqlCommand(
            "EXEC sp_set_session_context @key = N'Usuario', @value = @u",
con))
        {
            cmd.Parameters.AddWithValue("@u", (object)Sesion.Usuario ??
DBNull.Value);
            cmd.ExecuteNonQuery();
        }
    }
    catch { }
}
```

Leerlo paso a paso: el primer `if` filtra para que el resto solo corra en el instante en que la conexión pasa de *cerrada* a *abierta*. En ese momento, el método ejecuta `sp_set_session_context`, que guarda en la sesión de SQL Server la clave `Usuario` con el nombre de quien inició sesión (`Sesion.Usuario`). Es la «etiqueta» que los triggers de auditoría leen después con `SESSION_CONTEXT(N'Usuario')`, y es lo que permite que el registro de actividad diga «María modificó este pago» y no «la cuenta de Windows modificó este pago». Si no hay nadie con sesión, `Sesion.Usuario` es `null`, se convierte en `DBNull.Value` y la auditoría lo muestra como `(sistema)`.

La elección de usar un evento es la parte ingeniosa. Podría haberse pedido a cada pantalla que etiquetara su conexión, pero bastaría que una lo olvidara para que sus cambios quedaran sin dueño. Con el evento, **ninguna conexión sale sin etiqueta**, y como se vuelve a etiquetar cada vez que se abre, nunca queda con el usuario de una sesión anterior. El `catch { }` vacío es deliberado: si la etiqueta fallara, el programa debe seguir funcionando; lo peor que pasa es que el cambio salga como `(sistema)`. Y como el método es `static`, no retiene objetos en memoria.

4.2 `Datos`: los cinco métodos que usa todo el programa

`Datos` es una clase `static` con una idea simple: cada vez que alguna parte del programa necesita algo de la base, llama a uno de estos métodos en lugar de armar conexiones y comandos a mano. Se distinguen por **qué devuelven**:

Método	Devuelve	Se usa para	Ejemplo real
`Datos.Tabla(sql, parámetros)`	un `DataTable` (filas y columnas)	listados, combos, cualquier `SELECT` con varias filas	la lista de personal
`Datos.Escalar(sql, parámetros)`	un solo valor (`object`)	contar, sumar, saber si algo existe, obtener un id	`SELECT COUNT(*) ...`
`Datos.Ejecutar(sql, parámetros)`	nada	`INSERT`, `UPDATE`, `DELETE`	la baja lógica de un empleado
`Datos.Procedimiento(nombre, parámetros)`	nada	llamar un procedimiento almacenado	`sp_RegistrarPrestamo`
`Datos.P(nombre, valor)`	un `SqlParameter`	crear un parámetro (convierte `null` en `NULL`)	`Datos.P("@id", id)`

Además están `PTexto`, que crea un parámetro de texto recortado y vacío-es-`NULL`, e `IdUnidad`, un ayudante que traduce un código de inventario en el id de la unidad y lanza un mensaje claro si no existe.

Veamos el código de los dos más usados:

```
public static DataTable Tabla(string sql, params SqlParameter[] ps)
{
    DataTable dt = new DataTable();
    using (SqlConnection con = Conexion.Conectar())
    using (SqlDataAdapter da = new SqlDataAdapter(sql, con))
    {
        if (ps != null) da.SelectCommand.Parameters.AddRange(ps);
        da.Fill(dt);
    }
    return dt;
}

public static object Escalar(string sql, params SqlParameter[] ps)
{
    using (SqlConnection con = Conexion.Conectar())
    using (SqlCommand cmd = new SqlCommand(sql, con))
    {
        if (ps != null) cmd.Parameters.AddRange(ps);
        con.Open();
        return cmd.ExecuteScalar();
    }
}
```

Hay varias ideas dentro de estas pocas líneas:

- **`params SqlParameter[] ps`**: la palabra `params` permite llamar al método con cualquier cantidad de parámetros separados por comas, o con ninguno: `Datos.Tabla(sql)`, `Datos.Tabla(sql, Datos.P("@a", 1))` o `Datos.Tabla(sql, Datos.P("@a", 1), Datos.P("@b", 2))`. Por eso las llamadas se leen tan limpias.
- **`SqlDataAdapter`** es el puente entre SQL Server y las tablas en memoria. `da.Fill(dt)` ejecuta la consulta y vuelca el resultado en el `DataTable`. El adaptador **abre y cierra la conexión por sí mismo** cuando ve que está cerrada, y por eso `Tabla` no llama a `Open()`, mientras que `Escalar`, `Ejecutar` y `Procedimiento` sí, porque usan `SqlCommand` directamente.
- **Una conexión nueva por llamada**, dentro de un `using`. Parece costoso pero no lo es: .NET mantiene un *pool* de conexiones y «cerrar» una conexión solo la devuelve al pool para reutilizarla. La ventaja es que nunca queda una conexión olvidada abierta, y es casi imposible agotarlas.

- **El `DataTable` devuelto está desconectado**: cuando el método termina, la conexión ya se cerró, pero los datos siguen en memoria y se pueden filtrar, ordenar y mostrar sin volver a la base.
- **`Escalar` devuelve `object`**, porque no sabe de antemano qué tipo será. Cuando no hay filas devuelve `null`; cuando hay una fila con `NULL` devuelve `DBNull.Value`. Por eso el código que lo usa suele convertir: `Convert.ToInt32(Datos.Escalar(...))`, o comprueba antes ambos casos, como hace `Datos.IdUnidad`: `if (r == null || r == DBNull.Value) throw ...`.

Hay una consecuencia de diseño importante: como **cada llamada abre su propia conexión**, dos llamadas seguidas a `Datos` no pueden compartir una transacción. Si necesitas que tres escrituras se guarden juntas o ninguna, `Datos` no sirve: hay que abrir una conexión propia y una `SqlTransaction`. Eso es exactamente lo que hace `frmAgregarPersonal` al guardar al empleado, su foto y sus materias, y se explica en la sección 5. La alternativa a mano es usar un procedimiento almacenado, que hace todo dentro de una transacción en el servidor.

4.3 `DataTable`, `DataView` y el filtro en memoria

Un `DataTable` es una tabla en memoria: tiene columnas, filas y tipos, igual que una de SQL Server pero viviendo en la RAM del programa. Un `DataView` es una **vista** de ese `DataTable`: no copia los datos, pero permite mirarlos con un filtro y un orden propios. La tabla visual (`DataGridView`) se conecta al `DataView`, no al `DataTable`, y eso es lo que hace posible el buscador en vivo:

```
dtEmpleados = Datos.Tabla("SELECT ...");        // 1. se traen los datos una
vez
vista = new DataView(dtEmpleados);              // 2. una vista sobre la
tabla
dgvEmpleados.DataSource = vista;                // 3. la tabla visual muestra
la vista
...
vista.RowFilter = "Nombre LIKE '%ana%'";        // 4. filtrar: la tabla
visual se actualiza sola
```

La propiedad `RowFilter` recibe un texto con una condición. **Ojo: no es SQL**, aunque se parezca. Es un pequeño lenguaje de expresiones de .NET que entiende `LIKE`, `AND`, `OR`, `IN` y funciones como `Convert`, pero que se aplica sobre los datos ya cargados, sin hablar con la base. En la pantalla de personal el filtro se arma así:

```
private void AplicarFiltro()
{
    var cond = new List<string>();

    string t = txtBuscar.Text.Trim();
    if (t.Length > 0)
    {
        string esc = Escapar(t);
        cond.Add("(Nombre LIKE '%" + esc + "%' OR Cargo LIKE '%" + esc +
                 "%' OR Convert(IdEmpleado, 'System.String') LIKE '%" + esc +
"%')");
    }

    int idCargo;
    if (cmbCargo.SelectedValue != null &&
        int.TryParse(cmbCargo.SelectedValue.ToString(), out idCargo) &&
idCargo > 0)
        cond.Add("IdCargo = " + idCargo);

    vista.RowFilter = string.Join(" AND ", cond);
    ActualizarTarjetas();
    ActualizarBotones();
}
```

La técnica es acumular condiciones en una lista y unirlas con `AND`. Si el cuadro de búsqueda tiene texto, se agrega una condición que busca en nombre, cargo o número de empleado (`Convert(IdEmpleado, 'System.String')` convierte el número a texto para poder usar `LIKE`). Si se eligió un cargo en el combo, se agrega `IdCargo = n`. Si no hay ninguna condición, `string.Join` da un texto vacío y el filtro queda sin efecto: se ven todos. Por último se recalculan las tarjetas de resumen (cuántos empleados, nómina total) recorriendo la vista filtrada.

El método `Escapar` existe porque el texto del usuario va dentro de la expresión:

```
private static string Escapar(string t)
{
    return t.Replace("'", "''").Replace("[", "[[]").Replace("%", "
[%]").Replace("*", "[*]");
}
```

Si alguien escribiera un apóstrofo, como en un apellido tipo O'Brien, la expresión se rompería; duplicar el apóstrofo lo evita. Y los caracteres `[`, `%` y `*` tienen significado especial dentro de un `LIKE` de `RowFilter`, así que se encierran entre corchetes para que se busquen como texto literal. No es una defensa contra inyección SQL (esto no es SQL ni toca la base), sino contra que el usuario rompa el filtro por accidente.

Cuándo filtrar en memoria y cuándo en la consulta: el filtro en memoria es instantáneo y no carga la base, ideal para listados de cientos o pocos miles de filas, que es el tamaño de este sistema. Filtrar en el servidor conviene cuando los datos son muchos o cuando el filtro depende de algo que la base calcula (como la columna `Alerta` de los préstamos, que depende de la hora actual).

4.4 Leer fila por fila: `SqlDataReader`

A veces no hace falta una tabla completa, sino una sola fila para rellenar un formulario. En ese caso se usa un `SqlDataReader`, que lee los resultados hacia adelante, una fila a la vez:

```
using (SqlDataReader r = cmd.ExecuteReader())
{
    if (!r.Read()) return;                          // sin fila: no hay nada
que leer
    NombreEmpresa = r["NombreEmpresa"].ToString();
    if (r["Logotipo"] != DBNull.Value)
    {
        byte[] bytes = (byte[])r["Logotipo"];     // la imagen guardada en la
base
        using (var ms = new MemoryStream(bytes))
            Logotipo = new Bitmap(Image.FromStream(ms));
    }
}
```

`Read()` avanza a la siguiente fila y devuelve `false` cuando ya no hay más. Las columnas se leen por nombre (`r["NombreEmpresa"]`). Este fragmento muestra además cómo se recupera una imagen: la columna `VARBINARY` llega como un arreglo de bytes, se envuelve en un `MemoryStream` (un archivo imaginario en memoria) y se convierte en un `Bitmap`. Así se carga el logotipo que se subió en el asistente de primer uso. La regla para elegir: `DataTable` si vas a mostrar o filtrar muchas filas; `SqlDataReader` si lees una sola fila o recorres resultados una vez; `Escalar` si solo quieres un valor.

4.5 Dos estilos que conviven, y qué dice eso del proyecto

Si abres varios archivos verás dos maneras de hacer lo mismo. Las pantallas más recientes usan `Datos.Tabla`, `Datos.Escalar` y compañía, en una línea. El código más antiguo (clases como `Salon`, `Planilla` y partes de `frmAgregarPersonal` y `frmLogin`) abre a mano `SqlConnection`, `SqlCommand` y `SqlDataAdapter`, y escribe diez líneas para lo mismo. Las dos hacen exactamente lo mismo: `Datos` solo es una envoltura que ahorra repetir la plantilla de conexión, comando, parámetros y limpieza.

El estilo manual sigue teniendo su lugar cuando se necesita algo que `Datos` no ofrece: una transacción que cubra varias operaciones (como al guardar un empleado), un `SqlDataReader`, o abrir la conexión una sola vez para varias consultas. Pero también hay código heredado que ya no se usa: la clase `Vistas`, `GenerarCredenciales`, `AsignarCargo`, `Dbsetup` y casi todos los métodos de `Empleado` no los llama ninguna pantalla actual (las reglas y consultas que hacen viven ahora dentro de los formularios o de los procedimientos). Es normal en un proyecto que creció por etapas, y conviene reconocerlo con naturalidad: es código para limpiar, no un fallo del sistema que funciona.

5. Errores, transacciones y procedimientos desde C#

En la guía de la base viste cómo un procedimiento lanza un error con `THROW`. Esta sección es la otra mitad de esa historia: qué hace el C# cuando ese error llega, cómo se agrupan varias escrituras en una sola operación desde el programa, y cómo se llama a un procedimiento. Son los tres temas donde un buen manejo se nota más, porque es lo que separa un programa que «se cae» de uno que «explica lo que pasó».

5.1 El viaje de un error, de la base a la pantalla

Cuando algo falla dentro de SQL Server (una regla de un procedimiento, una restricción de una tabla), la base no devuelve un valor: lanza un error. Ese error llega a C# convertido en una excepción de tipo **`SqlException`**, que trae dos datos que el programa usa constantemente:

- **`Message`**: el texto del error. Si lo lanzó un procedimiento con `THROW`, es exactamente la frase que escribió el autor del procedimiento, ya en español y pensada para el usuario («Stock insuficiente: no se puede despachar más de lo que hay»). Si lo lanzó una restricción de una tabla, es un texto técnico en inglés.

- **`Number`**: el número del error. Es la forma fiable de distinguir un error de otro sin depender del texto. Los números propios del sistema son los que empiezan en 50000 (los de los `THROW` de los procedimientos); los demás son de SQL Server. Los que el código reconoce son: **2601** y **2627** (se intentó duplicar algo que debe ser único: el primero viene de un índice único, el segundo de una restricción `UNIQUE`) y **547** (se violó una llave foránea o un `CHECK`).

La sección de la base lo decía: la pantalla ayuda, el procedimiento garantiza, la tabla es la última defensa. Aquí se ve el otro lado de esa frase. Cada pantalla que guarda tiene un `catch (SqlException ex)` que traduce lo que llegó a algo comprensible, y hay tres estilos de traducción en el proyecto.

5.2 Tres formas de traducir un error de SQL

Primera: mostrar el mensaje tal cual cuando es nuestro. La pantalla de reportes de daño lo hace en un método único y reutilizable:

```
private void MostrarErrorSql(SqlException ex)
{
    string msg;
    if (ex.Number == 2601 || ex.Number == 2627)
        msg = "Alguna de las unidades ya tiene un reporte abierto. No se
guardó ningún reporte.";
    else if (ex.Number >= 50000)
        msg = ex.Message;      // mensajes propios de los procedimientos: ya
vienen en español
    else
        msg = "Error de base de datos: " + ex.Message;

    MessageBox.Show(msg, "Aviso", MessageBoxButtons.OK,
MessageBoxIcon.Warning);
}
```

La lógica es una escalera: si es un duplicado, un mensaje redactado a mano; si el número es de 50000 en adelante, es un `THROW` nuestro y su `Message` ya sirve; si es cualquier otra cosa, un mensaje genérico con el detalle técnico. El método existe para que la misma traducción no se repita en cada botón.

Segunda: traducir cada número a una acción sobre un campo. La pantalla de registrar un pago va más lejos. En lugar de solo mostrar un mensaje, marca el campo culpable con el icono rojo y lo enfoca:

```
switch (ex.Number)
{
    case 50306:     // ya hay un pago de ese empleado en ese mes
        errorProvider.SetError(dtpPeriodo, "Este empleado ya tiene un pago en
este mes.");
        MessageBox.Show(empleado + " ya tiene un pago registrado en " + mes +
"...", "Pago ya registrado", ...);
        dtpPeriodo.Focus();
        break;
    case 50303:     // el descuento supera el salario base más el bono
        errorProvider.SetError(txtDescuento, "El descuento es mayor que el
salario base más el bono.");
        txtDescuento.Focus();
        break;
    ...
}
```

Es el mismo `switch` por número de error que viste catalogado en la base (los 50300 a 50311 son los de pagos). Cada `case` conecta un error del procedimiento con el control exacto de la pantalla que lo causó. La experiencia del usuario es mucho mejor: en vez de leer «monto inválido», ve el campo marcado y el cursor ya puesto sobre él.

Tercera: reconocer el nombre de la restricción dentro del mensaje. Cuando el error viene de una restricción de la tabla y no de un procedimiento, el texto de SQL Server incluye el nombre de la restricción. `frmAgregarPersonal` lo aprovecha:

```
if (ex.Message.Contains("UX_Empleados_DUI"))
    errorProvider.SetError(txtDui, "Ese DUI ya está registrado en otro
empleado.");
else if (ex.Message.Contains("UQ_Empleados_Nombre"))
    errorProvider.SetError(txtNombreEmpleado, "Ya existe un empleado con ese
nombre.");
else if (ex.Number >= 50000)
    MessageBox.Show(ex.Message, "No se pudo guardar", ...);   // mensajes de
los procedimientos
else
    MessageBox.Show("Error al guardar en la base de datos: " + ex.Message,
"Error SQL", ...);
```

Aquí se ve por qué los nombres que se dieron a las restricciones (`UX_Empleados_DUI` , `UQ_Empleados_Nombre`) importan: son parte del contrato con el programa. Es una técnica práctica pero algo frágil, porque depende de que el nombre aparezca en el texto del error.

Reconocer por `Number` es más robusto, y por eso el código más reciente prefiere esa vía.

Además de estos tres, hay un detalle de sintaxis que aparece en la pantalla de consumibles y conviene saber leer: `catch (SqlException ex) when (ex.Number == 2601 || ex.Number == 2627)`. La palabra `when` es un **filtro de excepción**: ese `catch` solo atrapa la excepción si se cumple la condición; si no, la excepción sigue buscando otro `catch` más abajo. Es una manera elegante de decir «atrapa el duplicado aquí, y todo lo demás en el `catch` general» sin un `if` adentro.

5.3 Validar antes, traducir después: el patrón de la pantalla de captura

Una pantalla de captura bien hecha tiene dos defensas con funciones distintas. La primera es **validar en C# antes de enviar**, para dar un aviso inmediato. En `frmAgregarPersonal` se ve claramente el patrón: se limpia el `ErrorProvider`, se recorre campo por campo marcando los que fallan y acumulando una bandera, y solo si no hubo ningún error se continúa.

```
errorProvider.Clear();
bool hayError = false;

if (nombre.Length == 0)
{ errorProvider.SetError(txtNombreEmpleado, "El nombre completo es
obligatorio."); hayError = true; }
...
if (hayError)
{
    MessageBox.Show("Revise los campos marcados con el icono rojo.", "Campos
incompletos", ...);
    return;
}
```

`ErrorProvider` es un componente de Windows Forms que dibuja un icono rojo junto a un control con un mensaje al pasar el ratón. La idea importante es que **no se detiene en el primer error**: marca todos los campos incorrectos de una vez, para que el usuario los corrija en una sola pasada, en lugar de descubrirlos uno por uno. La segunda defensa es la que se vio en 5.2: si algo se escapó a la validación y la base lo rechaza, se traduce el error. Entre las dos cubren casi todo.

Hay un tercer tipo de regla que no cabe en ninguna de las dos: las reglas de negocio que dependen de datos que la pantalla no tiene a mano, como «ya existe un director». Esas se consultan a la base desde C# justo antes de guardar (sección 3.2), y si fallan se muestra

un `MessageBox` con el motivo y se hace `return`.

5.4 Transacciones desde C#: `SqlTransaction`

En la guía de la base viste que una transacción agrupa varias escrituras en una sola operación: o se hacen todas o ninguna. Dentro de un procedimiento, la transacción se escribe en SQL. Pero a veces es la pantalla la que coordina varias escrituras, y entonces la transacción se abre desde C#. El ejemplo del proyecto es el guardado de un empleado, que escribe en tres lugares (la ficha, la foto y las materias):

```
using (SqlConnection con = Conexion.Conectar())
{
    con.Open();
    using (SqlTransaction tx = con.BeginTransaction())
    {
        try
        {
            int idEmp = GuardarEmpleado(con, tx);
            GuardarFoto(con, tx, idEmp);
            GuardarMaterias(con, tx, idEmp);
            tx.Commit();
        }
        catch
        {
            tx.Rollback();
            throw;
        }
    }
}
```

Los pasos son: abrir **una** conexión, iniciar la transacción con `BeginTransaction()`, ejecutar las operaciones pasándoles esa misma conexión y esa misma transacción, y al final `Commit()` para confirmar todo. Si alguna operación lanza una excepción, el `catch` ejecuta `Rollback()`, que deshace lo que se había hecho, y `throw;` vuelve a lanzar el error para que lo atrape el `catch` de afuera (el que muestra el mensaje al usuario). Esa estructura (abrir, comenzar, intentar, confirmar o deshacer y relanzar) es la plantilla de toda transacción escrita a mano.

Un detalle que se pasa por alto es el parámetro `tx`. **Cada comando que participa debe recibir la transacción** (`new SqlCommand(query, con, tx)`); si uno se olvida, SQL Server falla o, peor, ese comando se ejecuta fuera de la transacción y no se deshace. Y este es el

motivo por el que aquí no se usa `Datos`: `Datos.Ejecutar` abre su propia conexión y no sabe nada de una transacción abierta en otra.

La foto y las materias comparten la misma garantía. Por ejemplo, `GuardarMaterias` llama al procedimiento `sp_GuardarMateriasEmpleado` pasándole la transacción al crear el comando: `new SqlCommand("sp_GuardarMateriasEmpleado", con, tx)`. Es un procedimiento dentro de una transacción de C#. Si el procedimiento rechaza la lista de materias (por ejemplo, porque se quiere quitar una con grupos asignados), el empleado recién guardado también se deshace. Nada queda a medias.

En resumen, el proyecto usa tres formas de agrupar escrituras, y conviene saber cuál se eligió y por qué:

Forma	Dónde se ve	Cuándo conviene
Transacción dentro de un **procedimiento almacenado**	`sp_RegistrarMovimientoConsumible`, `sp_CerrarPeriodo`	La regla es de negocio y debe valer venga de donde venga. Es la preferida
Transacción abierta desde **C#** con `SqlTransaction`	`frmAgregarPersonal`	La pantalla coordina operaciones de naturaleza distinta (una `INSERT`, una imagen, un procedimiento)
Transacción escrita **en el texto SQL** de un lote	alta de un activo (`BEGIN TRAN ... COMMIT` con `UPDLOCK`)	Una secuencia corta y cerrada que se manda de una sola vez con `Datos.Ejecutar`

Y la regla de oro, la misma de la guía de la base: una sola escritura no necesita transacción; varias que deben ir juntas, sí.

5.5 Llamar a procedimientos y la capa de servicio

Llamar a un procedimiento ya se vio en la sección 3.10. Lo que falta es ver cómo se organiza. Para operaciones sencillas, la pantalla llama directamente a `Datos.Procedimiento`. Pero cuando la operación tiene lógica propia en C#, esa lógica se

separa en una clase de **servicio**. `ServicioUsuarios` es el ejemplo más completo: agrupa todo lo que tiene que ver con administrar usuarios, de modo que ninguna pantalla repite esas reglas.

```
public static string Crear(string usuario, string nombreCompleto, string
correo)
{
    string temporal = GenerarClaveTemporal();
    Datos.Procedimiento("sp_CrearUsuario",
        Datos.P("@Usuario", usuario),
        Datos.P("@NombreCompleto", nombreCompleto),
        Datos.PTexto("@Correo", correo),
        Datos.P("@HashClave", Modelos.Utils.PasswordHasher.Hash(temporal)),
        Datos.P("@CreadoPor", Sesion.Usuario));
    return temporal;
}
```

El método hace tres cosas que la base no puede hacer: genera una contraseña temporal aleatoria, la convierte en hash BCrypt antes de enviarla (así la base jamás la ve en claro), y devuelve la contraseña temporal al que llamó para mostrársela una sola vez al administrador. El procedimiento `sp_CrearUsuario` hace las validaciones (nombre, correo repetido, rol existente) y el `INSERT`. Es un reparto limpio: **C# hace lo que solo C# puede (aleatoriedad, hashing, mostrar), SQL hace lo que debe garantizarse (reglas y datos)**.

También hay reglas que el servicio valida y lanza como excepción propia, antes de llegar a la base. `CambiarMiClave` verifica la contraseña actual, que la nueva sea distinta, y que cumpla `ValidarClaveNueva` (al menos 8 caracteres, sin espacios en los extremos, con letras y números). Cuando alguna falla, lanza `throw new InvalidOperationException("La contraseña actual no es correcta.")`, y la pantalla que lo llamó lo atrapa con un `catch` y muestra el texto. Es la misma idea que `THROW` en SQL pero en C#: una excepción con un mensaje pensado para el usuario.

Finalmente, un caso de uso del `catch` vacío que sí está justificado: `RegistrarEvento` y `RegistrarAcceso` envuelven la llamada a la base en `try { ... } catch { }`. Registrar que alguien inició sesión es secundario: si esa escritura fallara, no debe impedir que la persona entre. La regla para decidir cuándo es válido ignorar un error: solo cuando lo que falla es accesorio y nunca cuando es el dato principal de la operación.

5.6 Cuando falla la propia conexión

Hay un error que no viene de una regla sino de la infraestructura: que SQL Server LocalDB no esté disponible o que la base no exista. Dos lugares lo manejan de forma explícita. En `Program.cs`, la comprobación inicial de si es el primer uso está envuelta en un `try/catch` que, si falla, muestra «No se pudo verificar la configuración» y cierra el programa en lugar de dejarlo abierto sin base. Y en el login, una bandera `conexionFallida` distingue un error de conexión de una contraseña incorrecta: si no hay conexión, el usuario ve el aviso de error de conexión y **no** se cuenta como un intento fallido de acceso, porque no es culpa suya. Es un detalle fino que muestra cuidado con la experiencia del usuario.

6. La lógica de las pantallas: cómo piensa un formulario

Si la sección 3 era el lenguaje y la 4 el transporte, esta es la gramática de las pantallas. Casi todas siguen los mismos patrones, así que aprenderlos una vez te permite leer cualquier formulario del sistema. Los ejemplos vienen de `frmGestionPersonal`, `frmAgregarPersonal`, `frmGestionActivos` (la parte de préstamos) y `frmDashboard`, que son cuatro pantallas distintas con la misma forma de pensar.

6.1 Una pantalla es una clase parcial reaccionando a eventos

Un formulario no tiene un `Main` que se ejecute de arriba abajo. Es un conjunto de **manejadores de eventos**: cada botón, cada cuadro de texto y cada tabla tiene métodos que Windows llama cuando el usuario hace algo (`btnGuardar_Click`, `txtBuscar_TextChanged`, `dgvEmpleados_SelectionChanged`). Entre un evento y otro el programa no hace nada: espera. Eso significa que el **estado** de la pantalla (qué empleado está seleccionado, si se está editando o creando, qué foto se cargó) tiene que guardarse en campos de la clase, y por eso al inicio de cada formulario hay una lista de variables privadas.

Los formularios grandes se reparten en varios archivos con `partial`. `frmGestionActivos` es una sola pantalla con pestañas, y su código está separado por tema: el catálogo, los préstamos, los consumibles, el mantenimiento y los reportes viven cada uno en su archivo, pero son una sola clase y comparten todos sus campos y métodos. Es una forma de ordenar sin partir la pantalla en piezas independientes.

6.2 La plantilla de carga: `Load`, métodos `Cargar...` y la bandera `cargando`

El arranque de casi todas las pantallas es el mismo. En el evento `Load` se hacen cuatro cosas en orden: poner los textos de ayuda (`toolTip1.SetToolTip(...)`), decidir qué puede hacer el usuario según su rol, llenar los combos y cargar la tabla principal. Mira cómo lo hace la pantalla de personal:

```
private void frmGestionPersonal_Load(object sender, EventArgs e)
{
    toolTip1.SetToolTip(btnAgregarPersonal, "Registrar un nuevo empleado");
    ...
    esAdmin = Sesion.Rol != null && Sesion.Rol.Trim().Equals("Administrador",
StringComparison.OrdinalIgnoreCase);
    btnAgregarPersonal.Enabled = esAdmin;
    ConstruirMenuMas();

    CargarCargos();
    MostrarEmpleados();
    ActualizarBotones();
}
```

Cada paso es un método con nombre claro (`CargarCargos`, `MostrarEmpleados`). Esa separación tiene un propósito práctico: después de agregar o editar un empleado, la pantalla vuelve a llamar a `MostrarEmpleados()` para refrescar la tabla sin repetir código.

Ahora el truco que más se repite y que más cuesta entender la primera vez: la bandera **`cargando`**. Mira `CargarCargos`:

```
private void CargarCargos()
{
    DataTable t = Datos.Tabla("SELECT IdCargo, NombreCargo FROM Cargos ORDER
BY NombreCargo");
    DataRow r = t.NewRow();
    r["IdCargo"] = 0; r["NombreCargo"] = "Todos los cargos";
    t.Rows.InsertAt(r, 0);

    cargando = true;
    cmbCargo.DataSource = t;
    cmbCargo.DisplayMember = "NombreCargo";
    cmbCargo.ValueMember = "IdCargo";
    cargando = false;
}
```

Primero se trae la lista de cargos y se le inserta a mano, en la posición 0, una fila falsa «Todos los cargos» con id 0 (así el filtro puede decir «ningún cargo en particular»). Después se asigna la tabla al combo. El problema es que **asignar el `DataSource` dispara el evento `SelectedIndexChanged`**, porque el combo pasa de vacío a tener un elemento seleccionado. Si ese evento ejecutara el filtro, se filtraría la tabla antes de que existan los datos. Por eso el manejador empieza con `if (cargando) return;` y el código que llena el combo enciende la bandera antes y la apaga después. Es un «no me interrumpas mientras me preparo».

En pantallas con varias cargas anidadas, como préstamos, se ve una variante más cuidadosa: `bool previo = cargando; cargando = true; ... cargando = previo;`. Guarda el valor anterior de la bandera y lo restaura después, en lugar de ponerlo siempre en `false`, para no apagarla por error si una carga se llamó desde dentro de otra.

6.3 Combos que muestran un texto y guardan un identificador

Un combo desplegable tiene que resolver un problema constante: el usuario ve «Planilla N° 3 · Base 600», pero el programa necesita el número 3 para guardarlo. Hay dos técnicas en el proyecto.

La primera es la del `DataTable`: se asigna como `DataSource` y se indican dos columnas, `DisplayMember` (la que se muestra) y `ValueMember` (la que se guarda). Después, `cmbCargo.SelectedValue` devuelve el id y `cmbCargo.Text` el texto. Es lo que usa `frmAgregarPersonal` para cargos y planillas. Observa que el texto mostrado se construye en la propia consulta SQL: `'Planilla N° ' + CAST(IdPlanilla AS VARCHAR(10)) + ' · Base ' + CONVERT(...)` es una columna calculada que existe solo para mostrarse.

La segunda técnica es una clase pequeña hecha para esto, `ElementoLista`:

```
private sealed class ElementoLista
{
    public readonly int Id;
    public readonly string Texto;
    public readonly string Extra;
    public ElementoLista(int id, string texto, string extra = null) { Id =
id; Texto = texto; Extra = extra; }
    public override string ToString() { return Texto; }
}

private static int? IdSeleccionado(ComboBox combo, ...)
{
    ElementoLista e = combo.SelectedItem as ElementoLista;
    return e == null ? (int?)null : e.Id;
}
```

Un combo puede contener **cualquier objeto**, y para mostrarlo llama a su método `ToString()`. Al sobrescribir `ToString()` para que devuelva `Texto`, el combo muestra el texto, pero el objeto guardado lleva también el `Id` y un dato `Extra` (el nombre limpio del empleado, por ejemplo). Al seleccionar, `combo.SelectedItem as ElementoLista` recupera el objeto completo. `IdSeleccionado` devuelve un `int?` (entero que admite nulo): si no hay nada seleccionado, `null`. Es más flexible que el `DataTable` cuando cada opción necesita llevar más de un dato.

6.4 Controles que cambian de forma según lo elegido

En la pantalla de préstamos, el campo del responsable es un cuadro de texto si el tipo es «Alumno» (se escribe el nombre) y un combo de empleados si es «Docente» o «Personal administrativo» (se elige de la lista, para que el préstamo quede ligado a un empleado real). Para lograrlo, el combo se **crea por código** en el mismo lugar del cuadro de texto y se alternan sus visibilidades:

```
cmbPrestEmpleado = new ComboOscuro();
cmbPrestEmpleado.Location = txtPrestResp.Location;
cmbPrestEmpleado.Size = txtPrestResp.Size;
cmbPrestEmpleado.Visible = false;
txtPrestResp.Parent.Controls.Add(cmbPrestEmpleado);
...
txtPrestResp.Visible = !esEmpleado;
cmbPrestEmpleado.Visible = esEmpleado;
```

Se hace así y no en el diseñador porque el combo es de la clase propia `ComboOscuro` y porque su contenido depende de datos. Y el cambio de tipo se conecta con una línea: `cmbPrestTipo.SelectedIndexChanged += (s, e) => AplicarTipoResponsable();`. Cada vez que el usuario cambia el tipo, `AplicarTipoResponsable` actualiza la etiqueta, recarga la lista de empleados (por si se agregó o dio de baja a alguien) y la filtra: solo docentes si el tipo es «Docente», solo no docentes si es «Personal administrativo».

Lo mismo ocurre con las unidades: al elegir un equipo, `CargarUnidadesPrestamo` consulta solo las unidades que **se pueden prestar** (que no estén dadas de baja, en reparación ni ya prestadas) y, si hay una sola, la selecciona sola. Aquí ves un principio de diseño importante: la pantalla no deja al usuario elegir algo inválido, pero **el procedimiento `sp_RegistrarPrestamo` lo vuelve a comprobar**. La pantalla es comodidad; el procedimiento es garantía.

6.5 Selección, botones y roles

Los botones que dependen de una selección se habilitan o deshabilitan en un solo método, y ese método se llama cada vez que algo relevante cambia:

```
private void ActualizarBotones()
{
    bool haySeleccion = dgvEmpleados.CurrentRow != null && vista != null &&
vista.Count > 0;
    btnActualizarPersonal.Enabled = esAdmin && haySeleccion;
    btnEliminarPersonal.Enabled = esAdmin && haySeleccion;
    btnExpediente.Enabled = haySeleccion;
}
```

Se llama desde `Load`, desde `SelectionChanged` de la tabla y desde `AplicarFiltro`. Centralizar esta lógica evita el error clásico de un botón que queda habilitado cuando no debe. Observa también `esAdmin`: la pantalla lee `Sesion.Rol` y activa o desactiva acciones. Una honestidad que conviene tener: como el login solo deja entrar a administradores, estas comprobaciones son hoy una defensa extra más que una barrera necesaria; dejan el sistema listo por si algún día se permiten otros roles, y el menú lateral ya oculta secciones completas para quien no sea administrador (`AplicarPermisosSidebar`).

6.6 Crear y editar en el mismo formulario

`frmAgregarPersonal` sirve para dos cosas: dar de alta y editar. La diferencia es un solo número, `idEmpleadoEditar`: si es 0, es un registro nuevo; si tiene valor, es una edición. El formulario tiene dos constructores (uno sin parámetros y otro que recibe el id), y el evento `Load` ajusta título, subtítulo y texto del botón, y en modo edición carga los datos del empleado. Al guardar, `GuardarEmpleado` elige entre `INSERT` y `UPDATE` con `if (idEmpleadoEditar == 0)`.

Hay un detalle muy cuidado en el modo edición. Al cargar los datos, el formulario guarda **copias de los valores originales** (`origNombre`, `origIdCargo`, `origTelefono`…). Al pulsar Guardar, lo primero que hace es compararlos con lo que hay ahora en pantalla (`HuboCambios()`), y si nada cambió, avisa «No se detectaron cambios» y no toca la base. Es un respeto al usuario (no genera un `UPDATE` ni un registro de auditoría inútil) y también ahorra ruido en el historial.

Otro detalle de este formulario: el DUI solo se muestra y se edita si el usuario es administrador (`lblDui.Visible = esAdmin`), y el `UPDATE` solo incluye la columna `DUI` cuando `esAdmin` es verdadero: `+ (esAdmin ? ", DUI = @dui" : "")`. Es un ejemplo de cómo se arma una consulta con una parte opcional sin dejar de usar parámetros: lo que cambia es la **estructura** del texto, que decide el programa y no el usuario, nunca un valor.

6.7 Controlar lo que se escribe

Además de validar al guardar, varias pantallas impiden que se escriban caracteres inválidos desde el principio, con el evento `KeyPress`:

```
private void txtNombreEmpleado_KeyPress(object sender, KeyPressEventArgs e)
{
    if (!char.IsControl(e.KeyChar) && !char.IsLetter(e.KeyChar) &&
!char.IsWhiteSpace(e.KeyChar))
        e.Handled = true;
}
```

`KeyPress` se dispara antes de que la tecla llegue al cuadro de texto. Si el manejador pone `e.Handled = true`, «ya lo atendí»: la tecla se descarta y no aparece. Aquí solo se aceptan letras y espacios (`char.IsControl` deja pasar teclas como borrar). El teléfono acepta solo números, `+`, `-` y espacio. Los campos numéricos con tope usan un ayudante, `NumericoLimitado.Aplicar`, que corrige en el acto si alguien escribe o pega un número mayor que el máximo (un `NumericUpDown` por sí solo deja escribirlo) y marca el campo con un aviso.

Las validaciones de formato viven en métodos pequeños y reutilizables. El correo se valida construyendo un `MailAddress`: si .NET lo acepta, es válido (`CorreoValido`). Para el DUI, `DuiValido` comprueba que tenga 10 caracteres, que el noveno sea un guion y que el resto sean dígitos.

Hay algo que conviene saber de `DuiValido` porque podrías ser preguntado: después de comprobar el formato, el método ejecuta `return true;`. Debajo de esa línea hay un cálculo del dígito verificador que **nunca se ejecuta** (el compilador lo marca como código inalcanzable). Es decir, el sistema valida el *formato* del DUI pero no su dígito verificador, aunque el mensaje y la ayuda dicen que sí. Si te lo preguntan, la respuesta honesta es que el cálculo está escrito y desactivado, y que activarlo es quitar un `return`.

6.8 Confirmaciones y mensajes: el idioma de los `MessageBox`

Todas las pantallas hablan con el usuario con el mismo vocabulario, y vale la pena notarlo porque es parte del cuidado del diseño. El icono del cuadro dice la gravedad: `Information` para un éxito o un dato, `Warning` para algo que el usuario puede corregir, `Error` para algo que falló de verdad, `Question` para pedir confirmación. Las acciones que no se pueden deshacer piden confirmación con `YesNo` y, cuando es una baja, dejan seleccionado por defecto el botón «No» (`MessageBoxDefaultButton.Button2`) para que un Enter distraído no destruya nada.

Un ejemplo del cuidado está en dar de baja a un empleado. Antes de preguntar, la pantalla hace una **revisión previa** con una sola consulta con cuatro subconsultas: cuántos salones tiene a cargo, cuántos pagos pendientes, cuántos grupos asignados este año y cuántos préstamos activos. Si hay pendientes, el mensaje los lista con viñetas («Es responsable de 2 salones») y pregunta «¿Dar de baja de todas formas?». Solo después ejecuta el `UPDATE ... SET IdEstado = 2`. La baja es lógica y reversible en la práctica, pero se trata con la seriedad de algo que afecta a otros módulos.

6.9 Pantalla dentro de pantalla: cómo funciona el panel principal

`frmDashboard` es la ventana que ve el usuario después de iniciar sesión: tiene el menú lateral a la izquierda y un panel contenedor a la derecha. Las demás pantallas (`frmGestionPersonal`, `frmGestionActivos`…) **no se abren como ventanas aparte**: se incrustan dentro de ese panel.

```
private void AbrirFormularioEnContenedor(Form formularioHijo)
{
    foreach (Control anterior in pnlContenedor.Controls.Cast<Control>
().ToList())
    {
        pnlContenedor.Controls.Remove(anterior);
        anterior.Dispose();
    }

    formularioHijo.TopLevel = false;
    formularioHijo.FormBorderStyle = FormBorderStyle.None;
    formularioHijo.Dock = DockStyle.Fill;

    this.pnlContenedor.Controls.Add(formularioHijo);
    formularioHijo.Show();
}
```

Los pasos son: retirar y liberar (`Dispose`) lo que hubiera en el panel; convertir el formulario en un control normal con `TopLevel = false` (por defecto un formulario es una ventana independiente); quitarle el borde; hacer que llene todo el panel con `Dock = Fill` ; agregarlo y mostrarlo. Es la razón por la que el sistema se siente como una sola aplicación y no como diez ventanas. Cada clic del menú llama a `Navegar(itmPersonal, () => new frmGestionPersonal())` , que crea la pantalla nueva dentro de un `try/catch` , la incrusta y marca el ítem del menú como activo. Como siempre se crea una pantalla nueva, **cada vez que entras a una sección se vuelven a cargar los datos de la base**, y por eso nunca ves información vieja.

El panel hace otras cosas que muestran cuidado: un `Timer` de 60 segundos actualiza las insignias del menú (cuántos reportes abiertos, cuántos pagos pendientes) con dos consultas pequeñas; `ProcessCmdKey` atiende los atajos F1 (abrir el manual en PDF) y Ctrl+B (plegar el menú); la preferencia de menú plegado se guarda en un archivo de texto en la carpeta de datos del usuario (`%APPDATA%\SistemaEscolar\preferencias.txt`) y se lee al iniciar; y al cerrar sesión se registra el evento, se borran `Sesion.Usuario` , `Sesion.Rol` y `Sesion.NombreCompleto` , y se vuelve a mostrar el login.

6.10 Exportar lo que se ve

Todas las pantallas con listados tienen un botón de exportar, y todas siguen la misma receta. Toman la **vista filtrada** (lo que el usuario está viendo, no toda la tabla), la convierten en un `DataTable` con las columnas que interesan, les ponen encabezados

amables y se la entregan al exportador:

```
DataTable dt = vista.ToTable(false, "IdEmpleado", "Nombre", "Cargo",
"Estado",
                             "FechaContratacion", "Telefono", "Correo");
dt.Columns["IdEmpleado"].ColumnName = "ID";
dt.Columns["Correo"].ColumnName = "Correo electrónico";
ExportadorPdf.Exportar("Listado de personal", dt);
```

`DataView.ToTable(false, columnas...)` crea una copia que contiene solo las filas que pasan el filtro actual y solo las columnas pedidas. Es la razón por la que lo que se exporta coincide con lo que el usuario filtró. El exportador pregunta si el archivo será PDF o Excel (se explica en la sección 8).

7. Seguridad en el código: sesión, login, contraseñas, recuperación y primer uso

La rúbrica pide seguridad en cuatro frentes (recuperación de credenciales, protección de contraseñas, validación de campos y retroalimentación al usuario), y casi todo eso está en C#. Esta sección recorre los archivos de seguridad del sistema: quién eres (sesión), cómo entras (login), cómo se guardan las contraseñas (BCrypt), cómo se recuperan (código por correo) y cómo nace el primer administrador (asistente de primer uso).

7.1 `Sesion`: quién está usando el programa

```
public static class Sesion
{
    public static string Usuario { get; set; }
    public static string Rol { get; set; }
    public static string NombreCompleto { get; set; }
}
```

Es la clase más corta del proyecto y de las más importantes. Son tres propiedades estáticas que viven en la memoria del programa mientras está abierto. El login las llena al entrar; el panel las lee para mostrar el nombre y decidir qué secciones aparecen; `Conexion` las usa para etiquetar cada conexión con el usuario; los formularios las leen para saber si el usuario es administrador; y al cerrar sesión se vacían a `null`. No hay un «token» ni nada que viaje por la red: es un programa de escritorio, y «estar con sesión» significa simplemente que estos valores no son nulos.

7.2 El login, paso a paso

El botón de iniciar sesión ejecuta `btnIngresar_Click`, que es una pequeña máquina de decisiones. Se entiende mejor si sigues el orden en que ocurren las cosas.

1. **Se limpian los avisos y se leen los campos.** El usuario con `Trim()`; la contraseña también, por lo que una contraseña no puede empezar ni terminar con espacios (y por eso las reglas de contraseña nueva los prohíben).
2. **Se validan los campos vacíos.** Si falta alguno, se marca en rojo la línea del campo, se muestra el aviso «Ingrese el usuario y la contraseña» y se enfoca el campo que falta. No se llega a tocar la base.
3. **Se revisa el bloqueo.** Si ese usuario está bloqueado temporalmente, se informa cuántos segundos faltan y se corta ahí. Se explica abajo.
4. **Se deshabilita el botón** y se cambia su texto a «Ingresando...» mientras se consulta, con un `try/finally` que lo vuelve a habilitar pase lo que pase. Es la defensa contra el doble clic.
5. **Se validan las credenciales** en `ValidarCredenciales`, que devuelve el rol si todo está bien o `null` si no.
6. **Según el resultado:** si fue bien, se borra el conteo de fallos, se llenan los datos de `Sesion`, se registra el acceso, se obliga a cambiar la contraseña si era temporal y se abre el panel; si falló, se registra el intento con su causa, se limpia la contraseña y se muestra el motivo.

El corazón es `ValidarCredenciales`. Primero hace una consulta parametrizada que trae, para el usuario escrito, su rol, su hash guardado, si está activo y su nombre. Aquí está la decisión más fina de seguridad: **la contraseña nunca se compara en SQL.** La base entrega el hash y es C# quien verifica con `PasswordHasher.Verificar(pass, hashGuardado)`. Así la contraseña escrita no viaja a la base ni queda en ningún registro de consultas. Después, cuando el valor guardado es un hash BCrypt y la contraseña es correcta, se comprueban dos condiciones más: que la cuenta esté activa (`Activo = 1`) y que el rol sea `Administrador`. Cada rechazo guarda su motivo en una variable (`inactivo`, `no_admin`), que después decide el mensaje y lo que se registra en la auditoría. Fíjate en la frase «cuando el valor guardado es un hash BCrypt»: hay una rama que no pasa por esas dos comprobaciones, y la explico justo debajo.

Un dato curioso es la rama de migración: si el valor guardado en `Clave` no empieza con `$2` (el prefijo de BCrypt), se asume que es una contraseña antigua en texto plano. Si coincide con lo que escribió el usuario, el login la acepta **y la reemplaza inmediatamente**

por su hash con un `UPDATE` parametrizado. Así los datos viejos se actualizan solos, sin obligar a nadie a hacer nada, y a partir de ese momento la contraseña ya no existe en claro.

Hay una salvedad que conviene conocer antes de que te la pregunten, porque la verifiqué leyendo el código: las comprobaciones de cuenta activa y de rol `Administrador` están escritas **dentro de la rama BCrypt**, no después de las dos ramas. Por eso, una cuenta cuya clave todavía esté en texto plano entraría aunque estuviera desactivada o no fuera administradora (y en ese primer ingreso tampoco se carga el nombre completo). El administrador que crea el asistente de primer uso ya guarda su clave con hash, así que esto solo afectaría a algún dato antiguo o de prueba que quede en tu base. La corrección natural es migrar esos valores una sola vez y borrar la rama, o mover las dos comprobaciones para que corran sin importar cómo se validó la clave. Si te lo preguntan, reconocerlo con esta precisión se ve mucho mejor que negarlo.

7.3 El bloqueo por intentos fallidos

Para frenar a quien prueba contraseñas una tras otra, el login lleva la cuenta de los fallos por usuario y lo bloquea un minuto tras cinco. El código usa dos diccionarios estáticos y tres constantes:

```
private const int MaxFallos = 5;
private const int SegundosBloqueo = 60;
private static readonly Dictionary<string, int> fallosPorUsuario = new
Dictionary<string, int>();
private static readonly Dictionary<string, DateTime> bloqueadoHasta = new
Dictionary<string, DateTime>();
```

Antes de validar, si el usuario está en `bloqueadoHasta` con una hora futura, se rechaza el intento sin consultar nada. Cuando falla una contraseña, el conteo sube; desde el tercer fallo el mensaje agrega «Le quedan N intentos»; al llegar a cinco se borra el conteo y se escribe una hora de desbloqueo: `bloqueadoHasta[usuario] = DateTime.Now.AddSeconds(60)`. Un acierto limpia ambos diccionarios. Detalles de diseño: la clave del diccionario es el usuario en minúsculas (`ToLowerInvariant()`), para que «Admin» y «admin» cuenten como el mismo; y **solo se cuentan las contraseñas equivocadas**, no los rechazos por cuenta desactivada ni por falta de permiso, porque no son intentos de adivinar.

Los diccionarios son `static`, así que sobreviven mientras el programa esté abierto aunque se cierre y se vuelva a crear la ventana de login, pero se pierden si se cierra el programa. Es una limitación honesta del enfoque: frena el uso casual de fuerza bruta, pero

no un ataque que reinicie el programa; para eso habría que guardar el bloqueo en la base de datos.

7.4 BCrypt: cómo se protegen las contraseñas

```
public static class PasswordHasher
{
    public static string Hash(string claveTextoPlano)
    {
        return BCrypt.Net.BCrypt.HashPassword(claveTextoPlano);
    }

    public static bool Verificar(string claveTextoPlano, string hashGuardado)
    {
        return BCrypt.Net.BCrypt.Verify(claveTextoPlano, hashGuardado);
    }
}
```

Son dos métodos de una línea que delegan en la librería `BCrypt.Net-Next`, y esa es la gracia: **no se inventa criptografía propia**. Usar una librería probada es la decisión correcta. Lo que conviene que entiendas es qué hace `Hash`. Genera una **sal** aleatoria (unos bytes al azar), la mezcla con la contraseña y aplica una función deliberadamente lenta muchas veces (el «factor de trabajo»). El resultado es un texto como `$2a$11$...` que contiene dentro, a la vista, el algoritmo, el factor y la sal, seguidos del hash en sí. Por eso `Verificar` solo necesita la contraseña escrita y el texto guardado: lee la sal del propio hash, repite el cálculo y compara.

Dos consecuencias que sirven para explicarlo. Primera: **dos usuarios con la misma contraseña tienen hashes distintos**, porque la sal es distinta; por eso no sirven las tablas precalculadas de contraseñas comunes. Segunda: **como es lento a propósito**, cada intento de adivinar cuesta, y probar millones de combinaciones se vuelve caro. Y una limitación natural: un hash no se puede «desencriptar». Si alguien olvida su contraseña, no hay forma de recuperarla; solo de reemplazarla por una nueva.

7.5 Contraseñas temporales: aleatoriedad bien hecha

Cuando un administrador crea otro usuario o le restablece la clave, el sistema genera una contraseña temporal. `ServicioUsuarios.GenerarClaveTemporal` es un buen ejemplo de cuidado en los detalles:

```
const string mayus = "ABCDEFGHJKLMNPQRSTUVWXYZ";
const string minus = "abcdefghijkmnpqrstuvwxyz";
const string numeros = "23456789";
string todo = mayus + minus + numeros;

using (RandomNumberGenerator rng = RandomNumberGenerator.Create())
{
    List<char> c = new List<char>();
    c.Add(mayus[Azar(rng, mayus.Length)]);
    c.Add(minus[Azar(rng, minus.Length)]);
    c.Add(numeros[Azar(rng, numeros.Length)]);
    while (c.Count < 10) c.Add(todo[Azar(rng, todo.Length)]);

    for (int i = c.Count - 1; i > 0; i--)
    {
        int j = Azar(rng, i + 1);
        char t = c[i]; c[i] = c[j]; c[j] = t;
    }
    return new string(c.ToArray());
}
```

Hay cinco decisiones buenas en pocas líneas. Los conjuntos de caracteres **omiten los que se confunden al leer** (la I y la O mayúsculas, la l y la o minúsculas, el 0 y el 1), porque la persona tendrá que copiarla a mano. Se **garantiza al menos una mayúscula, una minúscula y un número**, así que siempre cumple la regla de combinar letras y números. Se completa hasta 10 caracteres. Después se **mezclan** con el algoritmo de Fisher–Yates (el recorrido que intercambia cada posición con otra al azar) para que los tres caracteres garantizados no queden siempre al principio. Y el azar viene de `RandomNumberGenerator`, el generador **criptográfico** del sistema, y no de `Random`, cuyos números son predecibles.

El método auxiliar `Azar` tiene un detalle que pocos conocen y que demuestra rigor:

```
uint limite = uint.MaxValue - (uint.MaxValue % (uint)max);
do { rng.GetBytes(b); v = BitConverter.ToUInt32(b, 0); } while (v >= limite);
return (int)(v % (uint)max);
```

Si simplemente se hiciera `valor % max`, algunos resultados saldrían un poco más a menudo que otros (porque el rango de números no es divisible exacto entre `max`). Este ciclo descarta los valores que caen en la zona sobrante y vuelve a sacar: así todos los caracteres son exactamente igual de probables. Se llama **muestreo por rechazo**.

La contraseña temporal se crea con `DebeCambiarClave = 1`. Cuando esa persona entra por primera vez, el login detecta la bandera, abre `dlgCambiarClave` en modo forzado (donde no pide la contraseña actual porque acaba de autenticarse) y no la deja pasar al panel hasta que la cambie. Si cancela, se cierra la sesión con el aviso «Debe cambiar la contraseña temporal para poder entrar».

Una observación honesta que te conviene tener: las reglas de contraseña no son iguales en los tres lugares donde se escribe una. El cambio de contraseña (`ValidarClaveNueva`) exige al menos 8 caracteres con letras y números y sin espacios en los extremos; el asistente de primer uso exige al menos 8 caracteres; y la pantalla de recuperación exige al menos 6. Es una inconsistencia que conviene unificar en un solo lugar, y puedes presentarla como una mejora identificada.

7.6 Recuperar la contraseña por correo

La pantalla `frmRecuperarContraseña` implementa el flujo de dos pasos, y su estado vive en cuatro campos privados: el código generado, el correo destino, la hora de vencimiento y el contador de intentos.

En el primer paso, la persona escribe su correo. El programa valida el formato (con `MailAddress`), consulta que exista en `Usuarios` y, si existe, **genera un código de seis dígitos con el generador criptográfico** (`100000 + n % 900000`), lo envía por correo y guarda en memoria el código, el correo y la hora de vencimiento (15 minutos). El código no se escribe en la base de datos ni en ningún archivo. En el segundo paso, escribe el código y la contraseña nueva. Al pulsar Restablecer se valida, en orden: longitud de la contraseña, que las dos coincidan, que no haya vencido, y que el código sea correcto. Cada código equivocado suma un intento; al quinto, el código se invalida y el proceso se reinicia. Si todo es correcto, la contraseña nueva se convierte en hash y se guarda con `UPDATE Usuarios SET Clave = @clave WHERE Correo = @correo`.

El envío del correo merece su explicación porque es la única parte del sistema que sale a internet. `ServicioCorreo` no usa un servidor de correo propio: hace una petición HTTP `POST` con un JSON a un script publicado en Google Apps Script, que es quien envía el mensaje. El método arma el JSON a mano (con una función `Escapar` para las comillas y saltos de línea), le agrega un **token** que el script verifica para saber que la petición viene del sistema, fija un tiempo máximo de espera de 25 segundos, y lee la respuesta buscando `"ok":true` ; si no, extrae el mensaje de error. Todo se ejecuta en un hilo aparte con `Task.Run` , y la pantalla lo espera con `await` mientras muestra «Enviando…» y el cursor de espera, de modo que la ventana no se congela.

Hay dos cosas de seguridad que conviene mencionar con naturalidad. El mensaje «Este correo no está registrado» revela si un correo existe o no, lo cual es un equilibrio habitual entre claridad y discreción. Y la dirección del script y el token están escritos como constantes en `ServicioCorreo.cs`; como el token es lo que protege el envío de correos, no debe publicarse el código en un repositorio público con su valor real, y si se hubiera expuesto, lo correcto es generar uno nuevo.

7.7 El asistente de primer uso

En `Program.Main` lo primero que se hace es preguntar `ConfiguracionInicial.EsPrimerUso()`, que ejecuta `SELECT COUNT(*) FROM ConfiguracionEmpresa` y devuelve verdadero si no hay ninguna fila. Si es el primer uso, se muestra `frmConfiguracionInicial` como diálogo; si el usuario lo cancela, `Main` termina y el programa se cierra. Solo cuando termina bien se llama a `ConfiguracionInicial.Cargar()` (que trae el nombre y el logotipo de la institución a propiedades estáticas) y se abre el login.

El asistente es un formulario de **tres páginas** que se muestran y ocultan con paneles: la página 1 pide los datos de la institución y el logotipo (que se limita a 2 MB y se comprueba que sea una imagen válida), la 2 pide los datos del primer administrador y la 3 verifica su correo. Un método, `MostrarPagina(n)`, controla qué panel se ve, el texto del paso («PASO 2 DE 3»), la barra de progreso y el texto del botón, que cambia de «Siguiente» a «Verificar y guardar». Si ya existe un administrador (`necesitaAdmin` es falso), el asistente se reduce a una sola página.

La página de verificación es una pequeña máquina de estados con las mismas ideas que la recuperación de contraseña (código aleatorio, vencimiento de 15 minutos, máximo 5 intentos), más dos cuidados extra: un enfriamiento de 60 segundos entre reenvíos de código y dos banderas, `enviando` y `guardando`, que bloquean cualquier clic repetido mientras algo está en curso. Y un vía de escape: si la persona no puede verificar, puede «omitir», aceptando que el correo quede **sin verificar** (se guarda `CorreoVerificado = 0`) y advirtiéndole que, si está mal escrito, la recuperación no funcionará.

Al final, `Guardar` llama a `ConfiguracionInicial.GuardarInicial`, que convierte la contraseña en hash BCrypt **en C#** y envía todo al procedimiento `sp_GuardarConfiguracionInicial`, que guarda la configuración y crea al administrador dentro de una transacción. Si el procedimiento falla (por ejemplo, porque el usuario ya existe), el `catch (SqlException)` muestra su mensaje en la propia pantalla, sin cerrarla. Una pieza de cuidado: al cerrar el asistente sin terminar, `FormClosing` pregunta si se quiere salir, porque no completarlo significa que la aplicación termina.

7.8 Qué es protección real y qué es apariencia

Vale distinguir las defensas que de verdad protegen de las que ayudan a la experiencia. Protegen de verdad: guardar solo hashes BCrypt, comparar la contraseña en C# y no en SQL, usar parámetros en todas las consultas, aleatoriedad criptográfica para códigos y contraseñas temporales, vencimiento e intentos limitados en los códigos, y que solo un administrador pueda iniciar sesión. Son apoyo para el usuario: ocultar botones o secciones del menú según el rol, el icono del ojo para ver la contraseña, el aviso de Bloq Mayús. Si te preguntan cuál es la capa que realmente impide un acceso indebido, la respuesta es la verificación en el login y las reglas de la base de datos, no la interfaz.

8. Métodos y servicios importantes: exportadores, documentos PDF y utilidades

Hasta aquí todo era hablar con la base y mostrar pantallas. Esta sección cubre las piezas que **producen algo que sale del programa**: archivos PDF, libros de Excel y mensajes de correo. Son clases `static` de la biblioteca `Modelos`, sin pantalla propia, y todas siguen el mismo patrón que ya conoces: reciben datos, los transforman y pueden fallar de maneras que hay que explicar al usuario.

8.1 El patrón común de toda exportación

Cualquier archivo que el sistema genera sigue los mismos cinco pasos, y los puedes señalar en cada exportador:

1. **Validar** que haya algo que exportar («No hay datos para exportar»).
2. **Pedir dónde guardar** con un `SaveFileDialog`, con un nombre sugerido y un filtro de extensión. Si el usuario cancela, se sale sin hacer nada.
3. **Construir el archivo** dentro de un `try/catch`.
4. **Avisar del resultado** con un `MessageBox`: éxito, o error con un texto útil.
5. **Ofrecer abrirlo** (`Process.Start`), porque es lo que casi siempre se quiere hacer después.

El detalle de cuidado está en el error: cuando se guarda un Excel que el usuario todavía tiene abierto, Windows impide sobrescribirlo, y el mensaje dice explícitamente «Si está abierto en Excel, ciérrelo e intente de nuevo». Es un error que ocurre en la vida real y el programa lo anticipa.

8.2 `ExportadorPdf`: construir un PDF con iTextSharp

Para generar PDF el proyecto usa la librería iTextSharp. Un PDF se construye como un documento al que se le van agregando elementos. Los objetos que aparecen son pocos y se repiten:

Objeto	Qué representa
`Document`	La hoja: tamaño de página y márgenes
`PdfWriter`	El que escribe el documento en un archivo (un `FileStream`)
`Paragraph`, `Phrase`	Texto con una fuente
`PdfPTable`, `PdfPCell`	Una tabla y sus celdas
`Font` / `FontFactory`	Tipo de letra, tamaño, estilo y color

`ExportadorPdf.Exportar(titulo, dt)` recibe un `DataTable`, y lo primero que hace es preguntar el formato con `ElegirFormato()`: si el usuario escoge Excel, delega en `ExportadorExcel` y termina. Ese método de elección es un diálogo que se **construye por completo en código** (un `Form` con una etiqueta y tres botones), y usa funciones lambda que guardan la elección en una variable local: `bPdf.Click += (s, e) => { elegido = "pdf"; f.Close(); };`. Es un buen ejemplo de una lambda que «captura» una variable de afuera.

Lo interesante de la generación es cómo la tabla se adapta a los datos, sin saber de antemano qué columnas tendrá:

- **La orientación** depende del ancho: `columnas.Count > 6 ? PageSize.A4.Rotate() : PageSize.A4`. Si hay más de seis columnas, la página se gira a horizontal para que quepan.
- **`HeaderRows = 1`** hace que el encabezado se repita en cada página cuando la tabla es larga.
- **El formato depende del tipo de cada valor.** Un `DBNull` se escribe como texto vacío, un `DateTime` como `dd/MM/yyyy`, y un número decimal con dos decimales (`ToString("N2")`) y alineado a la derecha. Así una misma función sirve para listados de personal, de activos o de pagos.

- **Se puede ocultar columnas** pasando sus nombres en `columnasOcultas` (un `params string[]`); la lista de columnas se arma con LINQ: `dt.Columns.Cast<DataColumn>().Where(c => !columnasOcultas.Contains(c.ColumnName, ...)).ToList()`.

Todo el documento se escribe dentro de un `using (var fs = new FileStream(...))`, así que el archivo se cierra siempre. Y como se dijo en la sección 6, quien llama le pasa la **vista filtrada**, de modo que se exporta exactamente lo que el usuario ve.

8.3 `ExportadorExcel`: un `.xlsx` hecho a mano

Este es el exportador más original del sistema y el que más se presta a una pregunta de «¿cómo lo hicieron?». La respuesta corta: un archivo `.xlsx` **es en realidad un `.zip` que contiene varios archivos XML**. Excel lo abre porque el contenido sigue un formato estándar (Open XML). En vez de agregar una librería para Excel, el proyecto escribe esos XML directamente y los empaqueta en un zip armado a mano, **sin compresión**.

El método `ConstruirLibro` arma la lista de archivos que debe tener el paquete: `[Content_Types].xml` (qué tipo de archivo es cada parte), `_rels/.rels` y `xl/_rels/workbook.xml.rels` (cómo se relacionan), `xl/workbook.xml` (la lista de hojas), `xl/styles.xml` (los estilos) y `xl/worksheets/sheet1.xml` (los datos). Después `Zip(...)` los empaqueta escribiendo, para cada archivo, una cabecera local, su contenido y su **CRC32** (una suma de comprobación que el formato zip exige para detectar archivos dañados), y al final un directorio central con la lista de todos los archivos. Esa parte es la más técnica y no hace falta que la recites: basta con saber que existe y para qué sirve.

Lo que sí conviene entender es la hoja de datos, porque es donde se resolvió el problema de fondo. Antes se exportaba a CSV (texto), y las fechas se guardaban como texto o como números que Excel mostraba como `########` cuando la columna era angosta. Ahora cada celda se escribe **con el tipo que le corresponde**:

```
if (v is DateTime)
{
    DateTime d = (DateTime)v;
    bool conHora = d.TimeOfDay != TimeSpan.Zero;
    return "<c r=\"" + refer + "\" s=\"" + (conHora ? EstiloFechaHora :
EstiloFecha) + "\"><v>" +
           d.ToOADate().ToString("R", CultureInfo.InvariantCulture) + "</v>
</c>";
}
```

Excel guarda las fechas como números: los días transcurridos desde 1899-12-30. `ToOADate()` hace esa conversión, y el atributo `s` apunta a un estilo definido en `styles.xml` que le dice a Excel «muestra este número con formato dd/mm/aaaa». Resultado: en Excel es una fecha de verdad (se puede ordenar y filtrar por fecha), no un texto. Los enteros y decimales se escriben como números, y el texto como texto en línea (`inlineStr`), escapando los caracteres especiales con el método `Xml` (`&`, `<`, `>`, comillas y los caracteres de control que XML no admite).

Otros cuidados del libro: **el ancho de cada columna se calcula** mientras se recorren las celdas (el texto más largo manda, con un mínimo de 10 y un máximo de 60), la cabecera lleva fondo oscuro y letra blanca, la primera fila queda **inmovilizada** con `pane ... state="frozen"`, hay un **autofiltro** sobre toda la tabla y la hoja se imprime horizontal. Hay un truco chiquito pero ingenioso en `Referencia`, que convierte una posición numérica en el nombre de celda de Excel (0 → `A`, 25 → `Z`, 26 → `AA`):

```
for (int c = columna; c >= 0; c = c / 26 - 1) letras = (char)('A' + c % 26) +
letras;
```

Es una conversión a base 26 sin cero (como cuentan las columnas de Excel), y por eso el `- 1` al dividir: «Z» es la 26, y la siguiente no es «A0» sino «AA». Un ejemplo para comprobarlo: la columna 27 (índice 26) da primero `'A' + 26 % 26 = A`, luego `26/26 - 1 = 0`, que vuelve a dar `A`, y queda `AA`.

8.4 `DocumentosPdf` y la hoja de resguardo: documentos con formato propio

A diferencia del exportador general (que vuelca una tabla cualquiera), `DocumentosPdf` genera **documentos con diseño** propio: la ficha del empleado y la boleta de pago. Y `HojaResguardoPdf` genera la hoja que firma quien recibe un equipo prestado. Siguen una estructura constante: una consulta a una **vista**, validaciones previas, y la composición del documento.

La boleta de pago es el mejor ejemplo. Empieza con `SELECT * FROM vw_BoletaPago WHERE IdPago = @id`: la vista de la base ya trae unidos el empleado, su cargo, su DUI y los montos. Después se aplican dos guardas con mensaje claro: si no se encuentra el pago, y si está **anulado** (no se emite boleta de un pago anulado). Luego se arma el documento: un encabezado común (`Encabezado`), una tabla de datos del empleado (con el número de boleta formateado como `00000`), y una tabla de conceptos con ingresos y descuentos.

Hay una lógica de negocio escondida que vale la pena ver. En la base, `Descuento` es **un solo número** que mezcla el descuento de ley y el de ausencias, pero la boleta los muestra separados. C# los separa con una resta: `descOtros = desc - descAus`. Si `descOtros` es mayor que cero, agrega la fila «Descuentos de ley y otros»; si `descAus` es mayor que cero, agrega «Ausencias descontadas (N días)». Y el bono solo aparece si es mayor que cero. El resultado es una boleta que explica el pago en lugar de mostrar solo el neto.

Los métodos auxiliares pequeños (`Campo`, `FilaConcepto`, `Txt`, `Dinero`, `Edad`, `Antiguedad`) evitan repetir código y encapsulan decisiones de formato: `Txt` convierte los nulos de la base en texto vacío y recorta espacios, `Dinero` escribe el monto con `$` y dos decimales, y `Edad` y `Antiguedad` calculan a partir de fechas (la edad resta uno si el cumpleaños de este año todavía no llegó, y la antigüedad se expresa como «2 años y 3 meses»). Y `PedirArchivo` limpia el nombre sugerido reemplazando los caracteres que Windows no permite en un nombre de archivo (`Path.GetInvalidFileNameChars()`), para que un nombre de empleado con un carácter raro no rompa el cuadro de guardar. `Terminar` cierra el ciclo preguntando si se quiere abrir el PDF.

8.5 Servicios de apoyo

Tres clases pequeñas se explicaron en la sección 7 y aquí solo se ubican. **`ServicioCorreo`** manda los códigos de verificación y los reportes del Centro de soporte (`EnviarSoporteAsync`) por medio de un script en la nube. **`PasswordHasher`** convierte y verifica contraseñas. **`ServicioUsuarios`** concentra la administración de usuarios. Y **`ConfiguracionInicial`** guarda en propiedades estáticas el nombre y el logotipo de la institución al arrancar, para que el panel y los documentos los usen sin consultar la base cada vez.

Una nota sobre `Dbsetup.cs`: contiene un método que crearía la base de datos ejecutando un script incrustado en el programa, pero **ninguna parte del sistema lo llama**. La base se instala ejecutando `BaseDeDatos_Completa.sql` en Management Studio, como se explicó en la guía de la base, y el asistente de primer uso parte de que la base ya existe. Es uno de esos restos de una etapa anterior del proyecto.

9. Diseño por código: cómo se dibuja la identidad visual del sistema

Si hay algo que distingue a este proyecto a primera vista es su diseño: el tema oscuro, el amarillo como color de acento, el menú lateral con sus iconos, las tablas con filas que reaccionan al ratón, las cápsulas de color para los estados. Nada de eso viene «de fábrica» con Windows Forms, que por defecto se ve como una aplicación gris de los años noventa.

Es el resultado de código escrito a propósito, y es la parte del C# donde más se nota el trabajo del equipo. Esta sección explica cómo funciona, con suficiente detalle para que puedas defenderlo y, si hace falta, agregar un icono nuevo.

Hay dos maneras de conseguir un aspecto propio en Windows Forms y el proyecto usa las dos. La primera es **cambiar propiedades** de los controles estándar: el color de fondo, el estilo plano de un botón (`FlatStyle.Flat`), la fuente. Es rápida, pero limitada. La segunda es el **dibujo propio** (en inglés *owner-draw*): en lugar de dejar que Windows dibuje el control, el programa sobrescribe el método que lo pinta y lo dibuja él mismo con líneas, rellenos y texto. Es más trabajo, pero no tiene límites. Los iconos del menú, las cápsulas de estado y las barras de desplazamiento delgadas son dibujo propio.

9.1 La paleta: un solo lugar para los colores

Todo el sistema usa un conjunto reducido de colores, declarados una sola vez en la clase `PaletaSistema`:

Nombre	Valor (R, G, B)	Para qué se usa
`Fondo`	18, 18, 18	El fondo general, casi negro
`Tarjeta`	27, 29, 38	Paneles y tablas: un gris azulado un poco más claro que el fondo
`Campo`	38, 42, 58	Cuadros de texto y listas desplegables
`Borde`	45, 50, 63	Líneas finas de contorno
`Amarillo`	250, 204, 21	**El acento**: botones principales, el elemento activo, el foco
`Gris`	55, 60, 80	Botones secundarios y el ítem resaltado de una lista
`Rojo`	220, 53, 69	Acciones peligrosas
`Texto`	240, 244, 252	El texto principal (no es blanco puro: es un blanco azulado más suave para la vista)
`Tenue`	170, 180, 200	Texto secundario y flechas

La gracia de una paleta central es que cambiar el aspecto de todo el sistema sería tocar nueve números. Y el diseño sigue una regla de las que hacen que una interfaz se vea profesional: **un solo color de acento**. El amarillo se reserva para lo que importa (el botón

principal, la fila seleccionada, el campo con el foco, la opción activa del menú), y todo lo demás es una escala de grises. Por eso el amarillo «se nota» cuando aparece.

Una honestidad que conviene tener: algunas pantallas (el login, el inicio, el asistente) vuelven a declarar sus propios colores como constantes locales (`Amarillo`, `Rojo`…), con los mismos valores. Funciona, pero lo ideal sería que todas leyeran de `PaletaSistema`.

9.2 Cómo se dibuja en Windows Forms: lo mínimo que debes entender

Para entender el resto necesitas cuatro ideas, todas muy sencillas.

Primera: el evento de pintura. Cada control tiene un método `OnPaint` (o un evento `Paint`) que Windows llama cuando hay que dibujarlo. Recibe un `PaintEventArgs` que trae un objeto `Graphics`, el «lápiz y el lienzo» sobre el que se dibuja. **Segunda: `Invalidate()`.** Windows no redibuja un control cada fracción de segundo; solo cuando lo necesita. Si el programa cambia algo que afecta al aspecto (pasar el ratón por encima, seleccionar, marcar un error), llama a `Invalidate()`, que significa «este control ya no está al día, vuelve a pintarlo». Por eso casi todos los métodos que cambian el aspecto terminan en `Invalidate()`.

Tercera: GDI+. Es el conjunto de herramientas de dibujo de .NET (`System.Drawing`). Las piezas que usa el proyecto son estas:

Herramienta	Qué es	Ejemplo en el proyecto
`Pen`	Lápiz: dibuja **contornos** y líneas, con grosor y extremos	`new Pen(color, 1.8f)` para los iconos
`SolidBrush`	Pincel: **rellena** formas con un color	El fondo redondeado de la opción activa
`Draw...` / `Fill...`	`DrawLine`, `DrawEllipse`, `DrawArc`, `DrawPolygon`, `DrawBezier` dibujan el contorno; `FillRectangle`, `FillEllipse`, `FillPath` lo rellenan	Las líneas de los iconos; el punto de la insignia
`GraphicsPath`	Una forma cualquiera armada con curvas y rectas	Los rectángulos con esquinas redondeadas

Herramienta	Qué es	Ejemplo en el proyecto
`SmoothingMode.AntiAlias`	Suaviza los bordes para que los diagonales y las curvas no se vean con «escalones»	Se activa antes de dibujar cualquier icono
`TextRenderer`	Dibuja texto con la misma calidad que el texto normal de Windows	Los nombres de las opciones del menú

Las coordenadas empiezan en la **esquina superior izquierda** en (0, 0), el eje X crece hacia la derecha y el eje **Y crece hacia abajo**. Es lo contrario de un plano de matemáticas, y es la fuente de casi todos los errores al principio. Y como `Pen` y `SolidBrush` retienen recursos de Windows, se crean dentro de un `using` para liberarlos.

Cuarta: el doble búfer. Si un control se dibuja directamente en pantalla, a veces se ve un parpadeo: el fondo se pinta primero, luego los detalles encima, y el ojo alcanza a ver los pasos intermedios. La solución es dibujar primero en una imagen oculta y mostrar el resultado de golpe. Se activa con una línea que aparece en todos los controles propios:

```
SetStyle(ControlStyles.AllPaintingInWmPaint | ControlStyles.UserPaint |
         ControlStyles.OptimizedDoubleBuffer | ControlStyles.ResizeRedraw,
true);
```

`UserPaint` dice «yo dibujo este control»; `OptimizedDoubleBuffer` activa el doble búfer; `AllPaintingInWmPaint` evita un borrado previo que causa parpadeo; y `ResizeRedraw` repinta el control cuando cambia de tamaño (necesario porque lo que se dibuja depende del ancho y el alto).

9.3 Los iconos de la barra lateral: dibujados con líneas

Los iconos del menú (la casa, la caja, las dos personas, el reloj…) **no son imágenes**. No hay ningún archivo `.png` ni `.ico`: cada icono es un pequeño programa de dibujo de unas cuantas líneas, dentro de la clase `IconosMenu`. Esto tiene ventajas reales y puedes mencionarlas: se ven nítidos a cualquier tamaño (son vectores, no píxeles), **cambian de color según el estado** (gris normal, blanco al pasar el ratón, amarillo cuando está activo, rojo en el peligro) sin necesitar una imagen por cada color, no hay archivos que distribuir, y todos pertenecen a la misma familia visual porque comparten grosor, extremos y cuadrícula.

La cuadrícula de 24 × 24. El truco de diseño es que todos los iconos se dibujan en un lienzo imaginario de 24 × 24 unidades, y después se escalan al tamaño real:

```
public static void Dibujar(Graphics g, string nombre, RectangleF caja, Color
color)
{
    GraphicsState estado = g.Save();
    g.SmoothingMode = SmoothingMode.AntiAlias;
    g.TranslateTransform(caja.X, caja.Y);
    float k = caja.Width / 24f;
    g.ScaleTransform(k, k);

    using (Pen p = new Pen(color, 1.8f) { StartCap = LineCap.Round, EndCap =
LineCap.Round, LineJoin = LineJoin.Round })
    using (SolidBrush relleno = new SolidBrush(color))
    {
        switch (nombre) { ... }
    }
    g.Restore(estado);
}
```

Veamos qué hace cada pieza. `caja` es el rectángulo donde se quiere el icono (en el menú, 22 × 22 píxeles). `TranslateTransform` **mueve el origen** del lienzo a la esquina de esa caja: a partir de ahí, el punto (0, 0) es la esquina superior izquierda del icono, estemos donde estemos en el menú. `ScaleTransform(k, k)` **escala** todo por el factor `k = ancho / 24`: con un icono de 22 píxeles, `k` vale 0,92, y una línea dibujada de (3, 11) a (21, 11) se ve proporcionalmente reducida. Es lo que permite escribir las coordenadas pensando siempre en una hoja de 24 × 24 y que el icono funcione en cualquier tamaño. `g.Save()` y `g.Restore(estado)` guardan y devuelven el estado del lienzo (posición, escala, suavizado) para que estas transformaciones no afecten a lo que se dibuje después, como el texto de la opción.

El lápiz también está pensado: grosor 1,8 (que con la escala queda en unos 1,65 píxeles), `StartCap` y `EndCap` redondos para que los extremos de las líneas terminen en punta suave, y `LineJoin.Round` para que las esquinas donde se unen dos líneas sean redondeadas. Esos tres ajustes son los que dan a los iconos ese aspecto amable y uniforme.

Un icono, paso a paso: la casa. Mira cómo se arma el icono de inicio con tres trazos:

```
case "inicio":          // casa
    g.DrawLines(p, new[] { new PointF(3, 11), new PointF(12, 3), new
PointF(21, 11) });
    g.DrawLines(p, new[] { new PointF(5.5f, 9.5f), new PointF(5.5f, 20), new
PointF(18.5f, 20), new PointF(18.5f, 9.5f) });
    g.DrawLines(p, new[] { new PointF(10, 20), new PointF(10, 14), new
PointF(14, 14), new PointF(14, 20) });
    break;
```

`DrawLines` recibe una lista de puntos y los une uno tras otro con una línea, sin cerrar la figura. El primer trazo es el **techo**: parte de la izquierda (3, 11), sube al vértice (12, 3), que está justo en el centro horizontal del lienzo (la mitad de 24 es 12), y baja a la derecha (21, 11): un triángulo abierto, un «^». El segundo trazo son las **paredes**: baja desde el techo por la izquierda (5,5; 9,5) hasta el suelo (5,5; 20), cruza hasta (18,5; 20) y sube por la derecha. Observa que quedan abiertas por arriba: el techo las cubre. El tercero es la **puerta**: un «∩» cuadrado entre x = 10 y x = 14, de y = 20 a y = 14. Tres trazos, nueve puntos, y es una casa reconocible. Fijarse en que los números son simétricos respecto al centro (12) es lo que hace que el icono se vea equilibrado.

Los demás iconos usan las mismas herramientas combinadas de otra forma:

Icono	Qué dibuja	Con qué se dibuja
`activos`	Una caja en perspectiva (inventario)	`DrawPolygon` para el hexágono, más `DrawLines` y `DrawLine` para las aristas interiores
`personal`	Dos personas	`DrawEllipse` para cada cabeza y `DrawArc` (media elipse, de 180° a 360°) para cada par de hombros
`usuarios`	Una persona con un signo más	`DrawEllipse` y `DrawArc` para la persona; dos `DrawLine` cruzadas para el «+»
`actividad`	Un reloj	`DrawEllipse` para la esfera y dos `DrawLine` para las manecillas
`planillas`	Un documento con renglones	`DrawRectangle` y tres `DrawLine` de distinto largo
`salones`	Una pizarra	`DrawRectangle`, una línea de texto y dos líneas para el soporte

Icono	Qué dibuja	Con qué se dibuja
`catalogo`	Un billete	`DrawRectangle` con un `DrawEllipse` en el centro
`reportar`	Un triángulo de advertencia	`DrawPolygon`, una `DrawLine` para el trazo del «!» y un `FillEllipse` para su punto
`ayuda`	Un círculo con una interrogación	`DrawEllipse` y `DrawString` para el «?»
`salir`	Una puerta con una flecha	`DrawLines` para el marco, una `DrawLine` y un `DrawLines` para la flecha
`menu`	Tres rayitas	Tres `DrawLine` horizontales

Dos detalles que sirven para explicar la técnica. `DrawArc(p, 2.5f, 13.5f, 11, 11, 180, 180)` dibuja un **arco**: los dos primeros números son la esquina del rectángulo que contiene a la elipse completa, los dos siguientes su tamaño, y los dos últimos el ángulo de inicio y cuánto se recorre. En GDI+ los ángulos se miden desde el este y **giran en el sentido de las agujas del reloj**, así que partir de 180° (oeste) y recorrer 180° pasa por el norte (270°) y termina en el este: dibuja solo la mitad de arriba de la elipse, que es justo la forma de unos hombros. Y el icono `reportar` usa dos herramientas distintas a propósito: el trazo del «!» es una línea (`Pen`), pero el punto de abajo es un círculo **relleno** (`FillEllipse` con el pincel), porque un punto de 1,8 unidades dibujado con contorno se vería hueco.

Cómo se elige y se usa un icono. El control `ItemMenuLateral` tiene una propiedad de texto, `Icono`, con el nombre del icono («inicio», «activos»…). Al pintarse, el control llama a `IconosMenu.Dibujar(g, icono, new RectangleF(xIcono, yIcono, 22, 22), colIcono)`: le pasa el nombre, el rectángulo y el color que corresponde al estado. El `switch` interno dibuja la forma. **Para agregar un icono nuevo** (digamos «reportes») basta con añadir un `case "reportes":` con sus trazos en la cuadrícula de 24 × 24, y escribir ese nombre en la propiedad `Icono` del ítem. No hay que tocar nada más.

9.4 El ítem del menú completo: estados, insignia y modo compacto

`ItemMenuLateral` hereda de `Control`, el control más básico, y **dibuja todo él mismo** en su `OnPaint`: fondo, barra de acento, icono, texto e insignia. Su clase empieza con un comentario que lo resume: «dibuja ella sola su icono, texto, estado activo, efecto del mouse e insignia». Veamos el orden en que lo hace, porque cada capa se dibuja encima de la anterior.

1. **Se borra con el color del padre** (`g.Clear(Parent.BackColor)`), para que el control se integre con el panel donde está.
2. **Fondo redondeado** si está activo o con el ratón encima: un rectángulo con esquinas de radio 8, relleno con un color distinto según sea activo, hover o hover de peligro (rojizo, para «Cerrar sesión»).
3. **Barra amarilla** a la izquierda, solo si está activo: un rectángulo muy fino y alto con las esquinas redondeadas. Es la marca visual de «aquí estás».
4. **Se calculan los colores** con una cadena de decisiones: si está deshabilitado, gris apagado; si está activo, amarillo para el icono y blanco para el texto; si está el ratón encima, blanco (o rojo si es de peligro); si no, el gris normal.
5. **Se dibuja el icono**, a la izquierda o centrado si el menú está plegado.
6. **Se dibuja el texto** con `TextRenderer.DrawText`, con las opciones `Left | VerticalCenter | EndEllipsis | NoPadding`: alineado a la izquierda, centrado en vertical y, si no cabe, termina con «…» en lugar de cortarse.
7. **La insignia**: si `Insignia > 0`, dibuja una cápsula con el número (o «99+»). Mide primero el texto con `TextRenderer.MeasureText` para que la cápsula sea tan ancha como haga falta (mínimo 22). En **modo compacto**, en cambio, dibuja solo un punto de color en la esquina del icono, con un contorno del color del fondo que lo separa visualmente del icono.

Lo que hace que esto funcione sin escribir una línea más es que el aspecto es una **función del estado**. El control guarda sus estados en campos (`activo`, `hover`, `peligro`, `compacto`, `insignia`), expone propiedades cuyo `set` llama a `Invalidate()`, y `OnPaint` simplemente lee esos campos y dibuja lo que corresponde. El ratón entra → `hover = true` + `Invalidate()` → se repinta con el fondo de hover. Nadie dice «cambia el color»: se cambia el estado y el dibujo lo refleja. Es el mismo principio de las aplicaciones modernas.

Otro detalle de oficio: las propiedades llevan atributos como `[Category("Menú"), DefaultValue(false)]`. Con ellos, el control aparece en el diseñador de Visual Studio con sus propiedades agrupadas bajo «Menú», y el diseñador sabe cuál es el valor por defecto para no escribirlo en el código generado. Y para el color de la insignia, `ShouldSerializeColorInsignia` y `ResetColorInsignia` son métodos con nombre especial que el diseñador reconoce para decidir cuándo guardar o restablecer esa propiedad. Por último, el control crea sus fuentes una sola vez y las libera en `Dispose`, para no dejar recursos sueltos.

La pieza que dibuja los fondos redondeados merece su propia explicación, porque se repite en el menú, el login, las cápsulas de las tablas y las barras de desplazamiento:

```
private static GraphicsPath Redondeado(Rectangle r, int radio)
{
    int d = Math.Max(1, radio * 2);
    GraphicsPath p = new GraphicsPath();
    p.AddArc(r.X, r.Y, d, d, 180, 90);                  // esquina superior
izquierda
    p.AddArc(r.Right - d, r.Y, d, d, 270, 90);          // esquina superior
derecha
    p.AddArc(r.Right - d, r.Bottom - d, d, d, 0, 90);   // esquina inferior
derecha
    p.AddArc(r.X, r.Bottom - d, d, d, 90, 90);          // esquina inferior
izquierda
    p.CloseFigure();
    return p;
}
```

Un rectángulo con esquinas redondeadas se construye con cuatro **cuartos de círculo** unidos. Cada `AddArc` agrega un arco de 90° dentro de un cuadrado de lado `d = 2 × radio` ubicado en cada esquina; GDI+ une los arcos con líneas rectas automáticamente, y `CloseFigure` cierra la forma. Los ángulos de inicio (180, 270, 0, 90) van recorriendo las esquinas en el sentido del reloj, empezando por la superior izquierda. El resultado se pinta con `FillPath` (relleno) o `DrawPath` (contorno).

9.5 Los controles del tema: una familia de clases propias

Para que el aspecto sea el mismo en todas las pantallas, el proyecto no estiliza cada botón o cada tabla por separado. Crea **clases propias que heredan de los controles estándar** y ya traen el estilo puesto. Después, en el diseñador, se usa `GridOscuro` en lugar de `DataGridView`, y la tabla nace oscura. Todas viven en `ControlesTema.cs` y archivos vecinos.

Clase	Hereda de	Qué cambia
`BotonTema`	`Button`	Tres tipos (principal amarillo, secundario gris, peligro rojo), colores para hover y clic, deshabilitado apagado
`PanelTarjeta`	`Panel`	Fondo de tarjeta y un borde fino dibujado

Clase	Hereda de	Qué cambia
`GridOscuro`	`DataGridView`	Colores, filas alternas, resaltado al pasar el ratón, franja amarilla en la fila seleccionada, barras de desplazamiento propias
`ComboOscuro`	`ComboBox`	Caja, flecha y lista desplegable oscuras
`FechaOscura`	`DateTimePicker`	Campo, flecha y calendario oscuros
`TabControlOscuro`	`TabControl`	Pestañas dibujadas con la paleta

`BotonTema` es el caso más simple y muestra la idea de «el aspecto es una función del estado». Tiene una propiedad `Tipo` (un `enum`: `Principal`, `Secundario`, `Peligro`) y un método `Aplicar()` que elige colores según el tipo y si está habilitado. Cuando está deshabilitado se vuelve gris apagado y el cursor deja de ser la mano. Los colores de hover y de clic se calculan con una función pequeña, `Aclarar`, que suma o resta un valor a cada componente rojo, verde y azul sin salirse del rango 0 a 255 (`Math.Max(0, Math.Min(255, c.R + delta))`): el hover es un poco más claro y el clic un poco más oscuro que el color base, de modo que no hace falta declarar tres colores por cada tipo.

`GridOscuro` es el control más trabajado, y vale la pena conocer sus trucos porque son los que hacen que las tablas se sientan modernas:

- **El estilo base** se fija en el constructor: sin encabezados de fila, selección de fila completa, solo lectura, filas de 36 píxeles, encabezado de 40, y dos colores alternados para las filas (`FilaA`, `FilaB`) que facilitan seguir una fila con la vista.
- **Resaltado al pasar el ratón.** El grid guarda en `filaHover` qué fila está bajo el cursor, la actualiza en `OnCellMouseEnter` y `OnCellMouseLeave`, y pide repintar solo las dos filas afectadas (`InvalidateRow`), no toda la tabla. Después, `OnCellFormatting` (que Windows llama antes de pintar cada celda) cambia el color de fondo de la fila que está en hover, siempre que no esté seleccionada.
- **La franja amarilla de la selección.** `OnRowPostPaint` se ejecuta después de pintar cada fila; si está seleccionada, dibuja encima un rectángulo amarillo de 3 píxeles en el borde izquierdo. Y en `OnPaint` se traza una línea amarilla de 2 píxeles bajo el encabezado, con algo de transparencia.
- **Barras de desplazamiento delgadas.** Las barras nativas de Windows son anchas y claras, rompen el tema oscuro. En lugar de intentar pintarlas, el grid **coloca encima unas barras propias**, instancias de `BarraFina`, que copian la posición y el tamaño de las nativas (`Sincronizar`) y las tapan. Se crean en `OnHandleCreated`, no en el

constructor, y solo fuera del modo de diseño. La sincronización se programa con `BeginInvoke` para ejecutarse justo después de que el grid termine su propio diseño (`OnLayout`, `OnScroll`, `OnRowsAdded`…).

`BarraFina` es una barra de desplazamiento escrita desde cero, y tiene una pequeña lógica matemática que puedes explicar. El **tamaño del pulgar** (la parte que se arrastra) es proporcional a cuánto del contenido se ve: `largo = pista × visible ÷ total`, con un mínimo de 28 píxeles para que siempre se pueda agarrar. La **posición** es proporcional a cuánto se ha desplazado: `inicio = 4 + (pista - largo) × posición ÷ rango`. Al arrastrar con el ratón se hace la cuenta inversa, de posición del puntero a fila inicial del grid. Dibuja el pulgar como una pastilla redondeada (la misma técnica de `AddArc`) que engrosa de 6 a 8 píxeles al pasar el ratón y cambia de tono al arrastrarlo.

`ComboOscuro` y `FechaOscura`: cuando el control no deja que lo dibujes. El `ComboBox` y el `DateTimePicker` son controles que Windows dibuja por su cuenta con el tema del sistema, y no ofrecen un `OnPaint` que se pueda sobrescribir con éxito. La solución es más profunda: interceptar los **mensajes de Windows**.

```
protected override void WndProc(ref Message m)
{
    base.WndProc(ref m);
    if (m.Msg == WM_PAINT) Dibujar();
}
```

Todo en Windows funciona con mensajes numerados que el sistema le envía a cada ventana; `WM_PAINT` (valor 0x000F) significa «dibújate». `WndProc` es el método que recibe todos esos mensajes. Aquí se deja primero que el control haga su dibujo normal (`base.WndProc`) y **después, encima, se pinta el estilo propio**: se rellena la caja con el color del tema, se dibuja el borde (amarillo si tiene el foco), se escribe el texto elegido y se traza la flecha con tres puntos unidos por `DrawLines`. Para la lista desplegable del combo, que sí permite dibujo propio, se usa `DrawMode.OwnerDrawFixed` y se sobrescribe `OnDrawItem` para pintar cada línea con la paleta. Para el calendario de `FechaOscura` hay que ir aún más abajo: se le pide a Windows el manejador del calendario (`SendMessage` con `DTM_GETMONTHCAL`) y se le quita el tema visual con `SetWindowTheme(cal, "", "")`, porque con los estilos visuales activos el calendario ignora los colores que se le asignan.

Esas llamadas (`SendMessage`, `SetWindowTheme`) no son de .NET sino de las bibliotecas del propio Windows (`user32.dll`, `uxtheme.dll`). Se invocan con `[DllImport("user32.dll")]`, una técnica llamada **P/Invoke**. Es una de las pocas veces en el proyecto donde se sale del mundo de .NET, y por eso está acotada en métodos privados

de pocas líneas. Una clase aparte, `BarrasOscuras.Aplicar(control)`, usa la misma llamada para aplicar el tema oscuro nativo (`"DarkMode_Explorer"`) a las barras de desplazamiento de controles que no son tablas.

Un último ayudante, `NumericoLimitado.Aplicar`, resuelve un defecto del `NumericUpDown`: deja **escribir** un número mayor que su máximo aunque las flechas no lo permitan. El ayudante escucha `TextChanged`, y si lo escrito supera el máximo, lo vuelve a poner en el máximo y marca el campo con un aviso. Una bandera local, `ajustando`, evita que el propio ajuste vuelva a disparar el evento y entre en un bucle: es el mismo truco de la bandera `cargando` de la sección 6.

9.6 Pintar dentro de las tablas: colores por regla y cápsulas

Hay dos eventos de la tabla para cambiar el aspecto de una celda, y es importante no confundirlos.

`CellFormatting` se ejecuta antes de pintar cada celda y permite **cambiar su estilo** (colores, fuente, formato) manteniendo el dibujo normal. Lo usa la pantalla de préstamos para colorear filas según la columna `Alerta`:

```
if (alerta == "Atrasado")      { e.CellStyle.BackColor = Color.FromArgb(72,
32, 42);  e.CellStyle.ForeColor = Color.FromArgb(252, 165, 165); }
else if (alerta == "Por vencer") { e.CellStyle.BackColor = Color.FromArgb(66,
50, 18);  e.CellStyle.ForeColor = Color.FromArgb(252, 211, 77); }
```

Los colores son versiones **oscuras y apagadas** del rojo y del ámbar para el fondo, con letra clara del mismo tono: se ve la alerta sin que la tabla se vuelva un arcoíris. Es la forma sencilla de personalizar.

`CellPainting` reemplaza el dibujo de la celda **por completo**, y es lo que se usa para las cápsulas de estado («Activo» en verde, otro en ámbar). La lógica tiene cuatro pasos:

```
e.Paint(e.CellBounds, e.PaintParts &
~DataGridViewPaintParts.ContentForeground); // 1. pinta fondo y borde, sin el
texto
e.Handled = true;
// 2. "yo me encargo del resto"
...
using (GraphicsPath p = Pastilla(r))
// 3. forma de cápsula
using (SolidBrush b = new SolidBrush(Color.FromArgb(55, baseColor)))
//    relleno semitransparente
    e.Graphics.FillPath(b, p);
TextRenderer.DrawText(e.Graphics, estado, f, r, letra, ...);
// 4. el texto encima
```

Primero se le pide a la celda que pinte todo salvo el texto (el operador `& ~` quita esa parte de las que se pintan); después se marca `e.Handled = true` para avisar que el resto lo hace el programa. Se mide el texto con `MeasureText` para que la cápsula tenga el ancho justo (texto más 22 píxeles), y se arma con `Pastilla`: dos medios círculos (`AddArc` de 180° a cada lado) unidos, lo que da los extremos totalmente redondos. El relleno usa `Color.FromArgb(55, baseColor)`, es decir, **el color con un nivel de opacidad de 55 sobre 255**: queda un tono suave que se mezcla con el fondo de la fila, y el texto va en una versión más clara del mismo color para mantener el contraste.

9.7 El inicio: tarjetas, alertas y gráficos

La pantalla de inicio combina tres tipos de elementos, cada uno armado de manera distinta.

Las tarjetas de indicadores son seis paneles dentro de un `TableLayoutPanel` (`tlpKpis`), una rejilla que reparte el ancho en columnas. Cada tarjeta tiene dos etiquetas, el número grande y su detalle, y un ayudante, `Kpi(valor, detalle, textoValor, textoDetalle, colorDetalle)`, que las llena. El **color transmite significado** de forma consistente: verde para «todo bien», ámbar para «atención», rojo para «problema», gris para neutro. Si el usuario no es administrador, `ConfigurarSegunRol` oculta las dos últimas tarjetas y reparte el ancho en cuatro columnas del 25 % para que no queden huecos.

Las alertas son tarjetas que se construyen en `AgregarAlerta` **por código, en el momento**, según lo que haya: solo aparece la de préstamos vencidos si los hay, y lo mismo con reportes urgentes. Cada alerta es un `Panel` con una barra de color de 5 píxeles a la izquierda (`Dock = Left`), dos etiquetas (título en negrita y detalle en gris) y, si lleva

destino, un «Abrir ›» a la derecha. Lo ingenioso es el **efecto de hover**: para que toda la tarjeta se ilumine al pasar el ratón y sea clicable, se conectan los mismos tres manejadores de eventos (clic, entrar, salir) a **todos sus controles hijos** con un `foreach`; y al salir se comprueba con `p.ClientRectangle.Contains(...)` si el cursor realmente salió de la tarjeta o solo pasó de un hijo a otro, para que no parpadee. Al hacer clic, la alerta le pide al panel principal que navegue: `panel.BeginInvoke(() => panel.IrA(seccion))`.

Los gráficos usan el control `Chart` de `System.Windows.Forms.DataVisualization`. No es dibujo propio sino configuración minuciosa. `EstilizarChart` aplica el tema: fondo del color de tarjeta, título en blanco a la izquierda, ejes y etiquetas en gris suave, sin líneas de cuadrícula verticales ni marcas. Cada gráfico elige su tipo (`Bar` para barras horizontales, `Column` para columnas), ancho de barra (`PointWidth`), y muestra el valor sobre cada barra (`IsValueShownAsLabel`). Dos detalles de cuidado: en las barras horizontales se invierte el eje (`AxisX.IsReversed = true`) para que el elemento con más valor quede **arriba** y no abajo; y si una consulta no devuelve datos, `SinDatos` agrega una anotación de texto en el centro, «Sin datos para mostrar», en lugar de dejar un gráfico vacío e intrigante.

9.8 El login: detalles que se ven

El login es la primera pantalla que ve quien abre el sistema, y concentra varios trucos de diseño:

- **La tarjeta redondeada.** El panel central tiene las esquinas redondeadas porque se le asigna una **región** (`pnlTarjeta.Region = new Region(Redondeado(...))`), que recorta el panel con esa forma. Encima, `pnlTarjeta_Paint` dibuja un borde fino de color amarillo semitransparente (`Color.FromArgb(130, 250, 204, 21)`) siguiendo la misma forma, y se centra en la ventana en cada cambio de tamaño.
- **Campos con línea inferior.** En lugar del cuadro con borde de siempre, cada campo es un `TextBox` sin borde dentro de un panel que dibuja **una línea de 2 píxeles en la parte baja**: amarilla si el campo tiene el foco, roja si tiene un error, gris en reposo. Un solo manejador `Campo_Paint` sirve para los dos campos, y distingue cuál es con `ReferenceEquals(sender, ...)`. Es el estilo «material» de formularios minimalistas.
- **El ojo para ver la contraseña** es otro icono dibujado por código: dos curvas **Bézier** (una arriba y otra abajo, que juntas forman la silueta del ojo), un círculo relleno para la pupila y, si la contraseña está visible, una diagonal que lo tacha. `DrawBezier` recibe cuatro puntos: el inicio, dos puntos de control que «jalan» la curva y el final.

- **Aviso de Bloq Mayús.** Un `Label` se muestra solo cuando `Control.IsKeyLocked(Keys.CapsLock)` es verdadero y el foco está en la contraseña. Es uno de esos detalles que nadie nota hasta que falta.
- **Contra el parpadeo.** El formulario sobrescribe `CreateParams` y activa el estilo extendido `0x02000000` (`WS_EX_COMPOSITED`), que le dice a Windows que dibuje toda la ventana y sus hijos de una vez en un búfer. Junto con el doble búfer, la ventana se redibuja sin parpadear al cambiar de tamaño.

9.9 Diálogos construidos por completo en código

Algunas ventanas pequeñas no tienen archivo de diseño: se construyen enteras en código. La base es `DialogoBase`, una clase que hereda de `Form` y entrega ya configurada la ventana oscura (`FixedDialog`, centrada, sin maximizar, fuente Segoe UI, colores de la paleta) más cuatro **métodos de fábrica**: `Etiqueta(...)`, `Caja(...)`, `Boton(...)` y `LineaError(...)`. Cada uno crea un control con el estilo del sistema, lo agrega a la ventana y lo devuelve. Los diálogos concretos (`dlgUsuario`, `dlgClaveTemporal`, `dlgCambiarClave`) solo dicen qué contiene cada uno, en pocas líneas, y heredan todo el aspecto. Es el patrón «clase base con ayudantes»: el estilo se escribe una vez. `ExportadorPdf.ElegirFormato` y el menú «Más acciones» (un `ContextMenuStrip` con un `ProfessionalColorTable` propio para los colores) siguen la misma filosofía.

9.10 Cómo explicar el diseño y sus límites

Si te piden que expliques el diseño, puedes ordenarlo en cuatro ideas que se apoyan en lo que ya viste. **Coherencia**: una paleta única y un solo acento, usados en todas las pantallas. **Reutilización**: controles propios (`BotonTema`, `GridOscuro`…) y clases base (`DialogoBase`) en lugar de estilizar cada cosa a mano. **Estados**: todos los controles responden al ratón, al foco, al error y a estar deshabilitados, y lo hacen cambiando de estado y repintando. **Retroalimentación**: colores con significado (verde, ámbar, rojo), mensajes claros, insignias, íconos de error junto al campo.

Y, como en todo, conviene conocer los límites. El diseño depende de Windows: la fuente Segoe UI viene con el sistema, las barras oscuras usan temas de Windows 10 en adelante y el dibujo sobre `WM_PAINT` es propio de esta plataforma. Los formularios usan tamaños fijos en píxeles (`AutoScaleMode.None` en los diálogos), por lo que no se adaptan a pantallas con escalado de DPI distinto con la misma soltura. Algunas tablas pueden mostrar la última fila cortada si la altura disponible no es múltiplo de la altura de las filas, un detalle estético conocido de `DataGridView`. Y los colores repetidos en varias pantallas

en lugar de leerse todos de `PaletaSistema` son una deuda menor de orden. Ninguno compromete el funcionamiento, y mencionarlos con naturalidad muestra que mirar el diseño con ojo crítico es parte de dominarlo.

10. Tres flujos de punta a punta en C#

Las secciones anteriores desarmaron las piezas una por una. Esta las vuelve a juntar, porque la mejor prueba de que entiendes un sistema es poder **seguir un dato desde el clic hasta la base y de vuelta a la pantalla**, sin saltarte ningún paso. Elegí tres recorridos que, entre los tres, tocan casi todo lo importante: entrar al sistema (seguridad y sesión), registrar un préstamo (SQL, reglas, auditoría y color en la tabla) y generar la planilla del mes (simulación, transacción y funciones). Cada uno lo cuento en el orden en que realmente ocurre, con el nombre del método donde pasa.

10.1 Flujo 1: de escribir la contraseña a ver el panel

Todo empieza en `btnIngresar_Click` de `frmLogin`. Los pasos, en orden:

1. **Recoge y limpia lo escrito** con `Trim()`. Si falta el usuario, la contraseña o ambos, activa las banderas `errUsuario` / `errClave` (el formulario las lee al pintar para dibujar el subrayado en rojo), muestra el mensaje adecuado, pone el foco en el campo que falta y hace `return`. Es una **cláusula de guarda**: lo inválido se descarta al principio para que el resto del método pueda asumir que los datos existen.
2. **Revisa si el usuario está bloqueado.** Convierte el nombre a minúsculas (`ToLowerInvariant()`) para usarlo como llave del diccionario `bloqueadoHasta`. Si hay una hora futura guardada, calcula los segundos que faltan con `Math.Ceiling((hastaCuando - DateTime.Now).TotalSeconds)`, limpia la contraseña, avisa y sale. **En este punto todavía no se tocó la base de datos**: un atacante bloqueado no consume consultas.
3. **Pone el botón en «Ingresando...»** y llama a `Refresh()` para que Windows lo repinte de inmediato (el hilo de la interfaz está a punto de quedar ocupado esperando a SQL Server y, si no se fuerza, el usuario no vería el cambio). El `try / finally` garantiza que el botón vuelva a su estado normal pase lo que pase.
4. **Llama a `ValidarCredenciales(usuario, clave)`**, que ejecuta la consulta parametrizada con `INNER JOIN Roles`, lee la fila con un `SqlDataReader` y decide: si el valor guardado empieza con `$2` verifica con BCrypt; si no, usa la rama de migración de texto plano (sección 7.2). Devuelve el nombre del rol si todo salió bien, o `null` si algo falló. Además deja dos "mensajes" en campos del formulario: `motivoRechazo`

(`inactivo` , `no_admin` o `null`) y `conexionFallida` . Es un truco de estilo: el método solo puede devolver un valor, así que los detalles del rechazo viajan en variables del propio formulario.

5. **Si hay rol**, es un ingreso válido: borra los contadores de fallos y de bloqueo de ese usuario, llena la clase estática `Sesion` (`Usuario` , `Rol` , `NombreCompleto`) y llama a `RegistrarAcceso` , que ejecuta `sp_RegistrarAcceso` . Esa llamada está envuelta en un `try / catch` **vacío a propósito**: dejar constancia del acceso es importante, pero si la bitácora fallara no debe impedir que alguien legítimo entre.
6. **Revisa si la clave es temporal** con `ServicioUsuarios.DebeCambiarClave` . Si lo es, abre `dlgCambiarClave` como diálogo modal. Si el usuario lo cierra sin cambiarla, se **deshace la sesión** (`Sesion.Usuario = null` , etc.), se vacía la contraseña, se avisa y no pasa nada más. Fíjate en que `Sesion` ya se había llenado en el paso anterior y aquí se revierte: por eso conviene leer el flujo completo antes de afirmar «la sesión se crea cuando la clave es correcta».
7. **Oculta el login y abre el panel**: `this.Hide()` y luego `new frmDashboard().ShowDialog()` dentro de un `using` (para liberar el formulario al terminar) y de un `try / catch` que muestra «Error Crítico» si algo falla al construirlo. `ShowDialog` **bloquea** en esa línea hasta que el panel se cierre. Cuando regresa, el código mira `Sesion.Rol` : si quedó vacío, el usuario cerró sesión desde el panel, así que se limpian los campos y se vuelve a mostrar el login; si no, se cierra el login y con él termina la aplicación.

El camino de fallo es igual de instructivo. Si `ValidarCredenciales` devolvió `null` **y** la conexión no falló, se registra el motivo exacto en la bitácora («Intento fallido», «Intento con cuenta desactivada», «Intento sin permiso»), se limpia la contraseña, se marcan ambos campos en rojo y se escoge el mensaje. Solo si el motivo es `null` (o sea, contraseña equivocada) se suma un fallo; al quinto se borra el conteo, se guarda la hora de desbloqueo y el mensaje cambia a «Espere 60 segundos». Si lo que falló fue la conexión, `ValidarCredenciales` ya mostró su propio cuadro de error y este bloque **no** cuenta ni registra nada: un servidor apagado no debe castigar al usuario con un bloqueo.

Una observación que te conviene tener lista: los diccionarios `fallosPorUsuario` y `bloqueadoHasta` son `static` y viven **en memoria**. Cerrar y volver a abrir la aplicación borra el bloqueo. Para un proyecto escolar es una decisión razonable y se puede explicar con naturalidad; en un sistema real el contador se guardaría en la base para que sobreviva a los reinicios y funcione con varios equipos.

10.2 Flujo 2: registrar un préstamo de equipo

Este es el flujo que mejor muestra cómo C# y SQL **viven juntos**. Cruza cuatro capas: la pantalla (`frmGestionActivos`, en su archivo parcial `frmGestionActivosPrestamos.cs`), la clase `Datos`, el procedimiento `sp_RegistrarPrestamo` con sus tablas y triggers, y la vista `vw_Prestamos` que alimenta la tabla de abajo.

Antes del clic: preparar el formulario. `InicializarPrestamos` llena los combos fijos, crea por código dos controles que el diseñador no tiene (`CrearSelectorEmpleado` y `CrearSelectorUnidad`: dos `ComboOscuro` puestos en la misma posición y tamaño que un cuadro de texto, que queda oculto) y deja todo con la bandera `cargando = true` mientras tanto, para que los eventos `SelectedIndexChanged` no disparen consultas a medio armar. Lo interesante son los dos selectores:

- **El de empleado** se alterna con el cuadro de texto libre según el tipo de responsable. `AplicarTipoResponsable` decide con una sola comparación: si el tipo es «Docente» o «Personal administrativo» muestra el combo y oculta el texto; si es «Alumno», al revés. Cada vez que se muestra el combo vuelve a pedir la lista de empleados a la base (`CargarEmpleadosPrestamo`), por si alguien fue dado de baja o agregado mientras la pantalla estaba abierta. La consulta calcula en SQL una columna `EsDocente` con `CASE WHEN LOWER(ISNULL(c.NombreCargo, '')) LIKE 'docente%' THEN 1 ELSE 0 END`, y el C# solo recorre las filas y mete en el combo las que coinciden con el tipo elegido.
- **El de unidad** (`CargarUnidadesPrestamo`) ofrece **solamente lo que sí se puede prestar**. Su consulta une `UnidadesActivo` con `Salones` y `Estados`, filtra por el equipo elegido (`WHERE u.IdActivo = @a`, con el parámetro `Datos.P("@a", idActivo)`), excluye los estados no prestables con `NOT IN (...)` y descarta con `NOT EXISTS` las unidades que ya tienen un préstamo activo. El texto de cada opción se arma en SQL concatenando código, salón y estado. Después, el C# conserva la elección anterior si sigue disponible y, si queda una sola, la selecciona sola.

Los elementos de ambos combos son objetos `ElementoLista` (`Id`, `Texto`, `Extra`) que sobrescriben `ToString()` para que el combo muestre el texto; así la pantalla muestra «Aula 3 · Proyector · Bueno» pero el código recupera el número con `IdSeleccionado(combo)`, que hace `combo.SelectedItem as ElementoLista` y devuelve un `int?` (nulo si no hay selección). Es el mismo patrón que en `Datos.P`: el valor ausente viaja como `null` y se convierte en `DBNull` justo antes de llegar a SQL.

El clic: `btnPrestar_Click`. Primero limpia el `ErrorProvider` y valida en pantalla: equipo, unidad, responsable (del combo o del texto, según `cmbPrestEmpleado.Visible`) y que la fecha límite sea posterior a ahora. Cada error se marca junto a su control con `SetError` y

se acumula en `error`; al final, `if (error) return;`. Esta validación es **de cortesía**: avisa rápido y sin viaje a la base. No es la defensa real, como verás enseguida.

Si todo pasa, arma los parámetros y llama a `Datos.Procedimiento("sp_RegistrarPrestamo", ...)`. Si es un empleado, manda su `IdEmpleado` y el nombre sale de `ElementoLista.Extra`; si es alumno, el `IdEmpleado` va como `null`. El aula opcional se manda como `aula > 0 ? (object)aula : null`; ese `(object)` es necesario porque el operador `?:` exige que ambos lados tengan el mismo tipo y `int` no puede convivir con `null`.

En SQL Server: `sp_RegistrarPrestamo`. Aquí ocurre la validación que de verdad protege los datos. En orden:

1. Si el tipo es Docente o Personal administrativo, exige `@IdEmpleado` (error 50950), comprueba que el empleado exista y no esté dado de baja (50951), y que su cargo coincida con el tipo (50952 y 50953). Después hace `SET @Responsable = LEFT(@NombreEmp, 100)`: **el nombre del responsable sale del registro del empleado, no de lo que mandó la pantalla**. La base no confía en el texto que le llega.
2. Exige un responsable no vacío (50031) y una fecha límite futura (50032).
3. Busca la unidad: si no existe, 50033; si su estado es «Dado de baja», «En reparación», «Malo» o «Mantenimiento Pendiente», 50034; si ya tiene un préstamo activo, 50035.
4. Recién entonces hace el `INSERT` en `Prestamos`.

Fíjate en que el procedimiento **repite** reglas que la pantalla ya había aplicado al filtrar el combo. No es redundancia por descuido, es defensa en profundidad, y se justifica de tres maneras: entre que la pantalla cargó la lista y el usuario hizo clic otra persona pudo prestar esa misma unidad; cualquier otra pantalla o script que llame al procedimiento queda igual de protegido; y, como respaldo final, existe el índice único filtrado `UX_Prestamos_UnidadActiva` (`UNIQUE ... WHERE Estado = 'Activo'`), que haría fracasar un segundo préstamo activo de la misma unidad aunque dos personas pasaran el `IF EXISTS` al mismo tiempo.

El trigger. Al insertarse la fila se dispara `trg_Aud_Prestamos`, que escribe en `Auditoria` el JSON del registro y el usuario tomado de `SESSION_CONTEXT(N'Usuario')`. Ese valor lo puso C# sin que la pantalla de préstamos lo supiera: cada conexión que abre `Datos` ejecuta `sp_set_session_context` en su evento `StateChange` con el nombre de `Sesion.Usuario`. Por eso la bitácora sabe quién prestó el equipo aunque el `INSERT` no mencione al usuario.

Vuelta a C#. Si el procedimiento lanza un error, llega como `SqlException` y esta pantalla lo atrapa con un `catch (Exception ex)` genérico que muestra `ex.Message` en un aviso «No se pudo registrar la salida». Funciona porque los mensajes de los `THROW` ya están escritos en lenguaje de persona. (La pantalla de pagos va un paso más allá: lee `ex.Number` y marca el error junto al campo exacto; ambas formas son válidas.) Si todo salió bien, muestra el éxito, limpia los campos, restablece la fecha límite a mañana y llama a `CargarPrestamos()` y `ActualizarIndicadores()`.

Cómo se vuelve a ver el resultado. `CargarPrestamos` elige el filtro con un `switch` sobre el texto del combo de vista («Atrasados», «Por vencer», «Devueltos», «Todos», o por defecto «Activos») y lo concatena en la consulta: `FROM vw_Prestamos WHERE " + donde + " ORDER BY CASE Alerta ...`. Esa concatenación es segura porque `donde` solo puede ser una de cinco constantes escritas en el código, nunca texto del usuario. La vista `vw_Prestamos` es la que calcula la columna `Alerta` con `GETDATE()`: «Devuelto» si ya volvió, «Atrasado» si el límite ya pasó, «Por vencer» si vence en menos de dos días, «En curso» en otro caso. El `ORDER BY CASE` pone primero lo urgente. Luego `dgvPrestamos_CellFormatting` lee esa columna fila por fila y pinta de rojo oscuro los atrasados y de ámbar los que están por vencer. **La lógica de negocio (qué es «atrasado») está en SQL; el color está en C#.** Una segunda consulta con `SUM(CASE WHEN Alerta = ... THEN 1 ELSE 0 END)` alimenta la etiqueta «N atrasados · M por vencer».

Los otros dos botones de la misma pestaña siguen el mismo esquema. `btnDevolver_Click` valida que haya una fila seleccionada y que no esté ya devuelta, pide confirmación con un `MessageBox` de Sí/No y llama a `sp_RegistrarDevolucion`, que sí usa una transacción con `WITH (UPDLOCK)`: marca el préstamo como devuelto y cambia el estado de la unidad en un solo bloque que sale completo o no sale. Si el equipo vuelve en mal estado, el aviso recuerda registrar el mantenimiento y se recarga también la lista de activos. `btnHoja_Click` consulta `SELECT * FROM vw_Prestamos WHERE IdPrestamo = @id` y le pasa la **fila** (`DataRow`) a `HojaResguardoPdf.Generar`, que dibuja el PDF para la firma.

10.3 Flujo 3: generar la planilla del mes

Este flujo enseña un patrón muy elegante: **la simulación**. `frmGenerarPlanilla` es un formulario hecho íntegramente por código (hereda de `Form` y no tiene archivo de diseñador), que se abre como diálogo desde la gestión de planillas.

Armar el selector de mes. `Iniciar` crea un `DataTable` vacío con dos columnas (`Fecha` y `Texto`), calcula el primer día del mes actual y, con un `for` de 12 vueltas, agrega ese mes y los 11 anteriores usando `actual.AddMonths(-i)`. El texto sale de `m.ToString("MMMM yyyy", Es)`, con una cultura española para que diga «octubre 2026» y no «October 2026», y se le pone la primera letra en mayúscula con `char.ToUpper(s[0]) + s.Substring(1)`. Se enlaza al combo con `DisplayMember = "Texto"` y `ValueMember = "Fecha"`: el usuario ve el nombre del mes, el código recupera una fecha real. No hay meses futuros porque ni siquiera entran a la lista. Observa también el orden de las últimas líneas: primero se elige el mes inicial, luego se **suscribe** el evento (`cmbMes.SelectedIndexChanged += (s, e) => VistaPrevia();`), luego se apaga `cargando` y recién entonces se llama a `VistaPrevia()` una vez a mano. Si se suscribiera antes, el propio llenado del combo dispararía consultas innecesarias.

La vista previa. `VistaPrevia` ejecuta `EXEC dbo.sp_GenerarPlanilla @Periodo = @p, @Usuario = @u, @Simular = 1` mediante `Datos.Tabla`, con los parámetros pasados por separado. El procedimiento, al ver `@Simular = 1`, calcula todo lo que haría y **no inserta nada**: devuelve una única fila con `Generados`, `YaExistian`, `SinSalario`, `NombresSinSalario` y `Cerrado`. El C# toma `.Rows[0]`, convierte con `Convert.ToInt32`, arma el texto («Se generarán: 12 pagos…»), lo pinta en rojo si el mes está cerrado y habilita el botón solo si `!cerrado && gen > 0`. Dentro del procedimiento hay una decisión sutil: los `THROW` por mes futuro (50802) y mes cerrado (50800) están dentro de `IF @Simular = 0`. Así la vista previa **puede mostrar** «este mes está CERRADO» como información en lugar de explotar con una excepción.

El cálculo. El procedimiento normaliza el período al día 1 con `DATEFROMPARTS`, consulta si está cerrado con `dbo.fn_PeriodoCerrado` y arma en una tabla temporal `#cand` los **candidatos**: empleados que no están dados de baja (`IdEstado <> 2`), que ya estaban contratados al fin de ese mes (`FechaContratacion <= EOMONTH(@Periodo)`) y, con un `LEFT JOIN` a `Planillas`, su salario base, bono y descuento (con `ISNULL(..., 0)` por si no tienen estructura asignada). Una columna `YaTiene` usa `CASE WHEN EXISTS (...)` para saber si ya existe un pago no anulado ese mes. De esa tabla salen los contadores con `COUNT(*)` y los nombres de quienes no tienen salario con `STRING_AGG`.

La generación real. Con `@Simular = 0`, el procedimiento valida mes futuro y mes cerrado, abre una transacción, hace **un solo** `INSERT ... SELECT` que crea un pago «Pendiente» por cada candidato sin pago y con salario mayor que cero, y confirma. Dos cosas lo hacen robusto: el descuento por ausencias se calcula con las funciones `fn_DiasDescontados` y `fn_DescuentoAusencias` mediante `CROSS APPLY`, y se **topa** para que el total descontado nunca supere base más bono; y, como excluye a quienes ya tienen

pago (`YaTiene = 0`), ejecutarlo dos veces seguidas no duplica nada: la segunda vez reporta «Ya existían: N» y crea cero. Esa propiedad se llama **idempotencia** y es una pregunta típica.

Vuelta a C#. `Generar` pide confirmación, llama otra vez al procedimiento, ahora con `@Simular = 0` , y muestra el resumen de pagos creados, existentes y omitidos. Sus `catch` están ordenados de lo específico a lo general: primero `SqlException` (un error de la base, como un mes cerrado que se cerró entre la vista previa y el clic, se muestra tal cual) y después `Exception` . Al terminar guarda la clave del mes en la propiedad pública `ClavePeriodo` (con `private set`), pone `DialogResult = OK` y cierra, y la pantalla que lo abrió usa esa propiedad para mostrar justo el mes recién generado.

¿Por qué simular en lugar de calcular la vista previa en C#? Porque entonces habría **dos copias de la misma lógica** (una en C#, otra en SQL) que tarde o temprano se contradirían. Con `@Simular` hay una sola fuente de verdad: lo que la vista previa promete es, por construcción, lo que el botón hace.

10.4 Lo que tienen en común los tres flujos

Si vuelves a leer los tres recorridos notarás que repiten las mismas ideas, y esas ideas son justamente las que el profesor busca cuando pregunta «¿cómo está organizado el código?».

- **La pantalla valida por comodidad; la base valida por seguridad.** Todo lo que la pantalla comprueba se vuelve a comprobar en el procedimiento.
- **Los mensajes de error nacen en SQL** (`THROW` con texto legible) y C# los muestra. No hay una tabla de traducciones.
- **Una conexión corta por operación**, a través de `Datos` , con parámetros siempre; nada se interpola con texto del usuario.
- **Banderas de estado** (`cargando` , `enviando` , `guardando`) para que los eventos no se disparen en momentos equivocados.
- **Después de cambiar algo, se recarga desde la base** (`CargarPrestamos()`) en lugar de editar la tabla a mano. Es más lento por un instante pero la pantalla nunca miente.
- **La auditoría no se programa pantalla por pantalla**: los triggers la hacen y `SESSION_CONTEXT` les dice quién fue.

Para contar cualquier flujo en voz alta, sigue esta receta de cinco frases: *qué botón se pulsa y en qué método; qué valida la pantalla; qué consulta o procedimiento se ejecuta y con qué parámetros; qué reglas y qué tablas toca la base; y cómo se refresca la pantalla o*

cómo se muestra el error. Si puedes responder esas cinco cosas para el login, el préstamo y la planilla, puedes responder para casi cualquier otra pantalla del sistema.

11. Plan de repaso y tiempo estimado

Para que puedas armar tu plan, conté las palabras de esta guía: tiene unas **46.000 palabras** entre explicaciones, código y preguntas (alrededor de 26.000 en las secciones 1 a 10 y 20.000 en el banco de preguntas, secciones 12 a 17). Un texto técnico con código se lee a unas 200 palabras por minuto si vas comprendiendo, así que la **lectura pura** son aproximadamente **3 horas y 50 minutos**. Pero leer no es lo mismo que dominar: para poder explicarlo sin mirar hace falta practicar, y con la práctica el total realista sube a **entre 8 y 10 horas**, repartidas mejor en dos o tres días que en uno. Si solo dispones de un día, hay una versión de **6 a 7 horas** que prioriza lo que más rinde (más abajo).

Es una estimación por conteo de palabras y no una medición con una persona real: depende de cuánto C# ya manejes. Si los conceptos de la sección 2 te resultan conocidos, puedes ojearla y ahorrar unos 10 minutos; si el dibujo con GDI+ te es nuevo, la sección 9 te tomará más de lo previsto.

11.1 Tabla de estudio por bloques

Bloque	Secciones	Lectura	Práctica	Al terminar deberías poder…
1. Mapa y fundamentos	1 y 2	15 min	15 min	Explicar qué es una clase estática, una propiedad, un evento y `using`, y localizar en qué archivo está cada cosa.
2. SQL dentro de C#	3 y 4	30 min	45 min	Reescribir de memoria `YaExisteDirector` y explicarla línea por línea; decir qué hace `Datos.Tabla` y por qué se usan parámetros.
3. Errores y pantallas	5 y 6	25 min	30 min	Dibujar el camino de un error desde `THROW` hasta el aviso; explicar `cargando`, `hayError` y el modo dual de un formulario.

Bloque	Secciones	Lectura	Práctica	Al terminar deberías poder…
4. Seguridad	7	12 min	30 min	Contar el login paso a paso, el bloqueo, BCrypt y la recuperación, incluyendo sus límites.
5. Métodos y servicios	8	8 min	10 min	Explicar el patrón de exportación y qué hace cada servicio.
6. Diseño por código	9	25 min	45 min	Dibujar en papel un icono con coordenadas de 0 a 24; explicar `Invalidate`, el redondeo con `AddArc` y la diferencia entre `CellFormatting` y `CellPainting`.
7. Flujos de punta a punta	10	15 min	30 min	Contar en voz alta el login, el préstamo y la planilla con la receta de cinco frases.
8. Banco de preguntas	12 a 17	100 min	60 min	Responder las preguntas con tus palabras, sin mirar el texto.

Sumando, son unos 230 minutos de lectura y unos 265 de práctica: algo más de 8 horas, y con descansos y relecturas llegas a las 9 o 10.

11.2 Qué priorizar si el tiempo es corto

No todo pesa igual en la defensa. Según los indicadores de la rúbrica (conocimiento del dominio, seguridad, CRUD, búsqueda y filtros, flujo de primer uso, solidez de la base de datos, identidad visual), este es el orden de importancia.

- **Imprescindible**: secciones 3, 4, 5 y 10, y las preguntas 13 y 14. Es la parte donde C# y la base de datos se encuentran, que es justo lo que dijiste que más te importa. Si dominas esto, puedes responder casi cualquier pregunta sobre «cómo funciona».
- **Muy recomendable**: secciones 6 y 7 con las preguntas 15, y la sección 9 con las preguntas 16, porque seguridad y diseño son dos indicadores de la rúbrica.
- **Para leer una vez**: secciones 1, 2 y 8, y el bloque 17. En el 17, lee al menos 17.1, 17.2, 17.3, 17.10 y 17.12, que son las que más probablemente te hagan.

11.3 Versión de un solo día (6 a 7 horas)

1. **Primeras tres horas**: secciones 3, 4 y 5, y la sección 10. Después de cada sección, cierra el documento y cuenta en voz alta lo que entendiste. Si te trabas, relee solo ese punto.
2. **Siguiente hora y media**: sección 7 completa y de la 9 solo 9.1 a 9.4 y 9.6; luego las preguntas 15.6 a 15.10 y 16.2 a 16.4.
3. **Siguiente hora y media**: preguntas 13 y 14 en voz alta, respondiendo antes de leer la respuesta. Después 17.1, 17.2, 17.3, 17.10 y 17.12.
4. **Últimos 30 minutos**: la lista de comprobaciones de abajo, en tu computadora.

11.4 Cómo estudiar para que se quede

La técnica que mejor funciona para entender y no solo reconocer es la **recuperación activa**: leer una pregunta, taparla, intentar responder en voz alta como si hablaras con el profesor y recién entonces comparar con el texto. Si lees la respuesta de corrido, la sensación de que «ya lo sé» es engañosa; lo que de verdad prueba que lo sabes es poder decirlo sin mirar. Tres ejercicios concretos para este proyecto. Primero, **abre el código real**, no solo la guía: busca `YaExisteDirector` en `frmAgregarPersonal.cs` y comprueba que dice lo que dice la sección 3. Segundo, **dibuja con lápiz y papel**: el camino de un dato (pantalla → `Datos` → procedimiento → tabla → trigger → vista → tabla de la pantalla) y un icono sobre una cuadrícula de 24 por 24. Tercero, **cuenta un flujo en tres minutos** con cronómetro, que es más o menos lo que durará tu explicación frente al profesor.

No necesitas memorizar los números de error, los valores RGB ni las coordenadas exactas de cada icono. Sí necesitas entender **por qué** se hace cada cosa: por qué hay parámetros, por qué `INNER` y `LEFT`, por qué `Invalidate`, por qué BCrypt. Los porqués son lo que se pregunta.

11.5 Comprobaciones prácticas antes de presentar

Son cosas que puedes verificar en tu computadora en pocos minutos y que evitan sorpresas.

- **Compilar y presentar en modo Release**, no en Debug. En Debug el programa se salta el login (pregunta 15.6).
- Ejecutar `SELECT Usuario FROM Usuarios WHERE Clave NOT LIKE '$2%'`. Si no devuelve filas, todas las contraseñas están con hash (pregunta 17.2).
- Comprobar que junto al ejecutable existe la carpeta `Ayuda` con el manual de usuario en PDF, porque la tecla **F1** lo abre desde esa ruta.

- Probar el login con el truco `' OR '1'='1` para mostrar que no hay inyección (pregunta 17.1).
- Hacer un recorrido de demostración completo (primer uso, login, registrar un empleado, un préstamo y una planilla) en la base que vas a usar, y dejarla en el estado en que quieres presentarla.
- Tener a mano esta guía y la de base de datos abiertas, pero la meta es no necesitarlas.

12. Preguntas y respuestas, bloque 1: C# y WinForms básicos

Este bloque cubre las bases del lenguaje y de la forma en que Windows Forms organiza una aplicación. Son las preguntas que un profesor hace para comprobar que no solo copiaste código, sino que sabes qué significa cada palabra que aparece en él. Como en la guía de base de datos, las respuestas están escritas para que las leas en voz alta y las adaptes con tus palabras; no las memorices tal cual.

12.1 ¿Qué diferencia hay entre una clase `static` como `Datos` y una clase normal como `ItemMenuLateral`? ¿Por qué `Sesion` es estática?

Una clase normal es un **molde**: para usarla tienes que crear un objeto con `new`, y cada objeto tiene sus propios datos. `ItemMenuLateral` es el ejemplo perfecto: en el menú lateral hay varios ítems (Inicio, Activos, Personal, Planillas…) y cada uno guarda su propio texto, su propio icono y su propio estado de «seleccionado» o «con el ratón encima». Si fuera estática, todos los botones del menú compartirían un único estado y al marcar uno se marcarían todos.

Una clase `static` es lo contrario: **no se instancia nunca** y todo lo que tiene pertenece a la clase misma. Escribes `Datos.Tabla(...)` sin crear nada. Se usa cuando lo que ofrece la clase no depende de un objeto concreto. `Datos` es un conjunto de herramientas (abrir una conexión, ejecutar una consulta, devolver una tabla) que no necesita recordar nada entre una llamada y la siguiente; `PasswordHasher` también: recibe un texto y devuelve un hash.

`Sesion` es estática por una razón distinta: porque lo que guarda debe ser **único y visible desde cualquier pantalla**. Solo hay un usuario conectado a la vez, y cualquier formulario (el panel, la gestión de personal, la de planillas) necesita saber quién es sin que nadie se lo pase como parámetro por una cadena de constructores. Al ser estática, `Sesion.Usuario` vale lo mismo en todas partes.

El precio de ese cuidado es que lo estático es **estado global**: cualquiera puede cambiarlo desde cualquier lugar, y en un programa grande eso complica las pruebas y los errores. Para un sistema de escritorio de un solo usuario es una decisión sencilla y razonable, y si te lo preguntan puedes decir exactamente eso.

12.2 ¿Qué es una propiedad y por qué el `set` de `Activo` llama a `Invalidate()`?

Una propiedad es un campo con **control de acceso**. Por fuera se usa como una variable (`item.Activo = true;`) pero por dentro es un par de bloques, `get` y `set`, donde puedes ejecutar código cada vez que alguien lee o escribe el valor. En `ItemMenuLateral` hay un campo privado `activo` y una propiedad pública `Activo`. Nadie de fuera toca el campo; todos pasan por la propiedad.

Ese código extra es lo que hace funcionar el diseño. En Windows Forms un control no se redibuja solo cuando cambias una variable interna: la pantalla solo se actualiza cuando Windows le pide al control que se pinte. `Invalidate()` es esa petición: «mi aspecto ya no es válido, vuélveme a pintar». Al llamar a `Invalidate()` dentro del `set`, cada vez que el panel marca un ítem como activo, el ítem se repinta con el amarillo de fondo en ese mismo instante. Si no estuviera esa línea, el valor cambiaría en memoria pero el menú seguiría viendo igual hasta que otra cosa lo obligara a repintarse.

Te conviene recordar el flujo completo porque se repite en todas las clases de diseño: **estado → `Invalidate()` → `OnPaint` → pantalla**. La propiedad cambia el estado, `Invalidate` avisa y `OnPaint` dibuja según el estado actual. Ese orden es el esqueleto de `ItemMenuLateral`, `BotonTema`, `GridOscuro` y los demás controles propios.

12.3 ¿Qué significa `partial` y para qué existe `frmLogin.Designer.cs`?

`partial` quiere decir que una misma clase está **repartida en varios archivos**. Al compilar, C# los junta como si fueran uno solo. Se usa para separar dos tipos de código con dueños distintos. El archivo `frmLogin.Designer.cs` lo escribe y reescribe Visual Studio automáticamente cuando arrastras controles al formulario: contiene `InitializeComponent()`, que crea cada botón y cada etiqueta, les da posición, tamaño y color, y los agrega al formulario. El archivo `frmLogin.cs` es el tuyo, donde escribes la lógica.

La separación evita que el diseñador pise tu código, y que tú rompas sin querer lo que el diseñador genera. Por eso en el constructor de cada formulario aparece `InitializeComponent();` como primera instrucción: es la llamada que construye todo lo

que dibujaste en la vista de diseño. Si la borras, el formulario se abre completamente vacío.

También verás `partial` en `frmGestionActivos`, y aquí se usa de otra forma, más sofisticada: la clase está dividida por **temas** y no por dueño. Un archivo tiene la pestaña de activos, otro la de préstamos (`frmGestionActivosPrestamos.cs`), otro la de mantenimiento. Es una manera de que un formulario enorme no sea un solo archivo de miles de líneas imposible de navegar, sin tener que crear clases separadas que necesitarían compartir datos.

12.4 ¿Qué es `override OnPaint` y para qué sirve `base.OnPaint(e)`?

Cada control de Windows Forms tiene un método `OnPaint` que Windows invoca cuando hay que dibujarlo. Los controles estándar ya traen una versión que los pinta con el aspecto gris clásico. Cuando haces una clase propia que **hereda** de uno (`class BotonTema : Button`), puedes **sobrescribir** ese método con la palabra `override` para dibujar tus propias formas, colores y textos. Así se consigue un botón con esquinas redondeadas y un menú con iconos dibujados por código.

`base.OnPaint(e)` significa «ejecuta primero la versión del padre». Decidir si llamarla o no es una decisión de diseño. Si quieres **añadir** algo sobre lo que el control ya dibuja, la llamas primero y luego pintas encima. Si quieres **reemplazar por completo** el dibujo, no la llamas y pintas tú todo, que es lo que hacen varios controles de este proyecto porque su aspecto no se parece en nada al estándar. En `PanelTarjeta` se ven ambos casos.

El parámetro `e` es un `PaintEventArgs`, y lo valioso que trae es `e.Graphics`: el objeto con el que se dibuja (líneas, rellenos, textos). Todo el diseño del proyecto sale de usar ese objeto con pinceles (`SolidBrush`) y lápices (`Pen`) del color correcto. La sección 9 lo explica con detalle.

12.5 ¿Qué diferencia hay entre `Show()` y `ShowDialog()`? ¿Qué hace `DialogResult`?

`Show()` abre el formulario y **sigue de largo**: el código que lo llamó continúa ejecutándose y el usuario puede seguir usando la ventana anterior. `ShowDialog()` abre el formulario como **ventana modal**: la línea que lo llama se queda esperando y la ventana de atrás queda bloqueada hasta que el diálogo se cierre. Cuando se cierra, `ShowDialog()` devuelve un valor `DialogResult` que dice cómo terminó: `OK`, `Cancel`, `Yes`, etc.

Eso permite escribir código muy natural. En `frmGestionPersonal`, al editar un empleado, aparece algo como `if (frm.ShowDialog(this) == DialogResult.OK) MostrarEmpleados();`. Se lee: «muéstrame el formulario de edición; si el usuario terminó guardando, recarga la lista». El formulario hijo, cuando guarda con éxito, pone `DialogResult = DialogResult.OK`, y asignarla también lo cierra. Si el usuario cancela o cierra la ventana, el resultado no es `OK` y la lista no se recarga, porque no cambió nada.

El login usa esa misma idea al revés: abre el panel con `ShowDialog()` y la línea siguiente solo se ejecuta cuando el panel se cierra; por eso, al volver, puede preguntar si hay sesión y decidir si mostrar de nuevo el login o terminar. También `frmGenerarPlanilla` y los diálogos de usuarios usan `ShowDialog`. Las pantallas principales que viven dentro del panel no usan ninguna de las dos: se incrustan en un contenedor, como se explica en la sección 6.

12.6 ¿Qué es un evento? ¿Qué significa `+=` y qué es `Func<Form>`?

Un **evento** es un aviso que un objeto emite cuando le ocurre algo: un botón avisa `Click`, un temporizador avisa `Tick`, un combo avisa `SelectedIndexChanged`. Un **manejador** es el método que tú escribes para reaccionar. Se conectan con `+=`, que en este contexto no suma: significa «agrega este método a la lista de quienes quieren enterarse». Por eso `timerMenu.Tick += timerMenu_Tick;` se lee «cada vez que el temporizador marque, ejecuta `timerMenu_Tick`». Casi todos los métodos con firma `(object sender, EventArgs e)` de este proyecto son manejadores; `sender` es el control que disparó el aviso y `e` trae los detalles.

A veces el manejador es tan corto que se escribe en línea, como `cmbMes.SelectedIndexChanged += (s, e) => VistaPrevia();`. Esa flecha `=>` define una **función anónima** (lambda) sin nombre. Es la misma idea, pero escrita sin crear un método aparte.

`Func<Form>` es un **delegado**, es decir, un tipo que representa «una función guardada en una variable». Significa «una función que no recibe nada y devuelve un `Form`». En el panel, `Navegar(item, Func<Form> crearFormulario)` recibe como segundo argumento una **receta** para fabricar la pantalla (`() => new frmGestionPersonal()`), no la pantalla ya hecha. Así `Navegar` decide cuándo ejecutar la receta: después de liberar la pantalla anterior y dentro de un `try/catch`. Si le pasaran el formulario ya construido, cualquier error al crearlo ocurriría fuera del control de `Navegar`. Un delegado es una forma de pasar **comportamiento** como si fuera un dato.

12.7 ¿Para qué sirve `using`? ¿Qué pasa si no lo uso?

Algunos objetos manejan recursos que no son memoria normal: una conexión a la base de datos, un lápiz de dibujo (`Pen`), un archivo abierto. El recolector de basura de .NET termina por liberarlos, pero **no se sabe cuándo**, y mientras tanto siguen ocupando algo valioso: una conexión abierta de las pocas disponibles, o un recurso gráfico de Windows. Esos objetos implementan `IDisposable` y tienen un método `Dispose()` que libera el recurso en el acto.

`using (var x = ...) { ... }` garantiza que `Dispose()` se llame **al salir del bloque, pase lo que pase**, incluso si ocurre una excepción a mitad. Es una abreviatura de un `try/finally`. Por eso toda la clase `Datos` envuelve la conexión y el adaptador en `using`: aunque la consulta falle, la conexión se cierra y se devuelve al *pool*. En el dibujo ocurre igual: `using (Pen p = new Pen(color))` evita que cada repintado (que ocurre decenas de veces por segundo al mover el ratón) deje lápices sin liberar.

Si no lo usaras, el programa seguiría funcionando durante un rato, y eso es lo peligroso: la fuga es silenciosa. Con muchas consultas se agotarían las conexiones del *pool* y el sistema empezaría a fallar al azar, y en dibujo se acumularían objetos gráficos hasta que Windows se negara a crear más. Un detalle que te hará ver como alguien que lo entendió: en `frmLogin`, `using (frmDashboard dashboard = new frmDashboard())` libera también un formulario completo cuando el panel se cierra.

12.8 ¿Cómo funciona `try / catch / finally`? ¿Importa el orden de los `catch`? ¿Qué hace `throw;`?

`try` envuelve el código que podría fallar. Si algo lanza una excepción, la ejecución salta al primer `catch` cuyo tipo coincida. `finally` se ejecuta siempre, haya o no error, y se usa para limpiar (volver a habilitar un botón, como hace el login después de validar).

El orden de los `catch` **sí importa**: C# los revisa de arriba a abajo y entra al primero que sirva. Como `SqlException` es una especialización de `Exception`, hay que poner el específico primero y el general al final, como hace `frmGenerarPlanilla.Generar`: primero `catch (SqlException ex)` para mostrar el mensaje de la base tal cual, y después `catch (Exception ex)` para todo lo demás. Si pusieras `Exception` primero, el compilador ni siquiera te deja compilar, porque el segundo bloque sería inalcanzable.

`throw;` sin nada más dentro de un `catch` **relanza la misma excepción** conservando su historial de dónde ocurrió. Se usa cuando quieres hacer algo (por ejemplo, un `ROLLBACK`) y aún así dejar que el error siga subiendo. Es distinto de `throw ex;`, que reinicia el rastro y borra el origen real. Y existe un `catch { }` vacío que debes poder defender: en

`RegistrarAcceso` del login está puesto a propósito porque dejar constancia es importante, pero si la bitácora fallara no debe impedir que alguien legítimo entre. Un `catch` vacío es sospechoso casi siempre; este tiene una justificación escrita, y esa diferencia es lo que conviene explicar.

12.9 ¿Qué es `DBNull` y por qué `Datos.P` usa `??`?

En SQL, `NULL` significa «no hay dato». En C#, `null` significa «esta variable no apunta a nada». Parecen lo mismo pero para ADO.NET **no lo son**: cuando un parámetro de un comando vale `null` de C#, ADO.NET lo considera «parámetro sin valor» y la ejecución falla diciendo que falta. Para enviar un `NULL` real hay que usar el objeto especial `DBNull.Value`.

`Datos.P` resuelve esto una vez para todo el proyecto: `new SqlParameter(nombre, valor ?? DBNull.Value)`. El operador `??` significa «si lo de la izquierda es `null`, usa lo de la derecha». Gracias a eso, el resto del código puede escribir `Datos.P("@IdEmpleado", idEmpleado)` con un `int?` nulo y funciona, como en el préstamo de un alumno, que no tiene empleado asociado.

El problema inverso también existe: cuando **lees** un valor, una columna `NULL` llega como `DBNull.Value` y no como `null`, así que `reader["Telefono"].ToString()` funciona (da texto vacío) pero `Convert.ToInt32(reader["x"])` lanzaría una excepción. Por eso el login revisa `reader["NombreCompleto"] == DBNull.Value ? "" : ...`. La regla práctica: al escribir, `Datos.P` cubre el caso; al leer, hay que preguntar antes si la columna puede ser nula.

12.10 ¿Para qué sirve la bandera `cargando` que aparece en tantas pantallas?

Es una variable `bool` que significa «estoy llenando la pantalla, no reacciones todavía». El problema que resuelve es sutil: cuando el código llena un combo con `Items.AddRange(...)` o asigna `SelectedIndex = 0`, el control **dispara su evento `SelectedIndexChanged` como si lo hubiera hecho una persona**. Si ese evento ejecuta una consulta (por ejemplo, `CargarPrestamos()`), tendrías consultas innecesarias, o peor, consultas que dependen de otros controles que todavía no se han llenado, con resultados erróneos o errores.

La solución es el patrón que se repite: `bool previo = cargando; cargando = true;` antes de llenar, y `cargando = previo;` al terminar. Y al inicio de cada manejador sensible: `if (cargando) return;`. Se guarda y restaura el valor anterior (`previo`) en vez de poner

`false` a ciegas para que funcione bien cuando un método que ya estaba cargando llama a otro que también lo hace; el segundo no debe apagar la bandera del primero.

Es el mismo tipo de idea que el `enviando` del asistente de primer uso o el `guardando` de otros formularios: una variable de estado que protege una operación larga o delicada. Si te preguntan por qué existen, la respuesta corta es: «porque en Windows Forms el código y el usuario disparan los mismos eventos, y hay que poder distinguirlos».

12.11 ¿Por qué el correo se envía con `async` / `Task.Run` y qué es `BeginInvoke`?

Windows Forms tiene **un solo hilo para la interfaz**: el que dibuja ventanas y atiende clics. Si ese hilo se pone a esperar algo lento, como una petición a internet para enviar un código por correo, la ventana se congela: no se repinta, no responde, y Windows puede marcarla como «no responde». `ServicioCorreo.EnviarCodigoAsync` evita eso: devuelve un `Task` y hace el trabajo real dentro de `Task.Run(...)`, es decir, en otro hilo. La pantalla lo espera con `await`, dentro de un método marcado `async`, y mientras tanto el hilo de la interfaz queda libre. Por eso el asistente muestra «Enviando…» y puedes mover la ventana.

Hay una regla importante: **un control solo se puede tocar desde el hilo que lo creó**. Si desde otro hilo intentas cambiar un `Label`, Windows Forms lanza una excepción. Cuando necesitas pedirle al hilo de la interfaz que haga algo, se usa `BeginInvoke(...)`, que significa «ejecuta esto en el hilo de la interfaz cuando puedas». En el proyecto aparece en `GridOscuro` / `ControlesTema` para actualizar las barras de desplazamiento personalizadas después de que el control terminó de procesar un mensaje, y en `frmInicio` para saltar a una sección justo después de un clic.

Para tu defensa: el sistema usa programación asíncrona para un solo tipo de tarea, el envío de correo (verificación del asistente, recuperación de contraseña y centro de soporte), porque es lo único que depende de internet. Todo lo demás va contra una base local que responde en milisegundos y por eso se ejecuta de forma síncrona, sin complicar el código.

13. Preguntas y respuestas, bloque 2: SQL dentro de C#

Este es el bloque más importante de la guía, porque aquí está el punto donde el C# y la base de datos se tocan. Casi cualquier pregunta sobre «cómo funciona esto» termina en una consulta escrita dentro de un método. Las dos primeras preguntas desmontan, línea por línea, los dos ejemplos que me pediste; el resto cubre los demás tipos de consulta que aparecen en el proyecto.

13.1 Explica la consulta de `YaExisteDirector` parte por parte. ¿Qué pregunta le está haciendo a la base?

El método vive en `frmAgregarPersonal` y responde una sola pregunta de sí o no: «¿hay ya otro director registrado?». Se usa antes de guardar, para impedir que el sistema tenga dos directores. La consulta es:

```
SELECT COUNT(*)
FROM Empleados e
INNER JOIN Cargos c ON e.IdCargo = c.IdCargo
WHERE LOWER(c.NombreCargo) LIKE '%director%'
  AND LOWER(c.NombreCargo) NOT LIKE '%sub%'
  AND e.IdEmpleado <> @idEmpleadoExcluido
```

Se lee de abajo hacia arriba, igual que lo hace la base. `FROM Empleados e` dice qué tabla se consulta y le da el alias `e`. El `INNER JOIN Cargos c ON e.IdCargo = c.IdCargo` une cada empleado con su cargo, porque el empleado guarda solo el número del cargo y el **nombre** («Director», «Docente») está en otra tabla. Después el `WHERE` va descartando filas: se quedan solo las cuyo nombre de cargo, pasado a minúsculas con `LOWER`, **contiene** la palabra «director» (`LIKE '%director%'`, donde cada `%` significa «cualquier cosa, incluso nada»), que **no contiene** «sub» (`NOT LIKE '%sub%'`, para que «Subdirectora» no cuente como director) y que no sea el empleado que estamos editando (`<>` es «distinto de»). Por último `SELECT COUNT(*)` no devuelve las filas, solo **cuántas** quedaron.

Ese `@idEmpleadoExcluido` es un **parámetro**: un hueco que C# rellena después con un valor. Cuando se registra un empleado nuevo, la pantalla manda `0` (ningún empleado tiene ese número, así que no se excluye a nadie). Cuando se edita al propio director, manda su Id, y así el director existente no se cuenta a sí mismo y puedes cambiarle el teléfono sin que el sistema diga que «ya hay un director».

La parte de C# es corta: abre una conexión dentro de un `using`, crea el `SqlCommand` con ese texto, le asocia el parámetro con `AddWithValue`, abre la conexión y llama a `ExecuteScalar()`, que ejecuta la consulta y devuelve **el valor de la primera columna de la primera fila**. Como `COUNT(*)` siempre devuelve una fila con un número, `Convert.ToInt32(...)` lo convierte y el método devuelve `count > 0`.

Un detalle que luce mucho en la defensa: la pantalla repite la misma regla **en C#**, antes de llamar al método: `nombreCargo.ToLower().Contains("director") && !nombreCargo.ToLower().Contains("sub")`. Es exactamente lo mismo que el `LIKE`

`'%director%'` y el `NOT LIKE '%sub%'`, escrito en el otro lenguaje. La primera mitad decide si el cargo que elegiste **es** de director; la consulta pregunta si **ya hay otro** con ese tipo de cargo.

Y las limitaciones, que conviene conocer con honestidad: la consulta no mira el estado del empleado, así que un director dado de baja seguiría contando; la regla se basa en el **texto** del nombre del cargo y no en un identificador, así que si alguien crea un cargo llamado «Director de Sistemas» el sistema lo trataría como el director; y la base de datos no impone esta regla (solo la pantalla), por lo que dos equipos guardando a la vez podrían colarse. Existe una versión anterior en `AsignarCargo.cs` que comparaba contra `IdCargo` fijos (1 y 2); la actual cambió a texto para no depender de números que podrían variar entre instalaciones.

13.2 Explica la consulta que llena la lista de personal (`dtEmpleados = Datos.Tabla(...)`).

Esta consulta no responde una pregunta de sí o no, sino que **trae una tabla completa** para mostrarla en la cuadrícula. Su forma es:

```
SELECT e.IdEmpleado, e.Nombre, c.NombreCargo AS Cargo, est.NombreEstado AS
Estado,
       e.FechaContratacion, p.SalarioNeto AS [Salario Neto], e.Telefono,
e.Correo, ...
FROM Empleados e
INNER JOIN Cargos c            ON e.IdCargo  = c.IdCargo
INNER JOIN EstadosEmpleado est ON e.IdEstado = est.IdEstado
LEFT  JOIN Planillas p         ON e.IdPlanilla = p.IdPlanilla
WHERE est.NombreEstado != 'Inactivo'
ORDER BY e.Nombre
```

El `SELECT` elige las columnas, y cada `AS` les pone el **nombre con el que aparecerán**: `c.NombreCargo AS Cargo` hace que la columna se llame «Cargo» en la tabla de C#. Los corchetes de `[Salario Neto]` son necesarios porque el nombre lleva un espacio; sin ellos SQL creería que son dos palabras. El `FROM` parte de `Empleados` y encadena **tres uniones** que traen datos de otras tablas: el cargo, el estado y la estructura de planilla (de donde salen los salarios). El `WHERE est.NombreEstado != 'Inactivo'` deja fuera a los empleados dados de baja, porque la lista es para ver al personal en funciones. Y `ORDER BY e.Nombre` los ordena alfabéticamente.

En C#, `Datos.Tabla(texto)` abre una conexión corta, usa un `SqlDataAdapter` para ejecutar la consulta y **llena un `DataTable`** en memoria con el resultado, y cierra la conexión. Esa tabla en memoria queda en la variable `dtEmpleados` y es la que después se enlaza a la cuadrícula. No hay parámetros porque ninguno de los valores depende de lo que escriba el usuario: `'Inactivo'` es una constante de la regla de negocio.

Observa que se trae **una sola vez** y después se filtra en memoria cuando el usuario escribe en el buscador (pregunta 13.7). Eso es una decisión de diseño: para unos cientos de empleados es más rápido y ahorra viajes a la base. Y la pregunta que casi seguro te harán sobre esta consulta es la siguiente.

13.3 ¿Por qué `INNER JOIN` con cargos y estados pero `LEFT JOIN` con planillas?

La diferencia está en qué pasa con las filas que **no encuentran pareja**. `INNER JOIN` es estricto: solo devuelve un empleado si encuentra su cargo. Si no hay cargo, el empleado desaparece del resultado. `LEFT JOIN` es indulgente: devuelve **todos** los empleados de la tabla de la izquierda y, si no encuentra planilla, rellena con `NULL` las columnas de la derecha.

Se elige según lo que significa que falte el dato. Un empleado **siempre** debe tener cargo y estado: el formulario de registro no deja guardar sin elegirlos y ambas columnas son llaves foráneas hacia `Cargos` y `EstadosEmpleado`. (Una precisión técnica por si te la piden: en la tabla esas columnas aceptan `NULL`, de modo que quien garantiza que siempre tengan valor es la pantalla y no un `NOT NULL`; si alguien insertara un empleado sin cargo por otra vía, el `INNER JOIN` lo haría desaparecer de esta lista.) Por eso el `INNER JOIN` es la forma natural de decir «esto es indispensable». En cambio, `IdPlanilla` es opcional en la base, y el sistema contempla de forma explícita a empleados sin salario asignado: `sp_GenerarPlanilla` los cuenta y los avisa como «sin salario asignado». El formulario actual sí pide elegir una planilla al registrar, pero personal cargado por script o en versiones anteriores podría no tenerla. Con un `INNER JOIN` ese empleado desaparecería de la lista y nadie podría encontrarlo para asignarle su planilla, que es justo el problema que se quiere resolver. Con `LEFT JOIN` aparece, con el salario vacío.

Puedes explicarlo con una imagen: la lista de asistencia de un curso (empleados) y una tabla aparte con las notas (planillas). `INNER JOIN` es «muéstrame solo a los alumnos que tienen nota»; `LEFT JOIN` es «muéstrame a todos los alumnos y, a quien no tenga nota, déjale el espacio en blanco». La regla práctica para el examen: **`LEFT JOIN` cuando el dato de la derecha es opcional; `INNER JOIN` cuando es obligatorio**.

13.4 ¿Qué hacen `LIKE`, `%` y `LOWER`? ¿Se pueden poner los comodines en un parámetro?

`LIKE` compara texto contra un **patrón**, no contra un valor exacto. El comodín `%` representa «cualquier cantidad de caracteres, incluso ninguno» y `_` representa exactamente uno. `LIKE 'doc%'` encuentra los que **empiezan** con «doc»; `LIKE '%dor'`, los que **terminan** así; `LIKE '%director%'`, los que lo **contienen** en cualquier parte, que es lo que necesitas cuando el cargo puede llamarse «Director», «Directora» o «Director General». `NOT LIKE` es el contrario. `LOWER()` pasa el texto a minúsculas antes de comparar, para que las mayúsculas no importen (con la configuración habitual de SQL Server ya se ignoran, así que `LOWER` es un refuerzo que deja la intención escrita).

En el proyecto hay **tres maneras** de combinar `LIKE` con valores que escribe el usuario, y las tres son seguras:

- Patrón constante escrito en la consulta: `LIKE '%director%'`. Sirve cuando la palabra la decide el programador, no el usuario.
- Comodines en SQL alrededor del parámetro: `LOWER(Nombre) LIKE '%' + LOWER(TRIM(@criterio)) + '%'`, que es lo que hace la búsqueda de `Empleado.BuscarEmpleadoPorNombreIdOCargo`. El usuario aporta solo la palabra y la consulta le pone los `%`.
- Comodines dentro del valor del parámetro: en `Salon.cs`, `AddWithValue("@Criterio", "%" + criterio.Trim().ToLower() + "%")`. Aquí la consulta es `LIKE @Criterio` y es C# quien arma el patrón.

Todas son seguras por la misma razón: lo que escribió el usuario viaja **como valor**, nunca pegado al texto de la consulta. Un detalle para el examen: un patrón que empieza con `%` obliga a SQL Server a revisar todas las filas en lugar de usar un índice por nombre. Con cientos de registros es irrelevante, y con millones se notaría.

13.5 ¿Qué es un parámetro y por qué nunca se concatena el texto del usuario? ¿Qué es una inyección SQL?

Un parámetro es un hueco con nombre (`@idEmpleadoExcluido`) dentro de la consulta. C# manda **por separado** el texto de la consulta y los valores: la base compila primero la consulta con los huecos y después inserta los valores como datos, jamás como código.

La alternativa mala sería armar la consulta pegando texto: `"... WHERE Usuario = '" + txtUsuario.Text + "'"`. Si alguien escribe en el campo `' OR '1'='1`, la consulta resultante es `WHERE Usuario = '' OR '1'='1'`, que es verdadera para **todas** las filas: el

atacante pasó de poner un dato a modificar la lógica. Variantes peores terminan una instrucción y añaden otra, por ejemplo para borrar una tabla. A eso se le llama **inyección SQL**, y es una de las vulnerabilidades más conocidas y más fáciles de evitar. Con parámetros, ese mismo texto se busca literalmente como un nombre de usuario rarísimo y no se encuentra a nadie.

En el proyecto hay tres formas de crear parámetros, todas equivalentes en seguridad: `cmd.Parameters.AddWithValue("@x", valor)` en el código más directo; `Datos.P("@x", valor)`, que además convierte `null` en `DBNull` (pregunta 12.9); y `Datos.PTexto("@x", texto)`, que prepara textos opcionales para que un cuadro vacío se guarde como `NULL` y no como cadena vacía. Un matiz técnico por si te lo preguntan: `AddWithValue` deduce el tipo SQL desde el tipo de C#, y para cadenas lo hace como `NVARCHAR` según la longitud; en un sistema de gran volumen se prefiere declarar el tipo explícitamente. En el caso de la foto, el proyecto sí lo hace con `SqlDbType.VarBinary`.

13.6 ¿Qué diferencia hay entre `ExecuteScalar`, `ExecuteNonQuery`, `ExecuteReader` y `SqlDataAdapter.Fill`?

Las cuatro ejecutan SQL, pero devuelven cosas distintas, y elegir bien es cuestión de preguntarte qué necesitas de vuelta.

- **`ExecuteScalar()`** devuelve **un solo valor**: la primera columna de la primera fila, como `object`. Es ideal para `COUNT(*)`, `SUM(...)`, un `SELECT TOP 1` o un `INSERT` seguido de `SELECT SCOPE_IDENTITY()`. Si la consulta no devuelve ninguna fila, el resultado es `null`, y por eso `Salon.ObtenerIdEstadoInicial` revisa `r == null ? 1 : Convert.ToInt32(r)`.
- **`ExecuteNonQuery()`** ejecuta algo que **no devuelve filas**: `INSERT`, `UPDATE`, `DELETE`, o un procedimiento. Devuelve un `int` con el **número de filas afectadas**, que se usa como confirmación (`return cmd.ExecuteNonQuery() > 0;`).
- **`ExecuteReader()`** devuelve un `SqlDataReader`, un cursor que lees **fila por fila** con `while (reader.Read())`. Es lo más económico para recorrer resultados una sola vez, y es lo que usa el login para leer la fila del usuario. Exige mantener la conexión abierta mientras lees, por eso va dentro de un `using`.
- **`SqlDataAdapter.Fill(tabla)`** trae **todo el resultado de golpe** a un `DataTable` en memoria, abriendo y cerrando la conexión por ti. Es lo que hay detrás de `Datos.Tabla`, y es lo que permite luego enlazar la cuadrícula, filtrar y ordenar sin volver a la base.

La regla práctica: «un número o un dato → `ExecuteScalar`; guardar, cambiar o borrar → `ExecuteNonQuery`; una lista para mostrar → `Fill`; leer fila a fila sin guardar → `ExecuteReader`». Y una curiosidad: `Datos.Tabla` no llama a `Open()` porque `Fill` abre la conexión si está cerrada y la vuelve a cerrar al terminar; con `ExecuteScalar`, `ExecuteNonQuery` y `ExecuteReader` sí debes abrirla tú, como se ve en `YaExisteDirector` (`conexion.Open()`).

13.7 ¿Qué es un `DataTable` y qué hace `RowFilter`? ¿Por qué se filtra en memoria y no con otra consulta?

Un `DataTable` es una **tabla en la memoria del programa**: filas y columnas con tipos, desconectada de la base. Una vez llena, la conexión ya se cerró y la tabla se puede enlazar a una cuadrícula (`dgv.DataSource = dt`), recorrer con `foreach (DataRow r in dt.Rows)` o ver a través de un `DataView`.

Un `DataView` es una **ventana** sobre la tabla con su propio filtro y orden, que se asigna en la propiedad `RowFilter`. Cuando el usuario escribe en el buscador de personal o de salones, el código no consulta otra vez a SQL Server: arma un texto como `NombreSalon LIKE '%aula%' OR Tipo LIKE '%aula%'` y se lo asigna a `vista.RowFilter`. La cuadrícula se actualiza al instante, sin viaje a la base.

El detalle importante es que **`RowFilter` no es SQL**, aunque se parezca. Es un lenguaje de expresiones propio de .NET: los booleanos se escriben `Activo = true`, el comodín de `LIKE` puede ser `*` o `%`, y los corchetes tienen significado especial. Por eso aparece la función `Escapar`, que en `frmSalones` hace `t.Replace("'", "''").Replace("[", "[[]").Replace("%", "[%]").Replace("*", "[*]")`. Sin ella, buscar una comilla simple rompe el filtro, y buscar `%` coincidiría con todo. Aquí el texto del usuario sí se pega dentro de una expresión (no hay parámetros en `RowFilter`), pero no es una inyección SQL: la expresión nunca llega a la base, solo se evalúa sobre filas que ya estaban en memoria, y `Escapar` evita que rompa el filtro.

Por qué en memoria: porque la lista ya está cargada, los volúmenes son pequeños y filtrar cada pulsación con una consulta sería lento y sin necesidad. El límite: si la tabla tuviera cientos de miles de filas, traerla completa sería mala idea y habría que filtrar en SQL con paginación.

13.8 Explica `COUNT`, `SUM(CASE WHEN ...)`, `GROUP BY` e `ISNULL` como se usan en las tarjetas del inicio y en las etiquetas de alertas.

Son las herramientas para **resumir** datos en la propia consulta, de modo que a C# solo le llegue el resultado listo. `COUNT(*)` cuenta filas. `SUM(columna)` suma. Y cuando necesitas contar solo las que cumplen una condición, se combina con `CASE`, que es el `if` de SQL: `SUM(CASE WHEN Alerta = 'Atrasado' THEN 1 ELSE 0 END)` suma un 1 por cada préstamo atrasado y un 0 por los demás, que da como resultado el número de atrasados. Con dos de esas columnas en la misma consulta (atrasados y por vencer) obtienes dos contadores con **un solo viaje** a la base, que es lo que hace `CargarPrestamos` para la etiqueta de alertas.

`ISNULL(valor, 0)` cambia un `NULL` por un valor de reemplazo. Se envuelve alrededor de las sumas porque `SUM` sobre cero filas devuelve `NULL`, no `0`, y en C# un `DBNull` rompe `Convert.ToInt32`. Por eso aparece como `ISNULL(SUM(CASE ...), 0)`: garantiza que siempre vuelva un número.

`GROUP BY` agrupa las filas por una columna y permite calcular un resumen **por grupo**. Si quieres «unidades por categoría», agrupas por categoría y cuentas dentro de cada grupo. Una regla que se pregunta mucho: en un `SELECT` con `GROUP BY`, cada columna debe estar en el `GROUP BY` o dentro de una función de agregado (`COUNT`, `SUM`…). Lo mismo hace `vw_ResumenActivosPorCategoria` en la base. Y `TOP N` con `ORDER BY` da los «N primeros», que se usa para rankings y para obtener «el más reciente».

En la gráfica de los últimos seis meses de `frmInicio` aparece una variante más ingeniosa: una tabla de meses fabricada **dentro de la consulta** con `DATEFROMPARTS(YEAR(DATEADD(MONTH, -n, GETDATE())), MONTH(DATEADD(MONTH, -n, GETDATE())), 1)` para cada `n` de 0 a 5, unida con `LEFT JOIN` a los datos. Así los meses sin registros aparecen con cero en vez de desaparecer, y la gráfica no tiene huecos.

13.9 ¿Cómo obtiene C# el Id que la base le puso a un registro recién insertado?

Cuando un empleado se inserta, la columna `IdEmpleado` se genera sola (es `IDENTITY`), así que C# no sabe qué número le tocó. Lo necesita de inmediato para guardar, por ejemplo, su foto y sus materias, que se apoyan en ese Id. La solución en `frmAgregarPersonal.GuardarEmpleado` es terminar la instrucción con un segundo `SELECT`:

```
INSERT INTO Empleados (...) VALUES (...);
SELECT CAST(SCOPE_IDENTITY() AS INT);
```

`SCOPE_IDENTITY()` devuelve el último Id generado **en esta ejecución** (el ámbito actual), de modo que dos personas guardando al mismo tiempo no se confunden. El `CAST` lo convierte a `INT` porque la función devuelve un tipo numérico más grande. Como el texto contiene dos instrucciones, C# lo ejecuta con `ExecuteScalar()`: el `INSERT` no devuelve filas y el `SELECT` final devuelve una, y `ExecuteScalar` se queda con ese valor. En un edición, en cambio, la consulta es un `UPDATE` y el Id ya se conoce.

Hay una advertencia clásica: existe también `@@IDENTITY`, que puede devolver el Id de **otra tabla** si un trigger insertó algo (y este sistema tiene triggers de auditoría en casi todas las tablas). `SCOPE_IDENTITY()` evita ese problema. Si en la defensa explicas por qué se eligió esa función y no `@@IDENTITY`, demuestras que entiendes cómo conviven los triggers con el código.

13.10 ¿Cómo se hacen `UPDATE` y `DELETE` desde C#? ¿Para qué sirve el número que devuelve `ExecuteNonQuery`?

Se escriben igual que en SQL, con parámetros, y se ejecutan con `ExecuteNonQuery()`. Lo que devuelve ese método es **cuántas filas cambió**, y es la manera de saber si la operación hizo algo: un `UPDATE ... WHERE IdEmpleado = @id` que devuelve `0` significa que ese empleado ya no existe, quizá porque otro usuario lo eliminó mientras tenías el formulario abierto. Por eso los métodos terminan con `return cmd.ExecuteNonQuery() > 0;`.

El ejemplo más interesante de `DELETE` es `Salon.QuitarActivo`:

```
DELETE TOP (@n) FROM UnidadesActivo
WHERE IdSalon = @IdSalon AND IdActivo = @IdActivo
  AND NOT EXISTS (SELECT 1 FROM Prestamos p WHERE p.IdUnidad =
UnidadesActivo.IdUnidad)
  AND NOT EXISTS (SELECT 1 FROM MantenimientosActivo m ...)
  AND NOT EXISTS (SELECT 1 FROM ReportesDanio r ...)
```

Se lee: «borra hasta N unidades de ese equipo en ese salón, pero solo las que **no tienen historial** de préstamos, mantenimientos ni reportes de daño». `NOT EXISTS (SELECT 1 ...)` es una pregunta de «no hay ninguna fila que cumpla esto»; el `1` es solo un relleno porque lo que importa es si existe una fila, no su contenido. Después C# compara `borradas < cantidadAQuitar` y, si se borraron menos de las pedidas, avisa que el resto tiene historial y sugiere «Dado de baja». Es la misma filosofía de integridad de la base de datos, aplicada con una sola instrucción.

Y una observación sobre estilo: en las pantallas modernas, la mayoría de las eliminaciones son **bajas lógicas** (`UPDATE ... SET IdEstado = 2`), no `DELETE`, para conservar el historial. El `DELETE` real queda para casos como este, donde se comprobó que no hay nada que perder.

13.11 ¿Para qué sirve `UNION ALL` y qué reglas tiene? ¿Qué es `OUTER APPLY`?

`UNION ALL` **apila** dos consultas una debajo de otra en una sola tabla de resultados. En `Salon.ObtenerActivosPorSalon` se usa para mostrar, en una misma lista, los equipos de un salón y los consumibles que se le entregaron:

```
SELECT IdActivo, Activo AS NombreActivo, Categoria, 1 AS Cantidad,
CodigoInventario, Estado
FROM vw_UnidadesDetalle WHERE IdSalon = @IdSalon
UNION ALL
SELECT 0, Consumible, 'Consumible entregado', Entregado, NULL, 'Última
entrega: ' + CONVERT(VARCHAR(10), UltimaEntrega, 103)
FROM vw_ConsumiblesPorSalon WHERE IdSalon = @IdSalon
```

Las reglas son tres: ambas consultas deben tener **el mismo número de columnas**, en el mismo orden y con tipos compatibles; los nombres de columna los toma de la **primera** consulta; y el `ORDER BY` se escribe una sola vez al final y afecta al resultado completo. Por eso la segunda consulta rellena con constantes (`0`, `'Consumible entregado'`, `NULL`) las columnas que no tiene, para que encajen con la primera. Y la diferencia con `UNION` a secas: `UNION` elimina filas duplicadas (y por eso es más lento), `UNION ALL` las conserva todas, y aquí es lo correcto porque se mezclan dos cosas distintas que no pueden estar duplicadas entre sí.

`OUTER APPLY` aparece en el expediente del empleado, para traer el **estado del pago del mes actual** de cada persona. Piensa en él como un `LEFT JOIN` que puede ser una **subconsulta que depende de la fila actual**: `OUTER APPLY (SELECT TOP 1 ... ORDER BY ...)` ejecuta, para cada empleado, una pequeña consulta que busca su pago más reciente de este mes (sin contar los anulados) y, si no tiene ninguno, deja los campos en `NULL` (el `OUTER` es el equivalente del `LEFT`; después `ISNULL(pg.EstadoPago, 'Sin registrar')` convierte ese vacío en un texto legible). Un `JOIN` normal no puede hacer eso, porque no permite `TOP 1` por cada fila de la izquierda. Es una herramienta avanzada; con saber que existe y para qué sirve alcanza.

13.12 ¿Cómo se llama a un procedimiento almacenado desde C#? ¿Qué diferencia hay entre `Datos.Procedimiento` y `Datos.Tabla("EXEC ...")`?

Hay dos estilos y los dos aparecen en el proyecto. El **clásico** crea un `SqlCommand` con el nombre del procedimiento y le dice `cmd.CommandType = CommandType.StoredProcedure`, como hace `Salon.AsignarActivo` con `dbo.sp_AgregarUnidades`. Entonces el texto ya no se interpreta como una consulta sino como «el nombre de algo que existe en la base», y los parámetros se pasan con `Parameters.Add`. Tiene la ventaja de que el motor sabe exactamente qué está llamando.

El estilo que domina las pantallas recientes es la clase `Datos`, que lo empaqueta: `Datos.Procedimiento("sp_RegistrarPrestamo", Datos.P("@IdUnidad", idUnidad), ...)` abre la conexión, ejecuta el procedimiento sin esperar filas y la cierra. Es lo mismo escrito en una línea.

Cuando el procedimiento **devuelve una tabla** o una fila de resultados, se usa `Datos.Tabla("EXEC dbo.sp_GenerarPlanilla @Periodo = @p, @Usuario = @u, @Simular = 1", Datos.P(...), ...)`. Aquí el procedimiento se invoca como texto con `EXEC` y los valores se pasan como parámetros de ese texto; `Fill` recoge el `SELECT` final del procedimiento y lo deja en un `DataTable` del que se lee `.Rows[0]`. Regla práctica: **`Datos.Procedimiento` cuando no esperas respuesta; `Datos.Tabla("EXEC ...")` cuando el procedimiento termina con un `SELECT` que necesitas leer**. Y en ambos casos los mensajes de `THROW` del procedimiento llegan a C# como una `SqlException`.

13.13 Hay consultas donde se concatena texto (`" + donde + "`). ¿No es eso una inyección SQL?

Solo lo sería si lo que se concatena viniera del usuario. La pregunta correcta no es «¿hay un `+` en la consulta?» sino «¿de dónde sale ese texto?». En `CargarPrestamos`, la variable `donde` se asigna dentro de un `switch` que compara el texto del combo con cinco opciones fijas, y a cada una le asigna un fragmento escrito por el programador (`"Alerta = 'Atrasado'"`, `"Estado = 'Devuelto'"`…). Aunque alguien manipulara el combo, el resultado solo puede ser uno de esos cinco fragmentos constantes: ningún carácter del usuario llega a la consulta. Lo mismo ocurre en la edición del empleado, donde se agrega `, DUI = @dui` solo si el usuario es administrador (`+ (esAdmin ? ", DUI = @dui" : "")`): el fragmento es constante, y el valor del DUI, que sí viene del usuario, sigue viajando como parámetro.

El criterio completo es: **se puede concatenar estructura (qué columnas, qué condición) siempre que salga de constantes del programa; los valores que escribe el usuario siempre van como parámetros**. Si alguna vez ves un `TextBox.Text` pegado dentro de un texto SQL, eso sí sería un hallazgo grave, y no lo hay en el código que revisé. Un caso especial son las listas de números enteros para un `IN (...)`: se construyen a partir de números ya convertidos a `int`, lo cual tampoco puede contener comillas ni palabras SQL, así que es seguro.

13.14 ¿Qué pasa si una consulta no devuelve filas? ¿Cómo se evitan los errores por `null` y por `DBNull` al leer resultados?

Es la fuente de error más frecuente cuando se mezcla C# con SQL, y se maneja de maneras distintas según el método. Con `ExecuteScalar`, si no hay filas devuelve `null` (y si la fila existe pero el valor es `NULL`, devuelve `DBNull.Value`): hay que revisar ambas posibilidades antes de convertir. Con un `DataTable`, una consulta sin resultados devuelve una tabla con cero filas, así que `dt.Rows[0]` lanzaría una excepción; por eso el código cuidadoso pregunta `if (dt.Rows.Count > 0)` antes, como hace el botón de la hoja de resguardo (`if (dt.Rows.Count > 0) HojaResguardoPdf.Generar(dt.Rows[0]);`). Con `ExecuteReader`, `reader.Read()` devuelve `false` si no hay fila, y el login lo aprovecha: si no hay fila, el hash queda `null`, y eso se interpreta como «usuario o contraseña incorrectos» sin dar pistas de cuál de los dos estaba mal.

Hay casos donde la ausencia de filas está garantizada por diseño y se puede usar `Rows[0]` sin miedo: una consulta con `COUNT(*)`, o `SUM` envuelto en `ISNULL`, o el procedimiento `sp_GenerarPlanilla` en modo simulación, que siempre termina con un `SELECT` de exactamente una fila. Saber distinguir esos dos tipos (los que siempre devuelven una fila y los que pueden devolver cero) es parte de entender la consulta, no solo de copiarla.

14. Preguntas y respuestas, bloque 3: acceso a datos, errores y transacciones

Este bloque trata de lo que pasa **alrededor** de las consultas: cómo se abre la puerta a la base de datos, cómo se agrupan varias operaciones para que salgan completas o no salgan, y cómo viaja un error desde SQL Server hasta el aviso que ve el usuario. Es donde se nota si entiendes el sistema o solo lo usas.

14.1 ¿Qué hace la clase `Conexion`? ¿Qué significan `(localdb)\MSSQLLocalDB` e `Integrated Security`?

`Conexion` tiene una sola responsabilidad: **fabricar conexiones**. `Conexion.Conectar()` arma la *cadena de conexión* (el texto que dice a qué servidor y a qué base conectarse) y devuelve un `SqlConnection` sin abrir. Quien la recibe decide cuándo abrirla. Tener ese texto en un solo lugar significa que, si mañana la base cambia de servidor, se edita una línea y no cincuenta pantallas.

`(localdb)\MSSQLLocalDB` es el nombre de **SQL Server LocalDB**, una versión liviana de SQL Server pensada para desarrollo y equipos de escritorio: no hay que instalar un servidor, se levanta bajo demanda cuando alguien se conecta y los datos viven en archivos del propio equipo. Por eso el sistema funciona en una computadora sin configuración de red. `Initial Catalog` indica la base de datos (`Gestion_administrativa_y_escolar`).

`Integrated Security=true` significa **autenticación de Windows**: la base reconoce al usuario de Windows que ejecuta el programa y no hace falta escribir usuario ni contraseña en la cadena. Esa es una decisión de seguridad importante: **no hay ninguna contraseña de base de datos guardada en el código**, así que no se puede filtrar al subir el proyecto a un repositorio. El límite de esa elección es que el sistema solo funciona donde el servidor es local o donde Windows reconoce a ese usuario; para un servidor remoto se necesitaría otra configuración.

También hay una `CadenaMaster`, que apunta a la base del sistema (`master`). Sirve para tareas que ocurren antes de que exista la base propia, como crearla. En el flujo normal de la aplicación el que se usa es `Conectar()`.

14.2 ¿Cómo sabe la base de datos quién hizo un cambio si el `INSERT` no menciona al usuario? ¿Qué es el evento `StateChange`?

Es uno de los trucos más elegantes del proyecto, así que merece una explicación tranquila. La auditoría de la base la hacen los **triggers**, que se disparan solos con cada `INSERT`, `UPDATE` o `DELETE` y escriben qué cambió. Pero un trigger vive dentro de SQL Server y no sabe quién está sentado frente al programa: para la base, todas las conexiones vienen del mismo usuario de Windows. Hace falta pasarle ese dato.

La pieza que lo permite es el *contexto de sesión* de SQL Server: un espacio de pares clave-valor adjunto a cada conexión, que se escribe con `sp_set_session_context` y se lee con `SESSION_CONTEXT(N'Usuario')`. `Conexion.Conectar()` suscribe a cada conexión un manejador del evento `StateChange`, que .NET dispara cada vez que cambia el estado de la conexión. El manejador comprueba que el cambio sea de **cerrada a abierta** y, en ese

preciso momento, ejecuta `EXEC sp_set_session_context @key = N'Usuario', @value = @u` con el valor de `Sesion.Usuario`. Después, cualquier trigger que se dispare en esa conexión lee ese valor y lo escribe en la columna `Usuario` de `Auditoria`.

Lo valioso es que ninguna pantalla tuvo que hacer nada: es **automático y transversal**. Si mañana se agrega una pantalla nueva, su auditoría ya queda atribuida a la persona correcta. Dos detalles para el examen. Primero, el `catch` del manejador está vacío a propósito: si no se pudiera registrar el usuario, la aplicación debe seguir funcionando, aunque el registro quede anónimo. Segundo, cuando `Sesion.Usuario` es `null` (por ejemplo antes del login, o en el modo de desarrollo que salta el login), se manda `DBNull` y la auditoría registra el cambio como «(sistema)».

14.3 ¿Por qué cada método de `Datos` abre y cierra su propia conexión? ¿No es lento?

Porque es la práctica recomendada en .NET: **abrir tarde, cerrar temprano**. Una conexión abierta ocupa recursos del servidor y una que se queda abierta por descuido es una fuga. Cada método (`Tabla`, `Escalar`, `Ejecutar`, `Procedimiento`) la abre justo antes de ejecutar y la libera al salir del `using`, aunque ocurra una excepción.

Para que eso no sea lento existe el **pool de conexiones** de ADO.NET: cuando haces `Close()` (o sale el `using`) la conexión física con SQL Server **no se destruye**, se guarda en una reserva, y la siguiente vez que alguien abre una conexión con la misma cadena se reutiliza una de la reserva en lugar de negociar una nueva. Para el programa parece que abre y cierra sin parar, pero por debajo hay unas pocas conexiones reales que se reciclan. Es una gran ventaja de cerrar siempre y pronto.

El precio es una limitación que conviene conocer: como cada llamada de `Datos` usa **su propia conexión**, dos llamadas seguidas no pueden compartir una transacción. Si necesitas que varias operaciones salgan completas o ninguna, no puedes encadenar `Datos.Ejecutar(...)` varias veces; hay que abrir una sola conexión, iniciar una transacción y pasar ambas a cada comando, que es exactamente lo que hace `frmAgregarPersonal` (pregunta siguiente). Y otra solución es meter todas las operaciones dentro de un procedimiento almacenado, que ya es una unidad atómica por sí mismo.

14.4 ¿Qué es una transacción y cómo se usa desde C#? Explica el guardado de un empleado.

Una transacción es un grupo de operaciones que se tratan como **una sola**: o todas se confirman (`Commit`) o ninguna queda (`Rollback`). Se explica con la transferencia bancaria: restar de una cuenta y sumar a otra tienen que ocurrir las dos o ninguna; si se cortara la luz a mitad, no puede quedar el dinero descontado y sin llegar.

En `frmAgregarPersonal` , guardar un empleado implica tres cosas: la fila del empleado, su foto y sus materias. Si el empleado se guardara y fallara la foto, quedaría un registro a medias. Por eso el código hace:

```
using (SqlConnection con = Conexion.Conectar())
{
    con.Open();
    using (SqlTransaction tx = con.BeginTransaction())
    {
        try
        {
            int idEmp = GuardarEmpleado(con, tx);
            GuardarFoto(con, tx, idEmp);
            GuardarMaterias(con, tx, idEmp);
            tx.Commit();
        }
        catch { tx.Rollback(); throw; }
    }
}
```

La conexión se abre una sola vez y la transacción se crea con `BeginTransaction()` . Cada método auxiliar recibe **la misma conexión y la misma transacción** y crea sus comandos con `new SqlCommand(texto, con, tx)` : ese tercer argumento es lo que une cada comando a la transacción; sin él, ADO.NET lanza una excepción. `GuardarEmpleado` devuelve el Id recién generado (con el `SCOPE_IDENTITY()` de la pregunta 13.9) y los otros dos lo usan. Si cualquier paso falla, el `catch` hace `Rollback()` (todo lo anterior se deshace, incluido el empleado) y `throw;` relanza el error para que la pantalla lo muestre. Si todo sale bien, `Commit()` confirma.

Hay una red de seguridad adicional: la transacción está en un `using` , y si alguna vez se saliera del bloque sin `Commit()` , `Dispose()` la desharía. Pero se escribe el `Rollback` explícito porque deja la intención clara. Y una frase que resume la respuesta: «la

transacción garantiza que el empleado, su foto y sus materias son una sola unidad: o se guarda todo o no se guarda nada», que es, de hecho, lo que dice el comentario del código.

14.5 En los procedimientos almacenados aparecen `SET XACT_ABORT ON`, `BEGIN TRY` y `THROW`. ¿Para qué sirven y cómo los ve C#?

Son las herramientas del lado de SQL para que los errores se comporten bien, y C# depende de ellas.

`SET XACT_ABORT ON` dice: «si ocurre cualquier error dentro de una transacción, cancélala entera automáticamente». Sin esa opción, SQL Server a veces continúa ejecutando las instrucciones siguientes después de un error, lo que puede dejar datos a medias. `BEGIN TRY ... END TRY BEGIN CATCH ... END CATCH` es el `try/catch` de SQL: si algo falla en el bloque `TRY`, el control salta al `CATCH`. Dentro del `CATCH`, el patrón del proyecto es `IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; THROW;`: primero se deshace la transacción abierta (si la hay) y después `THROW;` sin argumentos **relanza** el error original hacia quien llamó. Es la misma idea que el `throw;` de C#.

Los errores de negocio se lanzan a propósito con `THROW 50034, 'No se puede prestar una unidad en mal estado...', 1;`. El número (a partir de 50000, que SQL Server reserva para números definidos por el usuario) identifica el error, el texto es lo que verá la persona, y el último valor es el estado.

Desde C#, todo eso llega como una `SqlException`. Su `Message` contiene el texto del `THROW` y su `Number` contiene el número. Por eso muchas pantallas muestran directamente `ex.Message`: el procedimiento ya escribió un mensaje en lenguaje claro. Y por eso otras pantallas leen `ex.Number` para decidir qué hacer (pregunta 14.7). El procedimiento `sp_RegistrarDevolucion` agrega algo más: `WITH (UPDLOCK)` al leer el préstamo, que bloquea esa fila hasta que termine la transacción para que dos personas no puedan devolver el mismo préstamo a la vez.

14.6 ¿Qué es una `SqlException` y qué significan los números `50000+`, `2601`, `2627` y `547`?

`SqlException` es el tipo de excepción que ADO.NET lanza cuando **SQL Server devolvió un error**, a diferencia de una excepción genérica de C#. Trae dos datos útiles: `Message`, el texto del error, y `Number`, el número con el que SQL Server lo identifica. Los números que importan en este proyecto:

- **50000 en adelante**: errores definidos por nosotros con `THROW` en los procedimientos y triggers. Cada regla de negocio tiene el suyo (por ejemplo 50306 «ese empleado ya tiene un pago en ese mes» o 50035 «la unidad ya está prestada»). Son los que se muestran tal cual al usuario.
- **2601 y 2627**: violación de un índice o restricción **única**, es decir, se intentó guardar un valor que ya existe (un nombre de salón repetido, un DUI repetido). 2601 es para índices únicos y 2627 para restricciones `UNIQUE` / `PRIMARY KEY`; muchas pantallas preguntan por ambos a la vez porque desde fuera significan lo mismo.
- **547**: violación de una **llave foránea** o de un `CHECK`. En el catálogo de salarios, intentar borrar una estructura que algunos empleados usan produce 547, y la pantalla lo traduce a «la estructura N° X está en uso y no se puede eliminar».

La idea es que **la base nunca deja pasar datos incoherentes y C# traduce el motivo a lenguaje humano**. El orden de la defensa es: la pantalla avisa lo evidente, el procedimiento verifica las reglas, y las restricciones de la tabla (llaves, `UNIQUE`, `CHECK`) son la última barrera que no depende de que nadie recuerde programar nada.

14.7 Compara las formas de mostrar un error de la base que aparecen en el proyecto.

Hay cuatro estilos, de menos a más refinado, y vale la pena reconocerlos.

1. **Mostrar el mensaje tal cual.** `catch (Exception ex) { MessageBox.Show("No se pudo registrar la salida: " + ex.Message); }`, como en los préstamos. Funciona porque el procedimiento escribió el mensaje para personas. Es lo más sencillo y sirve cuando basta con avisar.
2. **Filtrar por número con `when`.** En consumibles aparece `catch (SqlException ex) when (ex.Number == 2601 || ex.Number == 2627)`. El `when` es un **filtro de excepción**: ese `catch` solo se activa si se cumple la condición; si no, la excepción sigue buscando otro `catch`. Evita un `if` dentro del bloque.
3. **Un `switch` sobre `ex.Number`.** En `frmRegistrarPago`, `MostrarErrorSql` tiene un caso para cada número (50306 pago repetido, 50303 descuento excesivo, 50305 mes futuro, 50308 pago ya pagado…). No solo muestra un mensaje: marca con `ErrorProvider` el campo exacto que tiene el problema y le pone el foco. Es la mejor experiencia de usuario.
4. **Buscar el nombre de la restricción en el mensaje.** En `frmAgregarPersonal`, `ex.Message.Contains("UX_Empleados_DUI")` marca el campo del DUI y `"UQ_Empleados_Nombre"` marca el del nombre. Funciona porque SQL Server incluye el

nombre del índice violado en el texto del error. Es muy práctico y a la vez el más frágil: si alguien renombra el índice, el aviso preciso deja de aparecer (aunque el caso general `ex.Number >= 50000` y el `else` final siguen mostrando algo útil).

Puedes presentar esto como una evolución del equipo: las pantallas más antiguas muestran el mensaje, y las más recientes traducen cada error al campo que lo causó. Cualquiera de los cuatro estilos es válido; lo importante es poder explicar por qué el usuario nunca ve un volcado técnico, sino un mensaje que le dice qué corregir.

14.8 ¿Por qué se valida en C# y otra vez en la base? ¿Cómo funciona el patrón `ErrorProvider` con `hayError`?

Son dos defensas con objetivos distintos. La validación de la pantalla es **comodidad**: avisa al instante y marca exactamente dónde está el problema sin ir a la base. La validación de la base es **garantía**: se aplica sin importar de dónde venga el dato (otra pantalla, un script, una modificación futura del programa). Una pantalla puede tener un error o puede ser reemplazada; una restricción de la base no depende de ella. Por eso la regla de oro es: **nunca confiar solo en la pantalla**.

El patrón de la pantalla está en casi todos los formularios de guardado. Se empieza con `errorProvider.Clear(); bool hayError = false;`. Luego, para cada campo, `if (campo inválido) { errorProvider.SetError(control, "mensaje"); hayError = true; }`. Al final, `if (hayError) { MessageBox.Show("Revise los campos marcados"); return; }`. El `ErrorProvider` es un componente de Windows Forms que dibuja un icono rojo junto al control con un mensaje al pasar el ratón. La clave del patrón es que **no se detiene en el primer error**: revisa todos los campos y los marca a la vez, para que el usuario corrija todo de una vez en lugar de descubrir un error por intento. Es la diferencia entre un formulario que cansa y uno que ayuda.

Después de las validaciones de campo vienen las *reglas de negocio* (por ejemplo, que no existan dos directores) que requieren una consulta, y recién al final el guardado. El orden es deliberado: lo barato y local primero, lo que cuesta una consulta después, y lo que escribe en la base al final.

14.9 ¿Cómo se maneja la eliminación de algo que está en uso? Compara las estrategias del proyecto.

El sistema usa tres estrategias distintas, según lo que se quiera proteger, y las tres son consecuencia de la integridad referencial de la base.

La primera es **intentar y traducir el error**. En el catálogo de salarios se ejecuta `DELETE FROM Planillas WHERE IdPlanilla = @id` y, si esa estructura la usa algún empleado, la llave foránea hace fallar el `DELETE` con el error 547. El `catch` lo reconoce y muestra «está en uso y no se puede eliminar». La base es la que sabe si algo está en uso; C# no tiene que consultarlo antes.

La segunda es **hacer que la propia instrucción sea selectiva**. `Salon.QuitarActivo` borra solo las unidades sin historial gracias a los tres `NOT EXISTS`, y después compara cuántas borró con cuántas se pidieron para avisar al usuario de las que no se pudieron retirar.

La tercera es la **baja lógica**: no se borra nada, se cambia el estado (`UPDATE Empleados SET IdEstado = 2`, un activo «Dado de baja»). El registro sigue existiendo con todo su historial, simplemente deja de aparecer en las listas de uso diario. Es la estrategia preferida para entidades con historial, porque borrar a un empleado dejaría pagos, préstamos y auditoría apuntando a la nada. La baja de los activos, además, está protegida por reglas en la base: una baja no se revierte, y exige un motivo de al menos cinco caracteres.

14.10 ¿Qué pasa si SQL Server está apagado o la base no existe al arrancar el programa?

Depende de dónde ocurra, y el proyecto lo contempla en dos puntos. Al iniciar, `Program.Main` ejecuta `ConfiguracionInicial.EsPrimerUso()` dentro de un `try/catch`: si la base no responde, el programa muestra «No se pudo verificar la configuración» con el texto del error y termina con `return`, en lugar de dejar al usuario frente a una ventana que falla de formas confusas. Es una forma de *fallar temprano y con un mensaje claro*.

En el login, `ValidarCredenciales` tiene su propio `try/catch` y, si la conexión falla, activa la bandera `conexionFallida` y muestra un cuadro «Error de conexión al validar credenciales». Esa bandera sirve para que el código posterior **no cuente el intento como contraseña equivocada ni lo registre como acceso fallido**: un servidor caído no es culpa del usuario y no debe bloquearlo.

En las demás pantallas, cada método de carga (`CargarPrestamos`, `CargarActivos`…) tiene su `try/catch` que muestra el error en un `MessageBox` en vez de cerrar el programa. No hay un manejador global que atrape todo lo imprevisto; cada operación protege la suya. Eso es sencillo de seguir, aunque implica que alguien debe acordarse de poner el `try/catch` en cada método nuevo.

14.11 ¿Qué pasa si dos personas editan el mismo registro a la vez?

Es una pregunta de diseño muy buena, y la respuesta honesta tiene dos partes. El sistema está pensado como una aplicación de escritorio con una base local, normalmente usada por una persona a la vez, así que el caso es poco probable. Aun así, la base tiene protecciones para los casos que sí duelen: `sp_RegistrarDevolucion` bloquea la fila del préstamo con `UPDLOCK` dentro de una transacción, el índice único filtrado `UX_Prestamos_UnidadActiva` impide dos préstamos activos de la misma unidad aunque dos personas lo intenten a la vez, y las restricciones `UNIQUE` frenan duplicados sin importar quién gane la carrera.

Lo que **no** existe es *concurrencia optimista* en la edición general: no hay una columna de versión (`rowversion`) que detecte «alguien modificó este registro mientras lo tenías abierto». Si dos personas editan el mismo empleado, gana quien guarda último y sus cambios sobrescriben los del otro sin aviso. Para el alcance del proyecto es razonable, y si te lo preguntan la mejora es concreta y fácil de explicar: agregar una columna `rowversion` y comprobar en el `UPDATE` que no haya cambiado desde que se cargó el formulario (`WHERE IdEmpleado = @id AND Version = @versionOriginal`), avisando al usuario si `ExecuteNonQuery` devuelve 0 filas.

15. Preguntas y respuestas, bloque 4: lógica de las pantallas y seguridad

Aquí está la lógica del programa: cómo se organiza el panel, cómo funciona un formulario de punta a punta y cómo se protege el acceso. Son preguntas donde un profesor suele pedir que «expliques con tus palabras», así que las respuestas siguen el hilo de una explicación oral: primero la idea, después el código que la sostiene y, cuando corresponde, el límite honesto.

15.1 ¿Cómo funciona el panel principal y por qué las pantallas se «incrustan» en lugar de abrirse como ventanas?

`frmDashboard` es una ventana con dos zonas: el menú lateral a la izquierda y un contenedor (`pnlContenedor`) a la derecha. Cuando haces clic en un ítem, por ejemplo Personal, no se abre otra ventana: se **crea el formulario de esa sección y se coloca dentro del contenedor**, como si fuera un panel más. El método `AbrirFormularioEnContenedor` hace tres cosas: primero elimina y libera (`Dispose`) lo que hubiera en el contenedor; luego configura el formulario nuevo con `TopLevel = false` (le dice a Windows que ya no es una

ventana independiente, sino un control hijo), `FormBorderStyle = None` (sin barra de título ni bordes) y `Dock = DockStyle.Fill` (que ocupe todo el espacio); y por último lo agrega y llama a `Show()`.

La ventaja es de experiencia: el usuario siente que está en una sola aplicación con un menú fijo, no en una pila de ventanas. Y de recursos: al cambiar de sección el formulario anterior se destruye, así que no se acumulan pantallas en memoria. La contrapartida es que cada vez que vuelves a una sección **se crea de nuevo y vuelve a consultar la base**; el estado de la pantalla anterior (un filtro escrito, una fila seleccionada) se pierde. Para este sistema es una buena decisión porque garantiza datos frescos.

Quien orquesta todo es `Navegar(item, Func<Form> crearFormulario)`: ejecuta la receta para crear el formulario, lo incrusta, marca el ítem del menú como activo (`MarcarActivo`, que cambia su color con el patrón «estado → Invalidate») y actualiza las insignias. Todo dentro de un `try/catch` que muestra «No se pudo abrir la sección» si algo falla al construir la pantalla, en lugar de dejar caer la aplicación. Cada clic de menú es una línea: `itmPersonal_Click → Navegar(itmPersonal, () => new frmGestionPersonal())`.

15.2 ¿Qué son las insignias del menú y qué hace el temporizador? ¿Por qué hay tantos `try/catch` vacíos?

Las insignias son los pequeños números sobre los ítems: los reportes de daño abiertos junto a Reportar, y los pagos pendientes del mes junto a Planillas. `ActualizarInsignias` los calcula con dos consultas pequeñas: una trae `COUNT(*)` y `SUM(CASE WHEN Prioridad = 'Alta' ...)` de `vw_ReportesDanio` donde el estado es abierto (el color de la insignia pasa a rojo si hay alguna de prioridad alta y a ámbar si no), y la otra cuenta con `Datos.Escalar` los pagos pendientes no anulados del mes actual. Un `Timer` de Windows Forms (`timerInsignias`) dispara el método cada 60 segundos, y también se llama tras cada navegación, de modo que los números se mantienen al día sin que nadie haga nada.

El `Timer` ejecuta su evento `Tick` en el hilo de la interfaz, así que puede tocar controles sin las precauciones de los hilos. Y como tiene vida propia, hay que apagarlo al cerrar: `frmDashboard_FormClosing` llama a `timerInsignias.Stop()` y `timerMenu.Stop()`. Es el mismo principio de «lo que enciendes, apaga» que el `Application.AddMessageFilter` del panel, que se quita en `FormClosed`.

Los `catch { itmReportar.Insignia = 0; }` que ves son una decisión consciente: una insignia es **decoración informativa**, no una operación crítica. Si la consulta falla (la base está ocupada un instante), lo peor que debe pasar es que el número se muestre en cero, y

no que aparezca un cuadro de error cada minuto interrumpiendo el trabajo. Diferencia importante con el catch vacío del login: allí lo que se silencia es una bitácora; aquí, un adorno. Saber justificar cada silencio es lo que separa un `catch` vacío útil de uno peligroso.

Además, el panel responde a atajos con `ProcessCmdKey`: **F1** abre el manual de usuario (un PDF en la carpeta `Ayuda` junto al programa) y **Ctrl+B** pliega o despliega el menú. Sobrescribir `ProcessCmdKey` y devolver `true` significa «este atajo ya lo atendí, no lo reenvíes a ningún control», lo que evita que, por ejemplo, la tecla llegue a un cuadro de texto enfocado.

15.3 Explica cómo funciona un CRUD completo con el ejemplo de Personal.

CRUD son las cuatro operaciones: crear, leer, actualizar, eliminar. En Personal están repartidas entre dos pantallas.

Leer. `frmGestionPersonal` carga la lista con la consulta de la pregunta 13.2 en un `DataTable` y la enlaza a la cuadrícula. El buscador y los filtros trabajan sobre esa tabla en memoria. Es la única pantalla de lectura y la que actúa de «puerta de entrada» a las demás operaciones.

Crear y actualizar comparten un solo formulario, `frmAgregarPersonal`. Se llama «modo dual»: el formulario recibe en su constructor un `idEmpleadoEditar`. Si vale `0`, es un empleado nuevo; si trae un número, es una edición y el formulario carga los datos de esa persona en los campos. Al guardar, el mismo método decide con `if (idEmpleadoEditar == 0)` si ejecuta un `INSERT` o un `UPDATE` (pregunta 13.9). La ventaja es que las validaciones, los campos y el aspecto existen una sola vez. El truco de `YaExisteDirector(idEmpleadoEditar)` encaja aquí: el mismo Id sirve para «no contéis al que estoy editando».

La pantalla de lista abre el formulario con `ShowDialog` y, si devuelve `DialogResult.OK`, vuelve a llamar a `MostrarEmpleados()` para recargar: la lista nunca se actualiza «por su cuenta», siempre se recarga desde la base tras un cambio, así que lo que ves es lo que hay guardado.

Eliminar no borra: es una **baja lógica** que cambia el estado del empleado. Desaparece de la lista de activos pero conserva pagos, ausencias y auditoría. Antes de hacerlo la pantalla comprueba si tiene asuntos pendientes y advierte. Hay una pequeña ayuda de seguridad en la interfaz: la confirmación tiene seleccionado el botón «No» por defecto (`MessageBoxDefaultButton.Button2`), de modo que pulsar Enter sin pensar cancela en lugar de confirmar.

15.4 ¿Cómo se combinan el buscador de texto y los filtros de una lista?

Con un patrón que se repite en personal, salones, usuarios y otras listas. Cada filtro que está activo aporta **una condición**, y las condiciones se unen con `AND`. En `frmSalones.AplicarFiltroSalones` se ve completo: se crea una `List<string> cond`; si la casilla de inactivos no está marcada, se agrega `"Activo = true"`; si el combo de tipo tiene una opción elegida, se agrega `"Tipo = '...'"`; si el cuadro de búsqueda tiene texto, se agrega un `LIKE` sobre varias columnas unidas con `OR`; y al final `vistaSalones.RowFilter = string.Join(" AND ", cond);`.

El resultado es que cada control suma una restricción y todas se cumplen a la vez: ver los salones activos, de cierto tipo, cuyo nombre contenga lo escrito. Si ningún filtro está activo, `cond` queda vacío, `string.Join` devuelve cadena vacía y `RowFilter` vacío muestra todas las filas, sin necesidad de un caso especial. Los eventos `TextChanged` y `SelectedIndexChanged` de los controles llaman a ese mismo método, por eso el resultado cambia mientras escribes. Y recuerda de la pregunta 13.7 que el texto del usuario pasa por `Escapar` antes de entrar a la expresión.

Para explicarlo en la defensa: «no hay un `if` para cada combinación de filtros; cada filtro agrega su condición a una lista y se aplican todas juntas». Ese diseño escala: agregar un filtro nuevo es agregar una línea, sin tocar las existentes.

15.5 ¿Cómo se restringe lo que el usuario puede escribir? ¿Qué pasa con el DUI?

Hay dos niveles. El primero es **impedir la tecla**: en el evento `KeyPress` del cuadro de texto, si el carácter no es válido se pone `e.Handled = true`, que significa «ya lo manejé, no lo escribas». El nombre solo admite letras y espacios, el teléfono solo dígitos, `+`, `-` y espacio, y el DUI solo dígitos y guion. Las teclas de control (borrar, flechas) se dejan pasar con `char.IsControl`. Con esto el usuario ni siquiera puede teclear una letra en el teléfono. El segundo nivel es validar **al guardar** (el patrón `hayError`), que sigue siendo necesario porque se puede pegar texto desde el portapapeles saltando el `KeyPress`.

Para el DUI hay ayudas extra: si escribes los nueve dígitos seguidos, el formulario coloca el guion solo, y al guardar `DuiValido` comprueba el formato `00000000-0`. El comentario sobre ese método explica la regla del dígito verificador (multiplicar los ocho primeros dígitos por 9, 8, 7, …, 2, sumar y calcular `(10 - suma % 10) % 10`). Y aquí hay un hallazgo que conviene que conozcas: en el código actual el método hace `return true;` justo después de comprobar el formato, **antes** de llegar al cálculo, así que ese cálculo nunca se ejecuta (el compilador lo marcaría como código inalcanzable). En la práctica el sistema valida el formato del DUI, pero no su dígito verificador, aunque el aviso de error y el tooltip

digan lo contrario. También conviene saber que esta validación solo se aplica cuando quien edita es administrador (`esAdmin`). Si te lo preguntan, la respuesta honesta es «está implementado el algoritmo pero quedó desactivado; activarlo es borrar esa línea», y conviene probarlo con DUI reales antes de la defensa.

15.6 ¿Cómo se controla quién entra y qué puede ver? ¿Por qué el modo Debug se comporta distinto?

El control tiene tres capas. La primera es el **login**: solo entran usuarios activos cuyo rol es `Administrador` ; cualquier otro rol recibe «esta cuenta no tiene permiso para usar el sistema». La segunda es la clase `Sesion` , que guarda `Usuario` , `Rol` y `NombreCompleto` mientras la aplicación está abierta. La tercera, en el panel, es `AplicarPermisosSidebar` : si el rol no es `Administrador` , oculta con `Ocultar(item)` casi todas las secciones. Como el login ya filtra, en la práctica hoy todos los que llegan al panel son administradores; esa tercera capa es una preparación para que, si algún día se permiten otros roles, el menú ya sepa qué esconder. Ocultar es una medida de interfaz: el ítem desaparece del menú, pero la protección real de datos para otros roles tendría que estar también en la base.

Sobre el modo Debug: en `Program.Main` hay una directiva de compilación `#if DEBUG` . Cuando compilas en la configuración Debug (la de desarrollo), el programa arranca **directamente en el panel**, y en `frmDashboard_Load` , si no hay rol, se asigna `Administrador` automáticamente. En Release (la configuración con la que se entrega el programa) arranca en `frmLogin` , y si alguien abriera el panel sin sesión, se cierra. Es una comodidad para que el equipo no tenga que escribir la contraseña en cada prueba. El riesgo es evidente y se explica en una frase: **hay que entregar y presentar siempre la compilación Release**, porque en Debug el login queda saltado. Además, con esa sesión sin usuario real, la auditoría registra los cambios como «(sistema)».

15.7 ¿Qué es BCrypt y por qué se usa un hash en lugar de guardar la contraseña (o cifrarla)?

Un **hash** es una función de un solo sentido: convierte un texto en una cadena fija de aspecto aleatorio, y es matemáticamente impracticable volver del hash al texto. Guardar contraseñas como hash significa que ni siquiera quien administra la base de datos puede leerlas. **Cifrar** es distinto: es reversible con una clave, y si alguien roba la clave, obtiene todas las contraseñas. Para validar un acceso no hace falta recuperar la contraseña; basta con calcular el hash de lo que escribió el usuario y compararlo con el guardado.

Pero no cualquier hash sirve. Uno rápido como MD5 o SHA se puede probar miles de millones de veces por segundo, así que un atacante con la tabla robada adivinaría las contraseñas comunes en minutos. **BCrypt** está diseñado para ser **deliberadamente lento** y tiene dos propiedades clave. Primero, incluye un *salt*: un valor aleatorio distinto para cada contraseña que se mezcla antes de calcular el hash, de modo que dos usuarios con la misma contraseña tienen hashes totalmente distintos y las tablas precalculadas no sirven. Segundo, un *factor de costo* que indica cuánto trabajo exige cada cálculo; se puede aumentar con los años conforme los equipos son más rápidos. El resultado es una cadena de unos 60 caracteres que empieza por `$2` (algo como `$2a$11$...`) y contiene la versión, el costo, el salt y el hash.

En el código es muy poco: `PasswordHasher.Hash(clave)` llama a `BCrypt.Net.BCrypt.HashPassword`, y `PasswordHasher.Verificar(clave, hash)` llama a `BCrypt.Net.BCrypt.Verify`, que lee el salt y el costo de dentro del propio hash para recalcular y comparar. La contraseña **nunca se compara en SQL** ni viaja a la base: SQL solo entrega el hash guardado y la comparación ocurre en C#. Un detalle de honestidad: en el login hay una rama de compatibilidad con valores antiguos guardados en texto plano (explicada en la sección 7.2), que los acepta y los convierte a hash al primer ingreso.

15.8 ¿Cómo se genera una contraseña temporal y por qué no se usa `Random`?

`ServicioUsuarios.GenerarClaveTemporal` crea una clave de 10 caracteres que cumple tres condiciones: tiene al menos una mayúscula, una minúscula y un número, y excluye los caracteres que se confunden al leer (`I`, `O`, `l`, `o`, `0` y `1`). Es una clave para dictar o enviar, y nadie debe dudar entre una ele y un uno.

La pieza clave es de dónde sale el azar. La clase `Random` de .NET genera números que **parecen** aleatorios pero son predecibles: parte de una semilla y, conociendo unos pocos resultados, se puede deducir el resto. Para contraseñas eso es inaceptable. Por eso el código usa `RandomNumberGenerator`, el generador **criptográfico** del sistema operativo, diseñado para que sus resultados sean impredecibles.

Hay un segundo detalle fino en el método auxiliar `Azar`. Para elegir un número entre 0 y `max` se podría hacer `valor % max`, pero eso favorece levemente a los números bajos cuando el rango del generador no es múltiplo exacto de `max` (el llamado *sesgo de módulo*). El método lo evita con un **muestreo por rechazo**: calcula un límite múltiplo exacto de `max` y, si el número aleatorio cae por encima, lo descarta y vuelve a sortear. Es un cuidado que casi nadie nota y que demuestra rigor.

Por último, la clave se **mezcla** con el algoritmo de Fisher–Yates (recorre de atrás hacia adelante intercambiando cada posición con una elegida al azar). Esto es necesario porque las primeras tres posiciones se llenan con una mayúscula, una minúscula y un dígito para garantizar la variedad; sin mezclar, todas las claves empezarían con esa misma estructura. La clave se guarda como hash, se marca `DebeCambiarClave = 1` en el usuario, y en el próximo ingreso el login obliga a cambiarla antes de entrar.

15.9 Explica el bloqueo por intentos fallidos y sus límites.

El login lleva dos diccionarios `static` en memoria: `fallosPorUsuario` (cuántos fallos lleva cada usuario) y `bloqueadoHasta` (hasta cuándo está bloqueado). La llave es el nombre de usuario en minúsculas. Con cada contraseña equivocada el contador sube; desde el tercer fallo el mensaje avisa cuántos intentos quedan; al quinto (`MaxFallos = 5`) se guarda una hora de desbloqueo 60 segundos en el futuro (`SegundosBloqueo = 60`) y se borra el contador. Mientras esa hora no llegue, `btnIngresar_Click` rechaza el intento **antes de consultar la base**. Un acierto borra ambos registros.

El objetivo es hacer inútil la adivinanza por fuerza bruta: con un bloqueo de un minuto cada cinco intentos, probar mil contraseñas le tomaría al atacante más de tres horas. Solo se cuentan las contraseñas equivocadas, no los rechazos por cuenta desactivada o sin permiso, ya que esos no son intentos de adivinar.

Los límites, para decirlos tú antes de que te los pregunten: el contador vive **en memoria**, así que cerrar y reabrir el programa lo borra; está por usuario y no por equipo, así que un atacante podría probar una contraseña por cada usuario sin llegar al bloqueo (aunque adivinar nombres de usuario ya es otro problema); y bloquear por nombre permite que alguien moleste a un usuario legítimo escribiendo mal su contraseña a propósito. La versión de producción guardaría los intentos en la base y los asociaría también al origen. Para un sistema de escritorio local la medida es proporcionada.

15.10 Describe el flujo de recuperación de contraseña por correo y qué decisiones de seguridad tiene.

Tiene dos pasos en la misma ventana. En el primero, el usuario escribe su correo; el programa valida el formato, consulta con `SELECT COUNT(1) FROM Usuarios WHERE Correo = @correo` si existe y, si existe, genera un **código de seis dígitos** con el generador criptográfico, lo envía por correo con `ServicioCorreo.EnviarCodigoAsync` (de forma asíncrona, para no congelar la ventana) y guarda en memoria el código, el correo y la hora de vencimiento (`DateTime.Now.AddMinutes(15)`). En el segundo paso aparecen los campos del código y de la contraseña nueva. Al pulsar Restablecer, el programa revisa **en**

este orden: que las dos contraseñas coincidan, que el código no haya expirado, que el código sea el enviado. Si es incorrecto suma un fallo y a los cinco (`MaxIntentos = 5`) invalida el proceso y pide solicitar un código nuevo. Si todo está bien, calcula el hash de la contraseña nueva y ejecuta `UPDATE Usuarios SET Clave = @clave WHERE Correo = @correo`.

Las decisiones que valen la pena: el código caduca (15 minutos), tiene intentos limitados (5), se genera con aleatoriedad criptográfica y la contraseña nunca se guarda sin hash. Las limitaciones honestas: la pantalla dice «Este correo no está registrado» cuando no existe, lo que revela a un desconocido qué correos pertenecen al sistema (se priorizó la claridad sobre la privacidad); el código se verifica en el propio programa y no en el servidor; y el `UPDATE` cambia la clave de todos los usuarios que compartan ese correo, lo que supone que cada correo es de un solo usuario. También está la regla de contraseña: en el sistema hay más de un criterio de longitud y composición según la pantalla donde se defina (6, 8, o 8 con letras y números), una incoherencia menor que se arreglaría con una función de validación única compartida.

15.11 ¿Cómo funciona el asistente de primer uso?

Al arrancar, `Program.Main` llama a `ConfiguracionInicial.EsPrimerUso()`, que ejecuta `SELECT COUNT(*) FROM ConfiguracionEmpresa` y devuelve `true` si la tabla está vacía. Si lo es, abre `frmConfiguracionInicial` como diálogo; si el usuario lo cancela (`ShowDialog() != OK`), el programa termina. Así el sistema **no puede usarse sin haber sido configurado**. Después se llama a `ConfiguracionInicial.Cargar()` para leer el nombre y el logotipo de la institución, que se usan en todo el programa.

El asistente son tres páginas: datos de la institución (nombre, dirección, teléfono, correo y logo), el administrador inicial (solo si todavía no existe uno, lo que se decide con `necesitaAdmin`) y la verificación del correo con un código. Lo más cuidado es el control de estado: las banderas `enviando` y `guardando` evitan que un doble clic envíe dos códigos o guarde dos veces; el reenvío del código tiene un enfriamiento de 60 segundos; y la verificación del correo se puede omitir, en cuyo caso se guarda como no verificado.

Al terminar, `GuardarInicial` **calcula el hash de la contraseña en C#** con BCrypt y luego llama al procedimiento `sp_GuardarConfiguracionInicial`, pasándole el hash (nunca la contraseña) y el logotipo como `VarBinary`. Dentro del procedimiento se guarda la configuración y se crea el administrador en una sola transacción. El logotipo se declara con `cmd.Parameters.Add("@Logotipo", SqlDbType.VarBinary, -1)`; el `-1` es el tamaño máximo, para que la imagen no se trunque.

15.12 ¿Cómo envía correos el sistema y qué riesgo hay en cómo está configurado?

El sistema no habla directamente con un servidor de correo. `ServicioCorreo.Enviar` hace una petición **HTTP POST** con un cuerpo JSON a un servicio web (un script de Google Apps Script) y ese servicio es el que manda el correo. El JSON incluye el tipo de mensaje (código o soporte), el destino y el contenido. El método lee la respuesta, verifica que contenga `"ok":true` y, si no, lanza una excepción con el motivo. Todo se ejecuta dentro de `Task.Run` para no bloquear la interfaz, y se fuerza TLS 1.2 con `SecurityProtocol` para conexiones seguras.

La ventaja de este diseño es que el programa de escritorio **no guarda la contraseña de ninguna cuenta de correo**: la credencial vive en el servicio de Google. Pero hay un riesgo que debes conocer: la dirección del servicio y un *token* de acceso están escritos como constantes en el código fuente de `ServicioCorreo`. Quien tenga el código (o descompile el programa, o vea el repositorio si es público) puede usar ese servicio para enviar correos. No es una contraseña de la base ni de una cuenta personal, pero sí un secreto compartido. Las mitigaciones son concretas: regenerar el token si el repositorio es o se vuelve público, no incluir el valor real en copias que se compartan, y a futuro sacarlo del código hacia un archivo de configuración o variable de entorno. En la defensa vale la pena presentarlo como una decisión de diseño con un riesgo conocido y una mejora definida, que es el tipo de respuesta que suena a ingeniero.

16. Preguntas y respuestas, bloque 5: diseño y dibujo por código

El diseño es lo primero que ve cualquiera que abra el sistema, así que es muy probable que te pidan explicar cómo se logró. La clave es hablar de ideas y no de líneas sueltas: qué es dibujar en Windows Forms, cómo un control cambia de aspecto según su estado y por qué se eligió dibujar por código en lugar de usar imágenes. Las respuestas siguientes se apoyan en el código real de `ItemMenuLateral`, `IconosMenu`, `ControlesTema` y `frmLogin`.

16.1 ¿Cómo se dibuja en Windows Forms? ¿Qué son `Graphics`, `Pen`, `SolidBrush` y `OnPaint`?

Cada control de Windows Forms tiene una superficie rectangular y, cuando Windows necesita mostrarla (porque apareció, porque cambió de tamaño, porque otra ventana la tapó y se destapó), llama al método `OnPaint` y le pasa un `PaintEventArgs`. Dentro viene `e.Graphics`, el **lienzo**: un objeto con métodos para dibujar líneas (`DrawLine`), formas (`DrawEllipse`, `DrawRectangle`, `DrawArc`), rellenos (`FillRectangle`, `FillPath`) y curvas (`DrawBezier`). La librería se llama GDI+.

Para dibujar necesitas dos herramientas: un **lápiz** (`Pen`) para contornos, que lleva color y grosor, y un **pincel** (`SolidBrush`) para rellenos, que lleva color. Son recursos de Windows, por eso se crean dentro de `using`, para liberarlos apenas terminan de usarse (pregunta 12.7); con decenas de repintados por segundo, olvidarlo sería una fuga real. El texto se pinta con `TextRenderer.DrawText`, que ofrece una calidad más parecida a la del resto de Windows que el método equivalente de `Graphics`, y permite alinear, centrar y recortar con «puntos suspensivos» (`EndEllipsis`).

La regla de oro del diseño por código es que **el dibujo se hace dentro del evento de pintado y nunca «por fuera»**. Si quieres que algo cambie de aspecto, no lo dibujas tú en ese momento: cambias el estado del control (`hover = true`) y llamas a `Invalidate()`, que le pide a Windows que vuelva a invocar `OnPaint`. Así el aspecto siempre es una función del estado actual, y es el patrón que se repite en todas las clases de diseño del proyecto.

16.2 Explica cómo se dibuja un icono de la barra lateral. ¿Por qué por código y no con imágenes?

Los iconos viven en la clase `IconosMenu`, en un solo método: `Dibujar(Graphics g, string nombre, RectangleF caja, Color color)`. Recibe el lienzo, el nombre del icono («inicio», «activos», «personal»…), el cuadrado donde debe caber y el color. La idea central es que **todos los iconos se diseñan sobre una cuadrícula imaginaria de 24 por 24 unidades**, y el método los amplía o reduce al tamaño pedido.

Eso se logra con tres líneas al inicio: `g.TranslateTransform(caja.X, caja.Y)` mueve el origen de coordenadas a la esquina del cuadrado; `float k = caja.Width / 24f;` `g.ScaleTransform(k, k)` aplica una escala para que 24 unidades equivalgan al ancho real. Desde ese momento, todo lo que se dibuje con coordenadas de 0 a 24 cae donde debe, sin importar el tamaño. El ítem del menú lo llama con una caja de 22 por 22 píxeles. Al comenzar, el método hace `GraphicsState estado = g.Save();` y al terminar restaura, para que las transformaciones no «se escapen» y descoloquen lo que se dibuje después.

El lápiz se crea con `new Pen(color, 1.8f)` y con `StartCap = LineCap.Round`, `EndCap = LineCap.Round` y `LineJoin = LineJoin.Round`, que redondean las puntas y las esquinas; ese detalle es el que da a todos los iconos un trazo suave y coherente, con aspecto de «familia». Después un `switch` por nombre dibuja cada icono con primitivas muy sencillas. La casa de «inicio» son tres llamadas a `DrawLines`: el tejado (tres puntos formando un ángulo), las paredes (cuatro puntos) y la puerta (cuatro puntos). La «caja» de activos es un polígono de seis vértices más dos líneas internas que sugieren profundidad. «Personal»

son dos círculos (`DrawEllipse`) para las cabezas y dos arcos de media circunferencia (`DrawArc(..., 180, 180)`) para los hombros. «Actividad» es un círculo y dos líneas que forman las agujas de un reloj.

¿Por qué por código? Por cuatro razones que conviene tener a mano. **Escala sin pérdida**: son geometría, no píxeles, así que se ven nítidos en el menú grande y en el plegado. **Color dinámico**: el icono recibe el color como parámetro, de modo que el mismo dibujo es gris, blanco, amarillo o rojo según el estado, sin necesidad de cuatro imágenes. **Cero archivos externos**: no hay carpeta de recursos que se pueda perder o corromper. Y **coherencia**: todos comparten el mismo grosor, puntas y cuadrícula. El costo es que diseñar un icono nuevo exige pensar en coordenadas en vez de dibujarlo con el ratón.

16.3 ¿Cómo cambia de aspecto un ítem del menú según su estado? Explica `ItemMenuLateral`.

`ItemMenuLateral` hereda de `Control` (no de `Button`), es decir, parte de un lienzo en blanco y lo pinta todo él. En su constructor activa `ControlStyles.UserPaint` («yo me pinto»), `OptimizedDoubleBuffer` y `AllPaintingInWmPaint` (el dibujo se arma primero fuera de pantalla y se muestra de una vez, para evitar parpadeo) y `ResizeRedraw` (que se repinte al cambiar de tamaño).

El control guarda su estado en variables privadas: `activo`, `hover`, `peligro`, `compacto`, `insignia`, el texto y el nombre del icono. Cada una tiene una propiedad pública cuyo `set` llama a `Invalidate()` (pregunta 12.2), y los eventos `OnMouseEnter` / `OnMouseLeave` cambian `hover` y repintan. Todo el aspecto sale de `OnPaint`, que sigue siempre el mismo orden. Primero limpia con el color del fondo del padre. Si el ítem está activo o con el ratón encima, dibuja detrás un rectángulo redondeado de color suave (más claro si está activo, rojizo si es el botón de «peligro», que es cerrar sesión). Si está activo, agrega la **barra amarilla** vertical a la izquierda. Luego elige el color del icono y del texto con cadenas de condiciones: gris si está deshabilitado, amarillo si está activo, blanco si el ratón está encima, tono normal en los demás casos. Dibuja el icono con `IconosMenu.Dibujar`, el texto con `TextRenderer.DrawText` (en negrita si está activo) y, si hay notificaciones, la **insignia**.

La insignia es una cápsula: se mide el texto con `TextRenderer.MeasureText`, se calcula un ancho (mínimo 22, o el texto más 14), se genera la forma redondeada con radio 10 (la mitad de la altura, lo que produce extremos totalmente curvos) y se rellena con el color de la insignia; el número se limita a «99+». En el modo **compacto** (menú plegado) no hay texto: la

insignia se reduce a un punto de color con un anillo del color del fondo que lo separa del icono. Un único método, `OnPaint`, resuelve las cuatro combinaciones de estado sin crear imágenes ni controles adicionales.

Finalmente, el control sobrescribe `Dispose` para liberar sus tres fuentes (`Font`), que se crean una vez y se reutilizan en cada repintado. Los atributos `[Category("Menú")]` y `[DefaultValue(...)]` sobre las propiedades hacen que, al seleccionarlo en el diseñador de Visual Studio, `Texto`, `Icono`, `Activo` y los demás aparezcan agrupados y se puedan editar sin escribir código.

16.4 ¿Cómo se logran las esquinas redondeadas? Explica `Redondeado` y `Region`.

GDI+ no tiene un «rectángulo redondeado» integrado, así que el proyecto lo construye con un `GraphicsPath`: una **ruta** formada por segmentos que luego se rellena (`FillPath`) o se contornea (`DrawPath`). El método `Redondeado(Rectangle r, int radio)` calcula el diámetro `d = radio * 2` y agrega cuatro arcos, uno por esquina:

```
p.AddArc(r.X,         r.Y,          d, d, 180, 90);   // esquina superior
izquierda
p.AddArc(r.Right - d, r.Y,          d, d, 270, 90);   // superior derecha
p.AddArc(r.Right - d, r.Bottom - d, d, d,   0, 90);   // inferior derecha
p.AddArc(r.X,         r.Bottom - d, d, d,  90, 90);   // inferior izquierda
p.CloseFigure();
```

`AddArc(x, y, ancho, alto, anguloInicial, barrido)` agrega un cuarto de circunferencia que cabe en un cuadrado de lado `d` ubicado en esa esquina; los ángulos 180, 270, 0 y 90 recorren las esquinas en el sentido del reloj empezando por la superior izquierda, y cada arco barre 90 grados. GDI+ une automáticamente el final de un arco con el inicio del siguiente mediante una línea recta (que forma los lados), y `CloseFigure()` cierra la forma. Con radio 8 se obtiene la tarjeta del ítem de menú; con radio 10 y altura 20, una cápsula perfecta.

`Region` es otra herramienta, distinta: define la **forma física del control**. En `frmLogin`, `pnlTarjeta.Region = new Region(Redondeado(..., 18))` recorta el panel de la tarjeta de inicio de sesión para que sea redondeado: lo que cae fuera de la región simplemente no existe, ni para el dibujo ni para el ratón. La diferencia con `FillPath` es la que conviene saber: `FillPath` **pinta** una forma dentro del control; `Region` **recorta** el control mismo. La contrapartida de `Region` es que sus bordes no son suavizados (no hay antialiasing), por eso la tarjeta se ve bien y las formas pequeñas de los ítems se dibujan con `FillPath`.

16.5 ¿Qué es `PaletaSistema` y por qué se centralizan los colores?

`PaletaSistema` es una clase estática con las nueve constantes de color de la identidad visual: `Fondo` (casi negro, 18,18,18), `Tarjeta` (azul muy oscuro, 27,29,38) para paneles, `Campo` (38,42,58) para cuadros de texto, `Borde` (45,50,63), `Amarillo` (250,204,21) como color de acento, `Gris` (55,60,80) para botones secundarios, `Rojo` (220,53,69) para lo destructivo, `Texto` (240,244,252) y `Tenue` (170,180,200) para texto secundario. Cada nombre dice **para qué sirve** el color, no cómo es; se escribe `PaletaSistema.Amarillo`, no `Color.FromArgb(250,204,21)` en cada sitio.

La ventaja de centralizar es que cambiar la identidad visual (otro color de acento, un tema más claro) sería tocar un archivo. Y la consistencia: dos pantallas que usan la misma constante nunca se ven «casi igual». El identificador del proyecto es el contraste: fondo oscuro, texto claro y un solo color cálido de acento que guía la mirada hacia lo importante (el ítem activo, el botón principal, el foco del campo).

Para ser honesto con el estado actual: la paleta está bien aprovechada en `BotonTema`, `PanelTarjeta` y `ComboOscuro`, pero varias clases (como `ItemMenuLateral` o `GridOscuro`) declaran sus propios colores `static readonly` con valores muy cercanos, y algunas pantallas escriben colores directamente. No es un fallo de funcionamiento, es una deuda de orden: el siguiente paso natural sería que todas lean de `PaletaSistema`. Presentarlo así demuestra que sabes distinguir la intención del diseño de su estado actual.

16.6 ¿Cómo funcionan `BotonTema` y `PanelTarjeta`? ¿Qué se gana con clases propias?

`BotonTema` hereda de `Button` y fija en el constructor todo lo que haga falta para el aspecto plano: `FlatStyle.Flat`, borde de tamaño cero, fuente semibold. Su propiedad `Tipo` (un `enum` con `Principal`, `Secundario` y `Peligro`) decide los colores en el método `Aplicar`: amarillo con letra oscura para el principal, gris para el secundario, rojo para el peligroso, y los tonos apagados si el botón está deshabilitado. El efecto al pasar el ratón se logra con `FlatAppearance.MouseOverBackColor` y `MouseDownBackColor`, calculados con `Aclarar(color, delta)`, que suma o resta un valor a los canales rojo, verde y azul con `Math.Max/Min` para no salirse del rango 0–255. Aquí no hay `OnPaint` propio: se aprovecha lo que el botón estándar ya sabe hacer y solo se configura. Es el ejemplo de la solución más sencilla que cumple el objetivo.

`PanelTarjeta` es un `Panel` con fondo `Tarjeta`, doble búfer, y un `OnPaint` que llama primero a `base.OnPaint(e)` y luego dibuja un contorno de un píxel con el color `Borde`. Un panel normal ya hace lo demás.

Lo que se gana con clases propias es que **el estilo se escribe una vez y se usa en todo el sistema**. En el diseñador solo hay que elegir `BotonTema` en vez de `Button`, y fijar `Tipo`; no hay que repetir cinco propiedades de color en cada botón de cada formulario. Si el equipo decide cambiar el amarillo, cambia en todos los botones a la vez. Esa es la razón por la que el sistema se siente uniforme: no porque cada pantalla se haya cuidado, sino porque todas usan las mismas piezas.

16.7 ¿Qué es el doble búfer? ¿Por qué aparecen `OptimizedDoubleBuffer`, `AllPaintingInWmPaint` y `ResizeRedraw`?

Sin ayuda, cuando un control se repinta, Windows primero borra el área con el color de fondo y después dibuja encima en varios pasos. Si el usuario alcanza a ver esos pasos intermedios, percibe un **parpadeo**. El doble búfer lo evita: el dibujo completo se arma primero en una imagen oculta en memoria y se copia a la pantalla de una sola vez, de modo que nunca se ve un cuadro a medias.

En Windows Forms se activa con indicadores de `ControlStyles`. `OptimizedDoubleBuffer` activa el doble búfer. `AllPaintingInWmPaint` indica que todo el dibujo, incluido el fondo, ocurre en el evento de pintado y no en un borrado previo separado. `UserPaint` dice que el control se pinta a sí mismo. `ResizeRedraw` ordena repintar completo cada vez que cambia de tamaño; sin él, al redimensionar quedarían restos del dibujo anterior, por ejemplo una forma redondeada mal estirada. Para los formularios completos hay un atajo, `DoubleBuffered = true`.

En `frmLogin` hay además una técnica más avanzada: se sobrescribe `CreateParams` y se agrega `cp.ExStyle |= 0x02000000`, que es el estilo de ventana `WS_EX_COMPOSITED`. Ese indicador pide a Windows que componga la ventana y todos sus hijos como una unidad antes de mostrarla, lo que elimina el parpadeo al pintar muchos controles superpuestos (tarjeta, campos, iconos). Es un valor numérico de la API de Windows, por eso aparece escrito como hexadecimal.

16.8 ¿Cómo se logra una tabla oscura con filas que reaccionan al ratón? Explica `GridOscuro`.

`GridOscuro` hereda de `DataGridView` y, en su constructor, configura de una vez todo lo que el diseñador pediría a mano: desactiva agregar, borrar y redimensionar filas, la pone de solo lectura, selección de fila completa (`FullRowSelect`) y una sola fila a la vez, oculta los encabezados de fila y fija alturas (40 para el encabezado y 36 para las filas). Después define los colores: encabezado `(34,38,52)`, filas alternas `FilaA` y `FilaB` ligeramente distintas (el efecto «cebra» ayuda a seguir una fila con la vista), un color de selección y otro de ratón encima. `EnableHeadersVisualStyles = false` es indispensable: sin él, Windows ignora los colores del encabezado y aplica su tema.

El efecto de **resaltar la fila bajo el ratón** sigue el patrón de estado e invalidación. Una variable `filaHover` guarda el índice de la fila actual (o -1). Los eventos `OnCellMouseEnter` y `OnCellMouseLeave` llaman a `CambiarHover`, que actualiza esa variable y, en lugar de repintar toda la tabla, invalida solo las dos filas afectadas con `InvalidateRow(anterior)` e `InvalidateRow(nueva)`. Repintar solo lo que cambió es lo que mantiene el movimiento fluido incluso con muchas filas. Luego, el evento de formato de celda (`OnCellFormatting`) mira si la fila es `filaHover` y cambia su color de fondo.

Las barras de desplazamiento también son propias: las estándar de Windows son claras y rompen el tema oscuro. El control crea dos instancias de `BarraFina` (una vertical y una horizontal), un control que se pinta a sí mismo como una línea fina con una píldora redondeada, y las coloca **justo encima de las barras nativas**: en `Sincronizar` copia la posición y el tamaño de la barra de Windows (`barra.Bounds = nativa.Bounds`), la trae al frente con `BringToFront()` y la muestra solo si la nativa está visible. Así las barras claras quedan tapadas y la tabla sigue usando su mecanismo normal de desplazamiento. La sincronización se dispara cuando cambian el diseño, el desplazamiento, las filas, las columnas o los datos, y se programa con `BeginInvoke` (con una bandera `actualizacionPendiente` para que varias solicitudes seguidas se junten en una sola) para esperar a que la tabla termine de procesar su mensaje. Es el control más trabajado del diseño, y el que más se nota cuando una tabla larga se desplaza.

Hay dos detalles más en `GridOscuro` que le dan su sello: `OnRowPostPaint` dibuja una **franja amarilla de tres píxeles** a la izquierda de la fila seleccionada, y `OnPaint` traza una **línea amarilla de dos píxeles bajo los encabezados**, con transparencia parcial. Son pequeños, y son lo que hace reconocible la tabla en cualquier pantalla del sistema.

16.9 ¿Qué diferencia hay entre `CellFormatting` y `CellPainting`? ¿Cómo se dibujan las cápsulas de estado en las tablas?

Son dos niveles de control sobre cómo se ve una celda. **`CellFormatting`** se ejecuta justo antes de mostrar la celda y te deja **cambiar sus propiedades**: color de fondo, color de letra, texto. Es sencillo y sirve cuando quieres «esta fila va en otro color». Lo usa la tabla de préstamos: lee la columna `Alerta` y pone un fondo rojo oscuro y letra rosada si es «Atrasado», ámbar si es «Por vencer» y letra atenuada si ya fue devuelto. **`CellPainting`**, en cambio, te permite **dibujar la celda tú mismo** con `e.Graphics`, sin las limitaciones del formato estándar. Es lo que hace falta para una cápsula, porque una celda normal solo puede pintar un fondo plano.

En `frmSalones` el método `PintarPildora` muestra el procedimiento completo. Primero llama a `e.Paint(e.CellBounds, e.PaintParts & ~DataGridViewPaintParts.ContentForeground)`: le pide al grid que pinte todo lo suyo (fondo, selección, bordes) **excepto el texto**, y marca `e.Handled = true` para avisar «ya me encargo del resto». Luego mide el texto con `TextRenderer.MeasureText`, calcula el ancho de la cápsula (texto más relleno, sin pasarse de la celda), la centra y la rellena con `FillPath` usando el color base con **transparencia parcial** (`Color.FromArgb(55, baseColor)`, donde 55 es el canal alfa de 0 a 255). La transparencia es lo que hace que la cápsula se adapte al fondo y se vea tenue sobre filas claras u oscuras, y que el texto, en el color pleno, destaque. Por último, escribe el texto encima.

La regla práctica para decidir: si solo necesitas cambiar un color o un texto, usa `CellFormatting`; si necesitas una forma nueva, usa `CellPainting` y recuerda `e.Handled = true`.

16.10 ¿Cómo se estilizan los menús desplegables y el calendario? ¿Qué es P/Invoke?

Algunos controles de Windows no permiten cambiar todos sus colores con propiedades. Para el desplegable, `ComboOscuro` usa dos técnicas. La lista de opciones se pinta con `DrawMode = OwnerDrawFixed` y se sobrescribe `OnDrawItem`: el propio código dibuja cada opción (fondo gris si está resaltada, fondo `Campo` si no, y el texto con `TextRenderer`). El cuadro cerrado, en cambio, no pasa por `OnDrawItem`, así que se intercepta el mensaje de Windows: se sobrescribe `WndProc`, el método que recibe todos los mensajes del sistema, y después de dejar que el control haga lo suyo (`base.WndProc`), si el mensaje es

`WM_PAINT` (0x000F, «pintémonos»), se llama a `Dibujar()`, que cubre el cuadro con el fondo oscuro, un borde que cambia de color (amarillo si tiene el foco, más claro si el ratón está encima), el texto y una flecha fina dibujada con `DrawLines`.

Para el calendario desplegable, `FechaOscura` necesita ir más lejos. El calendario emergente de un `DateTimePicker` es una ventana propia de Windows con tema claro. El código le pregunta a Windows cuál es esa ventana enviando el mensaje `DTM_GETMONTHCAL` (0x1008) con `SendMessage` y, con el manejador obtenido, llama a `SetWindowTheme(cal, "", "")`, que le quita el tema visual y permite que acepte los colores asignados. Algo parecido hace `BarrasOscuras` con las barras de desplazamiento estándar de otros controles: `SetWindowTheme(control.Handle, "DarkMode_Explorer", null)` pide a Windows su tema oscuro.

Esas funciones, `SendMessage` y `SetWindowTheme`, no existen en .NET: pertenecen a la API nativa de Windows (`user32.dll` y `uxtheme.dll`). Se acceden mediante **P/Invoke** (*Platform Invocation Services*): se declara la firma con `[DllImport("uxtheme.dll")] private static extern int SetWindowTheme(...)` y a partir de ahí se llama como cualquier método. Es una técnica avanzada y un buen ejemplo del trabajo del equipo para lograr un tema oscuro completo, no solo parcial. Su límite: depende de características de Windows 10 en adelante, y en versiones más antiguas el aspecto podría variar.

16.11 Explica los detalles del login: el subrayado de los campos, el icono del ojo y el aviso de mayúsculas.

El login no usa cuadros de texto con borde: cada campo es un `Panel` que contiene un `TextBox` sin borde, y el panel dibuja una **línea inferior de dos píxeles** en su evento `Paint` con `FillRectangle(b, 0, p.Height - 2, p.Width, 2)`. El color de esa línea lo decide una cadena de condiciones: `Rojo` si hay error en ese campo, `Amarillo` si el cuadro tiene el foco, y un tono neutro si no. Los eventos de foco, de cambio de texto y de clic llaman a `Invalidate()` para que la línea cambie al instante. Hacer clic en cualquier parte del panel pasa el foco al `TextBox` (`Campo_Click`), de modo que toda el área parece el campo.

El botón del **ojo** que muestra u oculta la contraseña se dibuja con curvas de Bézier. Una curva de Bézier `DrawBezier(lápiz, inicio, control1, control2, fin)` tiene cuatro puntos: parte del primero, termina en el último, y los dos intermedios «tiran» de la curva sin estar sobre ella. El ojo son **dos curvas** con los mismos extremos (`cx - 10` y `cx + 10`) y los puntos de control hacia arriba y hacia abajo (`cy - 9` y `cy + 9`), que forman la almendra del párpado; un círculo relleno de 7 píxeles (`FillEllipse`) es la pupila. Cuando

la contraseña es visible se agrega una diagonal que tacha el ojo y el color pasa a amarillo. El clic solo alterna una bandera `claveVisible`, cambia `PasswordChar` entre `'\0'` (mostrar) y `'•'` (ocultar) y llama a `Invalidate()`.

El aviso de **bloqueo de mayúsculas** usa `Control.IsKeyLocked(Keys.CapsLock)`: se revisa al escribir, al recibir foco y al activar la ventana, y se muestra solo si el bloqueo está activo y el cursor está en el campo de contraseña. Son detalles pequeños que no cambian la lógica, pero son los que hacen que el login se sienta cuidado, y por eso conviene que puedas explicarlos.

16.12 ¿Cómo se construyen las tarjetas y las gráficas de la pantalla de inicio?

`frmInicio` combina tres cosas: tarjetas de indicadores, tarjetas de alerta y gráficas, y las tres siguen la misma regla: **los datos vienen de consultas y la pantalla solo los presenta**. Los **indicadores** son seis paneles dentro de un `TableLayoutPanel` que ya existen en el diseñador, cada uno con dos etiquetas (el valor grande y un detalle pequeño). `CargarIndicadores` obtiene cada número con una consulta de agregado, normalmente un `COUNT(*)` o `SUM(CASE ...)` (pregunta 13.8), y un método auxiliar `Kpi(valor, detalle, texto, ...)` rellena las dos etiquetas y sus colores. Si quien entra no es administrador, `ConfigurarSegunRol` oculta dos de ellos y reparte el ancho entre los otros cuatro. Las consultas pasan por un método `Consulta` que envuelve `Datos.Tabla` en un `try/catch` y devuelve `null` si algo falla, y por ayudantes como `Entero` y `Primera` que convierten `DBNull` y tablas vacías en valores seguros: la pantalla de inicio es lo primero que se ve y no debe caerse porque una consulta falle.

Las **tarjetas de alerta** (préstamos atrasados, reportes de daño abiertos, pagos pendientes) sí se construyen enteras por código. Para cada alerta se crea un `Panel` con una barra de color de cinco píxeles a la izquierda (`Dock = Left`), un título, un detalle y, si es clicable, un texto «Abrir ›». Se enlazan los eventos de clic y de ratón de todos esos controles para que la tarjeta cambie de fondo al pasar el cursor y, al hacer clic, llame a `IrA(destino)`. Ese método busca el panel principal con `Application.OpenForms.OfType<frmDashboard>()` y le pide navegar con `BeginInvoke`, para esperar a que termine el clic. Un indicador que lleva al detalle convierte la pantalla de inicio en un tablero de control y no en un mero adorno. Un detalle simpático: el encabezado cambia el saludo según la hora («Buenos días», «Buenas tardes», «Buenas noches») y muestra una barra con el porcentaje del año transcurrido, calculado con `hoy.DayOfYear` entre 365 o 366.

Las **gráficas** usan el control `Chart` de `System.Windows.Forms.DataVisualization`. `EstilizarChart` limpia las series, pone el título en blanco arriba a la izquierda, fondo transparente, etiquetas de ejes en gris claro, línea de ejes tenue, cuadrícula horizontal suave y sin marcas de graduación ni cuadrícula vertical: el mismo criterio de «menos ruido, más contraste» del resto del sistema. Los datos llegan de una consulta SQL a un `DataTable` y se agregan a la serie recorrido por recorrido. Si la consulta no devuelve nada, `SinDatos(chart)` muestra un mensaje en lugar de una gráfica vacía. Y la consulta de los últimos seis meses con la tabla de meses fabricada en SQL (pregunta 13.8) es lo que garantiza que el eje temporal no tenga huecos.

16.13 ¿Cómo se comporta el diseño en pantallas distintas? ¿Es «responsive»?

Hay que separar dos ideas. Una aplicación de escritorio de Windows Forms no es «responsive» en el sentido de una página web, que reorganiza su contenido según el ancho. En este proyecto, el panel principal usa `FormBorderStyle.FixedSingle`, no permite maximizar (`MaximizeBox = false`) y fija un tamaño máximo igual al área de trabajo de la pantalla donde se abre (`MaximumSize = Screen.FromControl(this).WorkingArea.Size`). Las pantallas incrustadas usan `Dock = Fill` para ocupar todo el contenedor, de modo que se ajustan al espacio disponible, y algunas utilizan `Anchor` para que los controles sigan los bordes.

La decisión más importante es que el login y el panel tienen `AutoScaleMode = None`: es decir, el diseño no se reescala automáticamente según los puntos por pulgada (DPI) del monitor. Eso da control exacto del dibujo (los iconos y las cápsulas caen en el píxel previsto) a cambio de que en pantallas de muy alta densidad o con un escalado de Windows distinto del esperado el tamaño pueda no ser el ideal. Por eso el ajuste del tamaño máximo a la pantalla es relevante: evita que el panel quede más grande que el monitor.

Un detalle cosmético que conviene conocer: en algunas tablas la última fila puede aparecer cortada cuando la altura disponible no es múltiplo de la altura de las filas; es un comportamiento estándar de `DataGridView` y no afecta a los datos. La página web responsiva que pide la rúbrica es otra parte del trabajo, del equipo web, y es donde sí corresponde hablar de adaptación por ancho. En la defensa del programa de escritorio, lo honesto es decir que se priorizó un diseño cuidado y consistente para un tamaño de ventana controlado.

17. Preguntas y respuestas, bloque 6: preguntas trampa, arquitectura y mejoras

Este último bloque junta las preguntas difíciles: las que buscan una debilidad, las que piden comparar decisiones y las que preguntan qué harías distinto. Una defensa no se gana ocultando los límites del diseño, sino mostrando que los conoces y que sabes cómo se corregirían. Todo lo que se menciona como defecto está verificado en el código; si prefieres corregir algo antes de la presentación, el orden de prioridad está en la pregunta 17.10.

17.1 Demuéstrame que el sistema no tiene inyección SQL.

La mejor respuesta es una demostración y no una afirmación. En el login, escribe en el usuario `' OR '1'='1` y cualquier contraseña: el sistema responde «Usuario o contraseña incorrectos». Si la consulta estuviera armada pegando texto, esa entrada habría convertido la condición en algo siempre verdadero. No lo hace porque la consulta es `WHERE u.Usuario = @user` y el valor viaja como parámetro: la base busca literalmente un usuario con ese nombre extraño y no encuentra a nadie.

Luego explica el criterio con el que revisaste el código: todos los valores que escribe una persona (nombres, códigos, fechas, textos de búsqueda) entran a la base como parámetros, por `AddWithValue` o por `Datos.P`. Donde aparece un `+` dentro de una consulta, lo que se pega es una **constante del programa**: el fragmento de filtro elegido por un `switch` en los préstamos, o el `, DUI = @dui` condicionado a ser administrador. Los filtros de las listas usan `RowFilter`, que no es SQL y nunca llega a la base, y escapan el texto con `Escapar`. Y las listas de números para un `IN (...)` se construyen con enteros ya convertidos. Si el profesor encuentra un caso concreto que parezca sospechoso, la pregunta que se hace es siempre la misma: «¿de dónde viene lo que se concatena?».

Un límite que sí conviene reconocer: la defensa contra inyección depende de **disciplina del programador** (acordarse de usar parámetros en cada consulta nueva), no de una barrera técnica. Un ORM o un acceso solo mediante procedimientos reduciría ese riesgo, a costa de otras cosas (pregunta 17.7).

17.2 ¿Se puede entrar al sistema sin conocer la contraseña?

Por el camino normal, no: la contraseña se verifica con BCrypt y hay bloqueo por intentos. Pero hay dos excepciones reales que conviene conocer y corregir antes de presentar.

La primera es el **modo Debug**. En `Program.Main`, una directiva `#if DEBUG` hace que, al compilar para desarrollo, el programa abra directamente el panel sin pasar por el login (y `frmDashboard_Load` le asigna el rol Administrador). En la compilación Release esto no existe, el panel sin sesión se cierra solo. La solución es una regla de trabajo: **presentar y entregar siempre en Release**, y si quieres estar seguro, comprobarlo abriendo la aplicación compilada y viendo que pide usuario.

La segunda es la **rama de contraseñas antiguas en texto plano** de `ValidarCredenciales` (sección 7.2). Si una cuenta tiene en `Clave` un valor que no empieza con `$2`, el login lo compara directamente y, si coincide, lo acepta sin revisar si la cuenta está activa ni si su rol es Administrador. Es decir, esa rama solo se activa para cuentas que aún no han migrado a hash, pero para ellas se salta dos controles. Puedes comprobar en segundos si te afecta con una consulta: `SELECT Usuario FROM Usuarios WHERE Clave NOT LIKE '$2%'`. Si no devuelve filas, la rama no se usa con tus datos. Y la corrección es sencilla: migrar esos valores una vez (o regenerar sus claves) y borrar la rama, o mover las dos comprobaciones para que se apliquen siempre.

17.3 ¿Por qué las reglas de contraseña son distintas según la pantalla?

Es una inconsistencia real y se explica por cómo creció el proyecto: cada pantalla definió su propia regla. El asistente de primer uso exige mínimo 8 caracteres (`MinClave = 8`). La recuperación por correo exige solo 6 (`MinClave = 6`). Y `ServicioUsuarios.ValidarClaveNueva`, que usan el cambio de clave y la gestión de usuarios, exige 8, que no empiece ni termine con espacios y que combine letras y números. El efecto práctico es que un administrador puede acabar con una contraseña más débil por un camino que por otro: la recuperación acepta `abcdef`, que la gestión de usuarios rechazaría.

Hay además un detalle fino de coherencia con el login: el login recorta los espacios del inicio y del final de lo que escribes (`Trim()`), la recuperación también, pero el asistente de primer uso guarda la contraseña **tal cual** la escribió la persona. Si alguien eligiera en el asistente una contraseña que empieza o termina con un espacio, se guardaría con ese espacio y después el login, que lo recorta, no la reconocería nunca. Es un caso muy raro, pero es exactamente el tipo de cosa que se evita con una sola función de validación.

La corrección tiene una forma clara: que todas las pantallas llamen a `ServicioUsuarios.ValidarClaveNueva` y apliquen `Trim()` de la misma manera. Es un cambio pequeño y una buena muestra de refactorización: **una regla, un lugar**.

17.4 ¿Hay código duplicado o código que no se usa?

Sí, y es normal en un proyecto que crece en lotes, pero se debe poder nombrar. La **duplicación** más visible es la validación de correo: existe un método `CorreoValido` casi idéntico en cuatro formularios (`frmConfiguracionInicial` , `frmRecuperarContraseña` , `frmAgregarPersonal` y `frmCentroSoporte`), y `GenerarCodigo` aparece dos veces, en el asistente y en la recuperación. El riesgo de duplicar es que un arreglo se haga en un sitio y se olvide en los otros. La solución estándar es moverlos a una clase de utilidades compartida (por ejemplo, en `Modelos`) y que todos la llamen.

El **código sin uso** son clases de una etapa anterior del proyecto. `Modelos/Empleado.cs` , `AsignarCargo.cs` y `Planilla.cs` contienen consultas a la base, pero no encontré que las pantallas actuales las llamen: las pantallas hacen sus propias consultas con `Datos` . `Dbsetup.cs` tiene la lógica para crear la base, pero el arranque del programa no la utiliza (la base se instala con los scripts de la carpeta `db`). Dejarlas no rompe nada, pero tiene un costo: confunden a quien lee el código y envejecen mal. Un ejemplo: `Empleado.EliminarEmpleado` ejecuta un `DELETE` sin abrir antes la conexión, así que si alguien la llamara fallaría y devolvería `false` sin explicar nada. Nadie lo nota justamente porque nadie la usa. Y la versión antigua de `YaExisteDirector` en `AsignarCargo.cs` compara contra `IdCargo` fijos (1 y 2), distinta de la que sí está en uso.

Si te lo preguntan, la actitud correcta es decir: «sabemos qué es código heredado; no lo borramos antes de la entrega para no arriesgar nada a última hora, y está identificado para limpiarlo».

17.5 ¿El proyecto usa el patrón MVC?

No en sentido estricto, y es mejor decirlo así. MVC separa tres responsabilidades: el **Modelo** (datos y reglas), la **Vista** (lo que se ve) y el **Controlador** (que recibe las acciones y coordina). El proyecto tiene una separación parecida en dos proyectos de Visual Studio: `Vistass` contiene las pantallas y `Modelos` contiene la conexión, el acceso a datos, los servicios (correo, usuarios) y las utilidades. Pero la mayoría de las pantallas **mezclan** los tres roles en su propio archivo: el formulario dibuja la interfaz, atiende el clic (el controlador) y contiene la consulta y la regla de negocio (como `YaExisteDirector` en `frmAgregarPersonal` o `ValidarCredenciales` en `frmLogin`).

Ese estilo tiene nombre: *code-behind*, y es el que Windows Forms promueve por defecto. Es rápido de escribir y fácil de seguir para un equipo pequeño, porque todo lo de una pantalla está en un lugar. Su desventaja es que la lógica queda atada a la interfaz: no se puede probar sin abrir la ventana ni reutilizar en otra pantalla (de ahí la duplicación de la

pregunta anterior). Lo que sí está bien separado es la parte más importante: **las reglas de negocio críticas viven en la base de datos**, en procedimientos, triggers y restricciones, así que no dependen de qué pantalla las invoque. La respuesta honesta y completa es «usamos una arquitectura en capas simple, con el acceso a datos centralizado en `Datos` y las reglas críticas en SQL Server; no aplicamos MVC estricto».

17.6 ¿Qué patrones de diseño reconoces en el código?

Sin forzar la comparación, hay varios que puedes señalar con ejemplos reales. **Observador** (*Observer*): todos los eventos de Windows Forms lo son; un control avisa y los métodos suscritos reaccionan (`Click`, `Tick`, `SelectedIndexChanged`). **Estado global compartido**, parecido a un *Singleton*: la clase estática `Sesion` guarda el usuario actual para todo el programa. **Fachada** (*Facade*) o clase de ayuda: `Datos` oculta tres líneas de ADO.NET detrás de `Datos.Tabla(...)` o `Datos.Escalar(...)`, y `PasswordHasher` oculta la librería BCrypt detrás de dos métodos. **Plantilla o herencia especializada**: `BotonTema`, `GridOscuro` o `ComboOscuro` heredan de un control estándar y cambian solo lo necesario (`override`). **Estrategia** liviana mediante delegados: `Navegar(item, Func<Form>)` recibe la receta de lo que debe construir. Y los métodos de fábrica de `DialogoBase`, que construyen botones y campos de diálogo con el mismo estilo.

Lo importante en la defensa es no inventar: nombra el patrón, señala el archivo donde aparece y explica qué problema resuelve. Si no puedes señalar dónde está, mejor no mencionarlo.

17.7 ¿Por qué ADO.NET directo y no Entity Framework? ¿Por qué hay SQL en C# y también procedimientos?

Son dos decisiones relacionadas. **ADO.NET con SQL escrito a mano** (`SqlConnection`, `SqlCommand`) da control total y transparencia: ves exactamente qué se ejecuta, y para un proyecto de aprendizaje donde el objetivo es entender cómo se conectan C# y la base, es el camino más didáctico. Un ORM como Entity Framework traduce objetos a SQL automáticamente, reduce código repetitivo y previene errores de tipo, pero esconde el SQL y, para este alcance, habría agregado una capa de complejidad (migraciones, contexto, mapeos) que enseña menos y arriesga más a última hora.

Sobre **dónde va el SQL**, el proyecto usa un criterio mixto que puedes formular así: las **lecturas y los filtros** viven en C#, porque dependen de la pantalla y cambian seguido (la lista de personal, las consultas de resumen); las **operaciones que cambian datos y tienen reglas** viven en procedimientos almacenados (registrar un préstamo, generar la planilla, devolver un equipo), porque así la regla y su transacción son atómicas, se aplican igual

desde cualquier pantalla y pueden auditarse. Las inserciones y actualizaciones sencillas de personal van directo desde C# con parámetros y una transacción. Es un equilibrio razonable, aunque no un esquema rígido, y conviene decirlo así en lugar de afirmar que todo sigue una sola regla.

17.8 ¿Por qué Windows Forms y no WPF, o una aplicación web?

Windows Forms es la tecnología de escritorio más directa de .NET Framework: arrastras controles, escribes el código de los eventos y funciona. Para un sistema administrativo local, con una base en el mismo equipo (LocalDB), cubre lo necesario sin servidor web, hospedaje ni autenticación de red. Y tiene una ventaja que este proyecto aprovecha: GDI+ permite **dibujar cualquier control por código** (iconos, cápsulas, tablas oscuras), lo que hace posible una identidad visual propia sin librerías externas.

WPF permite interfaces más ricas con estilos declarativos (XAML) y un enlace de datos más potente, pero su curva de aprendizaje es mayor. Una aplicación web habría permitido acceso desde cualquier lugar, pero exige servidor, seguridad de red y otra arquitectura. La rúbrica pide ambas cosas, y el reparto es natural: el sistema de escritorio es la herramienta de gestión y la página web responsiva (trabajo del equipo web) es la cara pública. Si te preguntan qué limitación tiene WinForms, la respuesta honesta es que está en mantenimiento en lugar de evolución, y que no se adapta a pantallas de distintas densidades con la misma soltura que WPF.

17.9 ¿Hay pruebas automatizadas? ¿Cómo probaste el sistema?

Hay que responder con franqueza: en el repositorio no hay proyecto de pruebas automatizadas. Las pruebas fueron **manuales**, recorriendo los flujos de cada pantalla, intentando datos inválidos y revisando que los procedimientos rechacen lo que deben. Eso encuentra mucho, pero no se repite solo: cada cambio obliga a volver a probar a mano.

Lo que sí ayuda es que gran parte de las reglas están en la base de datos como procedimientos, que son fáciles de probar de forma independiente: ejecutas `EXEC sp_RegistrarPrestamo ...` con datos válidos e inválidos y compruebas qué error aparece. Para mejorar, hay dos niveles. Las **pruebas unitarias** de la lógica pura que hoy está atrapada en los formularios (validar un DUI, generar una clave temporal, el filtro de `Escapar`): al moverla a clases sin interfaz, se prueba con MSTest o NUnit en milisegundos. Y las **pruebas de integración** de los procedimientos contra una base de prueba, que verifiquen que, por ejemplo, un préstamo sobre una unidad ya prestada lance el error

50035. Un ejemplo concreto del valor de esas pruebas: el `return true;` que dejó desactivado el dígito verificador del DUI habría sido detectado de inmediato por una prueba con un DUI de dígito incorrecto.

17.10 ¿Qué mejorarías con una semana más? Ordena por prioridad.

Ordenado por riesgo y esfuerzo, de lo más urgente y barato a lo más estructural:

1. **Presentar en Release y revisar que no queden claves en texto plano** (`SELECT ... WHERE Clave NOT LIKE '$2%'`). Cinco minutos y cierra la única forma realista de saltarse el login.
2. **Sacar el token del servicio de correo del código** hacia un archivo de configuración o una variable de entorno, y regenerarlo si el repositorio es público. Es el único secreto que viaja con el código.
3. **Unificar la regla de contraseñas** en `ValidarClaveNueva` y aplicar `Trim()` igual en todas las pantallas.
4. **Reparar la rama de texto plano del login** (aplicar activo y rol siempre) o eliminarla después de migrar.
5. **Reactivar el dígito verificador del DUI** quitando el `return true;` y probando con DUI reales.
6. **Reglas de cargo (director, subdirectora, docente) basadas en identificadores** o en una columna propia, no en el texto del nombre, y reforçadas con un índice o un trigger en la base. Hoy un cargo renombrado cambia el comportamiento.
7. **Concurrencia optimista** con una columna `rowversion` en las tablas de edición frecuente.
8. **Bloqueo de login persistente** en la base y no en memoria.
9. **Una sola paleta**: que `ItemMenuLateral`, `GridOscuro` y las pantallas lean todos de `PaletaSistema`.
10. **Limpieza y pruebas**: borrar las clases heredadas sin uso, mover `CorreoValido` y `GenerarCodigo` a una utilidad compartida, y empezar un proyecto de pruebas.

Presentar una lista así, priorizada y con el motivo de cada ítem, es en sí misma una respuesta de alto nivel: muestra que ves el sistema como un producto que puede evolucionar.

17.11 Si te piden agregar un campo nuevo al empleado (por ejemplo, «Nivel educativo»), ¿qué tocas y en qué orden?

Es una pregunta excelente porque obliga a recorrer todas las capas. El orden natural es de abajo hacia arriba. **Primero la base**: `ALTER TABLE Empleados ADD NivelEducativo ...` con su tipo, si es obligatoria o no, y una restricción si tiene valores permitidos. Segundo, la **vista** `vw_EmpleadosDetalle` y cualquier consulta que deba mostrarlo. Tercero, la **auditoría**: los triggers `trg_Aud_*` listan las columnas de la tabla al generarse, así que hay que regenerar el de `Empleados` para que el cambio del campo nuevo quede registrado. Cuarto, el **C#**: en `frmAgregarPersonal`, agregar el control al formulario (en el diseñador), validarlo en el bloque de `hayError`, añadirlo al `INSERT` y al `UPDATE` de `GuardarEmpleado` con su parámetro, y cargarlo en el modo de edición; en `frmGestionPersonal`, agregarlo a la consulta de la lista si debe verse. Quinto, los **reportes** (expediente, exportaciones) si deben incluirlo. Y por último, probar: crear, editar, buscar y ver que la auditoría lo registre.

Observa lo que esa respuesta demuestra: que sabes que un dato recorre base, vista, auditoría, formulario, lista y reportes, y que un cambio «pequeño» toca todas las capas. Quien puede enumerarlas entiende la arquitectura mejor que quien solo sabe dibujar la pantalla.

17.12 Explícame el sistema en dos minutos.

Esta pregunta casi siempre aparece, y conviene tenerla ensayada. Una versión posible, para adaptar con tus palabras:

«Es un sistema de escritorio en C# y Windows Forms para administrar una institución escolar: personal, planillas, activos y préstamos, salones y reportes de daño. Por debajo hay una base de datos en SQL Server con tablas relacionadas, vistas, procedimientos almacenados y triggers de auditoría. El código se organiza en dos proyectos: uno con las pantallas y otro con el acceso a datos y los servicios. Todo el acceso a la base pasa por una clase, `Datos`, que abre una conexión corta, usa siempre parámetros y devuelve los resultados, y cada conexión le dice a la base qué usuario está conectado para que la auditoría sepa quién hizo cada cambio.

La seguridad tiene capas: las contraseñas se guardan con BCrypt, el login se bloquea tras cinco intentos, hay recuperación por correo con código que caduca, y la primera vez que se ejecuta un asistente crea la configuración y el administrador. Las reglas importantes viven en la base, así que se cumplen sin importar la pantalla. Y lo más visible es el diseño: un tema oscuro con acento amarillo, hecho enteramente por código con GDI+, desde los

iconos del menú hasta las tablas y los desplegables, con clases propias que cualquier pantalla reutiliza. Conocemos sus límites, como la validación de contraseñas dispar o la falta de pruebas automatizadas, y tenemos identificado cómo corregirlos.»

En cerca de dos minutos pasa por qué es, cómo está hecho, cómo se protege, qué lo distingue y qué le falta. Esa estructura funciona para cualquier proyecto.

17.13 ¿Qué respondes si te preguntan algo que no sabes?

La tentación es inventar, y es lo peor que puedes hacer, porque un profesor experimentado lo nota y a partir de ahí duda de todo lo demás. Hay tres movimientos que funcionan mejor. El primero es **reconocerlo con precisión**: «eso específico no lo tengo claro» es mejor que un rodeo. El segundo es **razonar en voz alta con lo que sí sabes**: «no recuerdo el nombre exacto de esa función, pero sé que su propósito es X y que se llama desde Y; puedo abrir el archivo y mostrártelo». Abrir el código y leerlo con el profesor es perfectamente válido y muestra que sabes dónde buscar. El tercero es **conectar con algo cercano que sí domines**, sin forzarlo: si te preguntan por una consulta que no recuerdas, explica el patrón general (parámetros, `JOIN`, `ExecuteScalar`) y cómo lo aplicarías.

Y una práctica para antes de la presentación: elige tres pantallas, ábrelas sin mirar la guía y cuenta en voz alta el flujo con la receta de cinco frases de la sección 10. Si te trabas en un punto, ese es el que debes releer.

18. Glosario: las palabras técnicas que conviene dominar

Un proyecto como este usa mucho vocabulario técnico, y buena parte de entender el sistema es no perderse con las palabras. Este glosario junta los términos que aparecen en la guía y en el código, con una definición corta y el lugar donde se explican a fondo. Están agrupados por tema para que puedas repasarlos por bloques. Si una palabra te suena pero no sabrías explicarla, ese es justo el término que te conviene practicar en voz alta.

18.1 Auditoría y seguridad

Término	Qué significa
Auditoría	El registro automático de quién hizo qué y cuándo. En este sistema vive en la tabla `Auditoria`: cada cambio importante deja una fila con el usuario, la fecha, la tabla, el tipo de operación y cómo era el dato antes y después. Se explica en la sección 6 de la guía de base de datos.
Auditar / tabla auditada	Auditar es registrar los cambios de algo. Una tabla auditada es una que tiene un trigger que anota cada inserción, modificación y borrado. En este proyecto son 17 de las 30 tablas.
Trigger (disparador)	Un programa de la base de datos que se ejecuta **solo**, cuando ocurre un `INSERT`, `UPDATE` o `DELETE` en una tabla. Los `trg_Aud_...` son los que escriben la auditoría. Nadie los llama desde el código C#.
Registro de actividad / bitácora	Otros nombres de la auditoría. En el menú se ve como la pantalla **Actividad**.
Inmutable	Que no se puede cambiar. La tabla `Auditoria` rechaza cualquier `UPDATE` o `DELETE` (error 50900), así nadie borra sus huellas.
`SESSION_CONTEXT`	Un espacio de datos adjunto a cada conexión de SQL Server. `Conexion.cs` guarda ahí el usuario del sistema para que los triggers sepan quién hizo el cambio (pregunta 14.2).
Autenticación y autorización	Autenticar es comprobar **quién eres** (usuario y contraseña); autorizar es decidir **qué puedes hacer** (el rol). El login hace lo primero y lo segundo (solo entran administradores).
Rol	Un tipo de usuario con ciertos permisos (`Administrador`, `Empleado`). Vive en la tabla `Roles`.

Término	Qué significa
Hash	El resultado de pasar un texto por una función de un solo sentido: de la contraseña sale una cadena de la que no se puede volver atrás. Sirve para guardar contraseñas sin que nadie pueda leerlas (7.4).
Salt	Un valor aleatorio que se mezcla con la contraseña antes del hash, para que dos usuarios con la misma contraseña tengan hashes distintos. BCrypt lo agrega solo.
BCrypt	El algoritmo de hash elegido para las contraseñas, diseñado para ser lento a propósito y así frenar los intentos de adivinar (15.7).
Fuerza bruta	Probar contraseñas una tras otra hasta acertar. El bloqueo de 60 segundos tras cinco intentos existe para frenarla (15.9).
Inyección SQL	Un ataque que consiste en escribir código SQL dentro de un campo para alterar una consulta. Se evita usando **parámetros** (13.5 y 17.1).
Token	Una clave secreta que autoriza el uso de un servicio. El servicio de correo usa uno, y no debe viajar con el código fuente (15.12).

18.2 Base de datos

Término	Qué significa
Tabla, fila, columna	La tabla es el conjunto de datos de una cosa (`Empleados`); cada fila es un registro (un empleado); cada columna es un dato (el nombre, el teléfono).
Llave primaria (`PRIMARY KEY`)	La columna que identifica de forma única cada fila, por ejemplo `IdEmpleado`.

Término	Qué significa
Llave foránea (`FOREIGN KEY`)	Una columna que apunta a la llave primaria de otra tabla (`IdCargo` apunta a `Cargos`). Impide valores que no existen.
Restricción (`CONSTRAINT`)	Una regla que la tabla hace cumplir sola: llave primaria, llave foránea, `UNIQUE` (sin repetidos), `CHECK` (una condición) y `DEFAULT` (valor por defecto). Guía de base de datos, secciones 3 y 16.
Índice	Una estructura que acelera las búsquedas, como el índice de un libro. Un índice único, además, impide repetidos.
`NULL`	«No hay dato». No es cero ni texto vacío. Se compara con `IS NULL`, nunca con `=` (12.9).
Consulta (*query*)	Una instrucción SQL que pide datos, normalmente un `SELECT`.
`JOIN`	Unir tablas en una consulta. `INNER JOIN` solo trae lo que tiene pareja; `LEFT JOIN` trae todo lo de la izquierda aunque no tenga pareja (13.3).
Agregado	Una función que resume muchas filas en un valor: `COUNT`, `SUM`, `MAX` (13.8).
Parámetro	Un hueco con nombre (`@id`) en una consulta, que se rellena con un valor aparte. Es lo que protege contra la inyección (13.5).
Vista	Una consulta guardada con nombre que se usa como si fuera una tabla (`vw_Prestamos`). Guía de base de datos, sección 4.
Función	Un cálculo guardado en la base que devuelve un valor, como `fn_DiasHabiles`.
Procedimiento almacenado	Un programa guardado en la base, que se llama por nombre (`sp_RegistrarPrestamo`) y contiene reglas y operaciones. Guía de base de datos, sección 5.
Transacción	Un grupo de operaciones que se confirman todas (`COMMIT`) o se deshacen todas (`ROLLBACK`) (14.4).

Término	Qué significa
`THROW`	La instrucción que lanza un error personalizado con número y mensaje, que C# recibe como `SqlException` (14.5).
Integridad referencial	La garantía de que ninguna fila apunta a algo que no existe. Lo logran las llaves foráneas.
Baja lógica	Marcar un registro como inactivo en lugar de borrarlo, para conservar su historial (14.9).
Idempotencia	Que repetir una operación da el mismo resultado: ejecutar el script otra vez no rompe nada.
Lote (*batch*) y `GO`	SQL Server recibe el script en pedazos llamados lotes; `GO` es la marca que los separa. Guía de base de datos, sección 16.

18.3 C# y Windows Forms

Término	Qué significa
Clase y objeto	La clase es el molde (`ItemMenuLateral`); el objeto es una pieza concreta hecha con ese molde.
`static`	Que pertenece a la clase y no a un objeto: se usa sin `new` , como `Datos.Tabla(...)` (12.1).
Propiedad	Un campo con código de lectura y escritura. En el diseño, su `set` suele llamar a `Invalidate()` (12.2).
Evento y manejador	El evento es un aviso (`Click`); el manejador es el método que reacciona. Se conectan con `+=` (12.6).
Delegado y *lambda*	Un delegado es una variable que guarda una función (`Func<Form>`); una lambda es una función escrita en línea con `=>` .
Formulario y control	El formulario es una ventana; un control es cada pieza dentro de ella (botón, etiqueta, tabla).

Término	Qué significa
`.Designer.cs` y **`partial`**	El archivo generado por Visual Studio con lo que arrastras en el diseñador; `partial` permite que una clase esté repartida en varios archivos (12.3).
`using` y **`Dispose`**	`using` libera un recurso (conexión, lápiz, formulario) al salir de su bloque, pase lo que pase (12.7).
Excepción	Un error en tiempo de ejecución que se captura con `try/catch` (12.8).
ADO.NET	El conjunto de clases de .NET para hablar con una base de datos: `SqlConnection`, `SqlCommand`, `SqlDataAdapter`.
Pool de conexiones	Una reserva de conexiones abiertas que se reutilizan, para que abrir y cerrar sea barato (14.3).
`DataTable` y **`DataView`**	La tabla en memoria con el resultado de una consulta, y la ventana con filtro y orden sobre ella (13.7).
Hilo de la interfaz	El único hilo que dibuja la ventana y atiende clics; no hay que bloquearlo con tareas lentas (12.11).
`async` / **`await`**	La forma de esperar una tarea lenta (como enviar un correo) sin congelar la ventana.
GDI+	La librería de dibujo de Windows que usa `e.Graphics`: líneas, formas, colores y texto (16.1).
Doble búfer	Dibujar primero en una imagen oculta y mostrarla de una vez, para evitar parpadeo (16.7).
P/Invoke	Llamar desde C# a funciones nativas de Windows, como `SetWindowTheme` (16.10).
Debug y Release	Las dos formas de compilar: Debug para desarrollar (aquí se salta el login) y Release para entregar (15.6).
Code-behind	El estilo de escribir la lógica de una pantalla en el propio archivo del formulario (17.5).

Término	Qué significa
Bandera (*flag*)	Una variable `bool` que recuerda un estado (`cargando`, `enviando`) para que los eventos no se disparen en el momento equivocado (12.10).